

AI Engineering

Un viaje incremental donde cada módulo añade una capa de complejidad y herramientas profesionales a tu sistema de IA, terminando en una arquitectura de producción.

● Nivel intermedio

● 30 horas

● 8 módulos

● 43 unidades

PARA QUIÉN ES ESTE CURSO

AI Developer

CONTENIDO

01 La interfaz base: conexión y abstracción de LLMs

6 unidades · Construir la capa de abstracción base para interactuar con LLMs utilizando Python 3.12, Asyncio y SDKs oficiales.

02 Encadenamiento lógico: orquestación con LangChain

6 unidades · Diseñar y ejecutar cadenas lógicas de procesamiento utilizando LangChain y LCEL para flujos de trabajo predecibles.

03 Persistencia de datos y vector DBs

6 unidades · Implementar un sistema de memoria de largo plazo mediante arquitecturas RAG básicas y bases de datos vectoriales locales.

04 Escalabilidad documental: RAG avanzado y pinecone

6 unidades · Optimizar la escalabilidad y precisión de sistemas de recuperación utilizando Pinecone y técnicas avanzadas de procesamiento documental.

05 Razonamiento autónomo: introducción de agentes con LangGraph

6 unidades · Desarrollar agentes capaces de razonar cíclicamente y utilizar herramientas externas mediante el paradigma de grafos de LangGraph.

06 Sistemas multi-agente: colaboración y especialización

7 unidades · Diseñar sistemas complejos donde múltiples agentes autónomos colaboran y se especializan para resolver tareas de alto nivel.

07 Producción y robustez: observabilidad, costos y despliegue

5 unidades · Asegurar la robustez del sistema mediante observabilidad detallada, control de costos y despliegue asíncrono bajo estándares de ingeniería.

08 Capstone: entrega final

1 unidades · Consolidar todas las capas del sistema inteligente en un entregable final profesional y listo para producción.

MÓDULO 1

La interfaz base: conexión y abstracción de LLMs

Construir la capa de abstracción base para interactuar con LLMs utilizando Python 3.12, Asyncio y SDKs oficiales.

• Lectura

Arquitectura asíncrona con Python 3.12

Aprende a implementar patrones de concurrencia asíncrona utilizando las últimas APIs de Python 3.12 para optimizar la interacción con LLMs y construir sistemas de IA escalables.

Arquitectura asíncrona con Python 3.12

Bienvenido a la primera unidad del programa AI Engineering. Si estás aquí, es porque ya no te conformas con escribir scripts lineales que llaman a un LLM; estás listo para construir sistemas de inteligencia artificial que soporten cientos de usuarios, gestionen múltiples flujos de datos en paralelo y ofrezcan una experiencia de usuario fluida mediante streaming.

En el mundo de la IA, el tiempo no se gasta "calculando" (eso ocurre en los servidores de OpenAI o Anthropic), sino **esperando**. Esperando a que el prompt llegue, esperando a que el modelo procese y esperando a que los tokens regresen por la red. La arquitectura asíncrona es la herramienta de ingeniería que convierte esa espera pasiva en una oportunidad de ejecución.

1. El Porqué del Asyncio en la Ingeniería de IA

Imagina que estás construyendo un agente de IA que debe:

1. Buscar información en una base de datos vectorial (ChromaDB).
2. Consultar una API externa de clima.
3. Generar una respuesta usando GPT-4.

En un modelo **síncrono (bloqueante)**, tu programa se detendría por completo en el paso 1 hasta recibir respuesta, luego en el 2, y finalmente en el 3. Si cada llamada tarda 2 segundos, tu usuario final esperará al menos 6 segundos. Peor aún, mientras tu servidor "espera", no puede atender a ningún otro usuario.

Asyncio resuelve esto permitiendo que un solo proceso gestione múltiples tareas de I/O (entrada/salida) de forma concurrente. No es paralelismo real (como tener varios procesadores trabajando), sino **conurrencia cooperativa**: mientras la "Tarea A" espera datos de la red, cede el control para que la "Tarea B" avance.

Python 3.12: La madurez del modelo

Python 3.12 no solo trae mejoras de rendimiento, sino que refina la API de `asyncio` para que sea más robusta y menos propensa a errores comunes. Se han eliminado comportamientos antiguos y ambiguos (como métodos automáticos de creación de loops) para obligar a los desarrolladores a seguir patrones más limpios y predecibles.

2. Conceptos Fundamentales: La Anatomía de `asyncio`

Para dominar esta arquitectura, debemos entender sus tres pilares:

A. Corrutinas (`async def` y `await`)

Una corrutina es una función que puede pausar su ejecución. Piénsala como una **receta con marcadores de libros**. Puedes empezar a cocinar (ejecutar código), llegar a un punto donde debes esperar a que el horno se caliente (`await`), poner un marcador ahí, y mientras tanto, empezar a picar verduras para otra receta.

```
async def fetch_llm_response(prompt: str):
    # La ejecución se pausa aquí, liberando el hilo para otras tareas
    response = await client.chat.completions.create(model="gpt-4", messages=[...])
    return response
```

B. El Event Loop (El Gestor del Restaurante)

El Event Loop es el motor invisible que coordina las corrutinas. Es como un gestor de restaurante extremadamente eficiente:

- Recibe pedidos (Tareas).
- Los asigna.
- Si un camarero está esperando que la cocina entregue un plato, el gestor le ordena atender otra mesa inmediatamente.

C. Tasks (Tareas)

Una tarea es una corrutina que ha sido programada para ejecutarse "lo antes posible" en el event loop. Mientras que una corrutina por sí sola no hace nada hasta que la esperas (`await`), una `Task` permite que el código se ejecute en segundo plano mientras tú sigues con otras operaciones.

3. Implementación de Patrones de Concurrency en IA

A partir de Python 3.12, la forma de interactuar con el loop ha cambiado. Ya no deberías usar `asyncio.get_event_loop()` de forma descuidada.

El punto de entrada moderno: `asyncio.run()`

Este es el estándar de oro para iniciar tu aplicación. Gestiona la creación del loop, la finalización de las tareas y el cierre limpio de los ejecutores.

```
import asyncio

async def main():
    print("Iniciando sistema de IA...")
    # Lógica de la aplicación aquí

if __name__ == "__main__":
    asyncio.run(main())
```

Concurrencia con `asyncio.gather`

Este es el patrón más común cuando necesitas disparar múltiples llamadas a modelos de lenguaje simultáneamente. Por ejemplo, si quieres comparar las respuestas de tres modelos diferentes sobre el mismo prompt:

```
async def compare_models(prompt):
    # Programamos las tres llamadas para que ocurran al mismo tiempo
    results = await asyncio.gather(
        call_openai(prompt),
        call_anthropic(prompt),
        call_local_llama(prompt),
        return_exceptions=True # Crucial para que un fallo no tumbe todo
    )
    return results
```

Escenario Real: En una canalización de RAG (Generación Aumentada por Recuperación), podrías usar `gather` para buscar en tu base de datos vectorial y, al mismo tiempo, hacer una búsqueda en Google Search API. No hay razón para esperar a que termine una para empezar la otra.

4. Gestión de Tareas y Latencia en Producción

Como Arquitecto de IA, tu prioridad no es solo que el código funcione, sino que sea resiliente.

Timeouts: El seguro contra la latencia infinita

Los LLMs pueden volverse lentos o las APIs pueden fallar. Nunca debes permitir que una llamada a un LLM quede abierta indefinidamente.

```
try:
    async with asyncio.timeout(10.0): # Nuevo en Python 3.11+, estándar en 3.12
        response = await call_llm(prompt)
except TimeoutError:
    print("El modelo tardó demasiado. Aplicando estrategia de fallback...")
```

Semáforos: Control de flujo y Rate Limits

Si intentas disparar 1,000 llamadas a OpenAI en un segundo usando `asyncio.gather`, recibirás un error de `429: Too Many Requests`. Los `asyncio.Semaphore` te permiten limitar cuántas tareas concurrentes pueden estar activas.

```
sem = asyncio.Semaphore(5) # Solo 5 llamadas concurrentes permitidas

async def throttled_call(prompt):
    async with sem:
        return await call_llm(prompt)
```

5. Errores Comunes y Malentendidos (Anti-patrones)

Incluso desarrolladores experimentados cometen estos errores al mover sus sistemas de IA a arquitecturas asíncronas:

"Async hace que el procesamiento de datos sea más rápido"

FALSO. Asyncio no acelera el cálculo matemático. Si estás haciendo limpieza de datos pesada con Pandas o corriendo una inferencia de PyTorch localmente en la CPU, `asyncio` no ayudará. Para tareas **CPU-bound**, necesitas `multiprocessing`. Asyncio es para tareas **I/O-bound** (red, bases de datos, APIs).

El "Bloqueo del Loop"

Este es el error más peligroso. Si dentro de una función `async` llamas a una función síncrona que tarda mucho (como `time.sleep(5)` en lugar de `await asyncio.sleep(5)`), **detendrás a todos tus usuarios**. El gestor del restaurante se queda congelado mirando una mosca, y nadie más recibe su comida.

Solución en Python 3.12: Si debes usar una librería antigua que no es asíncrona, usa `asyncio.to_thread()`:

```
# Esto ejecuta la función pesada en un hilo separado sin bloquear el loop principal
result = await asyncio.to_thread(blocking_sync_function, data)
```

6. Aplicaciones en la Industria: El caso del Streaming de Tokens

Una de las aplicaciones más críticas del asincronismo en IA es el **Streaming**. Los usuarios de ChatGPT esperan ver las palabras aparecer una a una. Esto se logra mediante generadores asíncronos.

En un entorno síncrono, el servidor tendría que esperar a que se generen los 500 tokens de una respuesta para enviárselos al cliente. Con `asyncio` y el SDK de OpenAI (que usa `httpx` por debajo de forma asíncrona), podemos procesar cada token según llega de la red y enviarlo al frontend mediante WebSockets o Server-Sent Events (SSE).

Esto reduce el **Time to First Token (TTFT)**, una de las métricas de UX más importantes en productos de IA actuales.

7. Ejemplo resuelto paso a paso: batch de embeddings con control de concurrencia

Hasta acá viste las piezas por separado (`gather` , `Semaphore` , `timeout`). Ahora las juntamos en un caso real y **completamente resuelto**: procesar una lista de textos generando sus embeddings de forma concurrente, pero respetando un límite de llamadas simultáneas, con timeout por llamada y sin que un fallo tumbe todo el lote.

El código corre tal cual (no necesita API keys: simulamos la latencia de red).

```
import asyncio
import random
import time

# --- 1. Simulamos una llamada a la API de embeddings (tarea I/O-bound) ---
async def get_embedding(text: str, sem: asyncio.Semaphore) -> dict:
    async with sem:
        # respeta el límite de concurrencia
        latency = random.uniform(0.5, 2.5)
        # latencia de red simulada
        try:
            async with asyncio.timeout(2.0):
                # corta si tarda más de 2s
                await asyncio.sleep(latency)
                # await, NO time.sleep (bloquearía el loop)
                return {"text": text, "dim": 1536, "ok": True}
        except TimeoutError:
            return {"text": text, "ok": False, "error": "timeout"}

# --- 2. Orquestador: dispara todas las tareas concurrentemente ---
async def embed_batch(texts: list[str]) -> list[dict]:
```

```

sem = asyncio.Semaphore(3)                # máximo 3 llamadas en paralelo
tasks = [get_embedding(t, sem) for t in texts]
return await asyncio.gather(*tasks, return_exceptions=True)

# --- 3. Punto de entrada ---
async def main():
    textos = [f"documento_{i}" for i in range(8)]
    inicio = time.perf_counter()
    resultados = await embed_batch(textos)
    elapsed = time.perf_counter() - inicio

    ok = [r for r in resultados if isinstance(r, dict) and r.get("ok")]
    fallidos = [r for r in resultados if isinstance(r, dict) and not r.get("ok")]

    print(f"Procesados OK: {len(ok)}/{len(textos)} en {elapsed:.2f}s")
    for f in fallidos:
        print(f" Falló: {f['text']} -> {f['error']}")

if __name__ == "__main__":
    asyncio.run(main())

```

Salida esperada (aproximada, varía por la latencia aleatoria):

Procesados OK: 6/8 en 3.42s Falló: documento_2 -> timeout Falló: documento_5 -> timeout

Qué pasó, paso a paso:

1. **asyncio.Semaphore(3)** actúa como un "portero": aunque creamos 8 tareas, solo 3 pueden estar dentro del bloque `async with sem` a la vez. Las otras esperan su turno. Por eso 8 textos se procesan en ~3 "tandas", no todos de golpe.
2. **asyncio.gather(*tasks, return_exceptions=True)** dispara las 8 tareas y espera a todas. El `return_exceptions=True` es clave: si una falla, el lote no se cae; el error vuelve como un resultado más.
3. **asyncio.timeout(2.0)** envuelve cada llamada: si la latencia simulada supera los 2 segundos, se lanza `TimeoutError`, lo capturamos y devolvemos un resultado marcado como fallido en vez de romper el programa.
4. **await asyncio.sleep(latency)** simula la espera de red sin bloquear el event loop. Si hubiéramos usado `time.sleep`, habríamos congelado a todos los usuarios (el anti-patrón de la sección 5).

Conexión con la pregunta de reflexión: este es exactamente el patrón que usarías para embeddear 10.000 filas. Nunca harías `gather` de 10.000 llamadas a la vez (te ganarías un **429: Too Many Requests**): las encolás todas, pero el semáforo asegura que solo N corran en simultáneo.

8. Síntesis y Mejores Prácticas

Para cerrar esta unidad, recuerda estas reglas de oro para tu arquitectura en Python 3.12:

1. Usa `asyncio.run()` como único punto de entrada para tus scripts.
2. Prefiere `asyncio.get_running_loop()` sobre la versión antigua si necesitas acceder al loop desde dentro de una función.
3. Nunca bloques el loop. Identifica funciones síncronas de I/O y envuélvelas en `to_thread()`.
4. Implementa Timeouts y Semáforos desde el día 1. Un sistema de IA sin control de flujo es una bomba de tiempo hacia el agotamiento de cuotas y latencias extremas.
5. Aprovecha `asyncio.TaskGroup` (introducido en 3.11 y refinado en 3.12) para gestionar grupos de tareas de forma más segura que `gather`, asegurando que si una falla, las demás se cancelen limpiamente.

En la siguiente unidad, daremos el paso lógico: utilizar estos cimientos asíncronos para construir **clientes de LLM robustos**, capaces de alternar entre proveedores como OpenAI y Anthropic sin cambiar una sola línea de lógica de negocio.

Pregunta para reflexionar: Si tuvieras que procesar un dataset de 10,000 filas haciendo un embedding de cada una, ¿usarías `asyncio.gather` con las 10,000 llamadas a la vez? ¿Por qué? (Pista: Revisa la sección de Semáforos).

Práctica

Ejercicio: Orquestador Concurrent de Modelos

En este ejercicio, simularás la lógica de un backend que debe consultar múltiples fuentes de IA de forma eficiente.

Pasos:

1. **Configuración Inicial:** Crea un entorno con Python 3.12+.
2. **Simulación de Llamadas:** Define tres corrutinas `async def` que simulen llamadas a diferentes modelos (por ejemplo, `gpt_4_call`, `claude_3_call`, `local_llama_call`). Cada una debe usar `await asyncio.sleep()` con diferentes tiempos de espera para simular latencia de red.
3. **Orquestación:** Implementa una función principal que dispare las tres llamadas simultáneamente usando `asyncio.gather`.
4. **Resiliencia:** Añade un `asyncio.timeout` de 2 segundos a la ejecución total. Si alguna llamada tarda más, el programa debe capturar la excepción y mostrar un mensaje de error sin detenerse.
5. **Control de Flujo:** Utiliza un `asyncio.Semaphore` para asegurar que, aunque intentes disparar 10 simulaciones de llamadas, solo 2 se ejecuten al mismo tiempo.

Criterios de Aceptación:

- El código no debe usar `time.sleep` (bloqueante).
- Se debe observar en los logs que las tareas inician casi al mismo tiempo.
- Se debe manejar correctamente la excepción `TimeoutError`.

Entregable

Tipo: FILE

Entregá la actividad en PDF.

Arquitectura Asíncrona con Python 3.12

Aprende a implementar patrones de concurrencia asíncrona utilizando las últimas APIs de Python 3.12 para optimizar la interacción con LLMs y construir sistemas de IA escalables.

Entregable

Tipo: FILE

Entregá la actividad en PDF.

Instrucciones

Ejercicio: Orquestador Concurrent de Modelos

En este ejercicio, simularás la lógica de un backend que debe consultar múltiples fuentes de IA de forma eficiente.

Pasos:

- 1. Configuración Inicial:** Crea un entorno con Python 3.12+.
- 2. Simulación de Llamadas:** Define tres corrutinas `async def` que simulen llamadas a diferentes modelos (por ejemplo, `gpt_4_call`, `claude_3_call`, `local_llama_call`). Cada una debe usar `await asyncio.sleep()` con diferentes tiempos de espera para simular latencia de red.
- 3. Orquestación:** Implementa una función principal que dispare las tres llamadas simultáneamente usando `asyncio.gather`.
- 4. Resiliencia:** Añade un `asyncio.timeout` de 2 segundos a la ejecución total. Si alguna llamada tarda más, el programa debe capturar la excepción y mostrar un mensaje de error sin detenerse.
- 5. Control de Flujo:** Utiliza un `asyncio.Semaphore` para asegurar que, aunque intentes disparar 10 simulaciones de llamadas, solo 2 se ejecuten al mismo tiempo.

Criterios de Aceptación:

- El código no debe usar `time.sleep` (bloqueante).
- Se debe observar en los logs que las tareas inician casi al mismo tiempo.
- Se debe manejar correctamente la excepción `TimeoutError`.

Objetivos de aprendizaje

- Diferenciar entre tareas I/O-bound y CPU-bound en el contexto de aplicaciones de IA.
- Implementar llamadas concurrentes a múltiples APIs de LLM utilizando `asyncio.gather` y `TaskGroups`.
- Gestionar la latencia y los límites de tasa (rate limits) mediante timeouts y semáforos asíncronos.
- Evitar errores comunes de bloqueo del event loop en Python 3.12.

Rúbrica: Orquestador Concurrente de Modelos LLM

Evaluación técnica sobre la implementación de arquitecturas asíncronas en Python 3.12 para la gestión eficiente de múltiples modelos de IA, enfocándose en la no bloqueidad, control de concurrencia y resiliencia mediante timeouts.

Criterio	Descripción	Peso
Implementación de Asincronía y No Bloqueo	Uso correcto de <code>async/await</code> y evitación de funciones bloqueantes como <code>time.sleep</code> .	30%
Gestión de Concurrencia con <code>asyncio.gather</code>	Capacidad para orquestar llamadas simultáneas y procesar resultados de forma colectiva.	25%
Control de Flujo con Semáforos	Uso de <code>asyncio.Semaphore</code> para limitar el consumo de recursos y prevenir la saturación del sistema.	25%
Resiliencia y Manejo de Timeouts	Implementación de límites de tiempo y captura de excepciones para garantizar la estabilidad del servicio.	20%

Total: 100 pts | Aprobación: 60 pts

Abstracción de clientes y configuración robusta

Aprende a estructurar la capa de conexión con LLMs utilizando patrones de diseño, validación de tipos con Pydantic y gestión segura de secretos en Python 3.12.

Abstracción de clientes y configuración robusta

En la arquitectura de sistemas de IA, la **capa de conexión** es el cimiento de todo el flujo. Si esta capa es frágil, tu sistema sufrirá de tiempos de inactividad, fugas de secretos y dificultades para escalar o cambiar de proveedor.

¿Por qué abstraer los clientes?

Muchos desarrolladores cometen el error de instanciar clientes de OpenAI o Anthropic directamente en su lógica de negocio. Esto genera tres problemas graves:

1. **Acoplamiento:** Cambiar de GPT-4o a Claude 3.5 requiere reescribir docenas de líneas de código.
2. **Dificultad de Testing:** Es complejo simular (mockear) llamadas a la API si el cliente está "enterrado" en la lógica.
3. **Inconsistencia:** Cada SDK tiene firmas diferentes (`choices[0].message` vs `content[0].text`).

Patrón Factory y Validación con Pydantic

Para solucionar esto, utilizamos un **Factory Pattern**. Este patrón permite que el sistema decida qué cliente usar en tiempo de ejecución basándose en la configuración.

Además, en Python 3.12, la validación de tipos con **Pydantic** se vuelve obligatoria para manejar secretos de forma segura con `SecretStr`, evitando que tus llaves de API aparezcan accidentalmente en los logs de error.

Diferencias Clave que debes abstraer

- **OpenAI:** Estructura de mensajes basada en `role` y `content`. Respuesta en `choices`.
- **Anthropic:** Estructura similar pero con diferencias en la extracción del texto (`content` es una lista de objetos).

A continuación, implementaremos una base sólida que maneje estas diferencias de forma transparente para el resto de tu aplicación.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import asyncio
from abc import ABC, abstractmethod
from enum import Enum
from pydantic import BaseModel, SecretStr
# TODO: Importa los clientes asíncronos de openai y anthropic

class Provider(str, Enum):
    OPENAI = "openai"
    ANTHROPIC = "anthropic"

class LLMConfig(BaseModel):
    """
    TODO: Define los campos para las API Keys usando SecretStr
    y una validación para asegurar que existan cuando se necesiten.
    """
    pass

class BaseLLMClient(ABC):
    @abstractmethod
    async def chat(self, prompt: str) -> str:
        pass

# TODO: Implementa OpenAIClient y AnthropicClient heredando de BaseLLMClient

class LLMFactory:
    @staticmethod
    def create_client(provider: Provider, config: LLMConfig) -> BaseLLMClient:
        # TODO: Implementa la lógica de selección de cliente
        pass

async def main():
    # TODO: Instancia la configuración y crea un cliente usando el Factory
    pass

if __name__ == "__main__":
    asyncio.run(main())
```

Código solución

```
import os
import asyncio
from abc import ABC, abstractmethod
from enum import Enum
from typing import Any, Dict, Optional
from pydantic import BaseModel, Field, SecretStr
from openai import AsyncOpenAI
from anthropic import AsyncAnthropic

class Provider(str, Enum):
    OPENAI = "openai"
    ANTHROPIC = "anthropic"

class LLMConfig(BaseModel):
    """Configuración validada para los clientes de LLM."""
    openai_api_key: Optional[SecretStr] = Field(default=None)
    anthropic_api_key: Optional[SecretStr] = Field(default=None)
    default_model: str = "gpt-4o"

class BaseLLMClient(ABC):
    """Interfaz base para clientes de LLM asíncronos."""
    @abstractmethod
    async def chat(self, prompt: str) -> str:
        pass

class OpenAIClient(BaseLLMClient):
    def __init__(self, api_key: str, model: str):
        self.client = AsyncOpenAI(api_key=api_key)
        self.model = model

    async def chat(self, prompt: str) -> str:
        response = await self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "user", "content": prompt}]
        )
        return response.choices[0].message.content

class AnthropicClient(BaseLLMClient):
    def __init__(self, api_key: str, model: str):
        self.client = AsyncAnthropic(api_key=api_key)
        self.model = model

    async def chat(self, prompt: str) -> str:
```

```

        response = await self.client.messages.create(
            model=self.model,
            max_tokens=1024,
            messages=[{"role": "user", "content": prompt}]
        )
        return response.content[0].text

class LLMFactory:
    """Factory para instanciar clientes basados en configuración profesional."""
    @staticmethod
    def create_client(provider: Provider, config: LLMConfig) -> BaseLLMClient:
        if provider == Provider.OPENAI:
            if not config.openai_api_key:
                raise ValueError("OpenAI API Key no configurada")
            return OpenAIClient(api_key=config.openai_api_key.get_secret_value(),
                                model=config.default_model)

            elif provider == Provider.ANTHROPIC:
                if not config.anthropic_api_key:
                    raise ValueError("Anthropic API Key no configurada")
                return AnthropicClient(api_key=config.anthropic_api_key.get_secret_value(),
                                       model="claude-3-5-sonnet-20240620")

                raise ValueError(f"Proveedor {provider} no soportado")

async def main():
    # Simulación de entorno
    config = LLMConfig(
        openai_api_key=SecretStr("sk-dummy-openai"),
        anthropic_api_key=SecretStr("sk-ant-dummy")
    )

    # Uso del Factory
    try:
        openai_client = LLMFactory.create_client(Provider.OPENAI, config)
        print(f"Cliente {Provider.OPENAI} creado con éxito.")

        # En una app real:
        # response = await openai_client.chat("Hola, ¿cómo estás?")
    except Exception as e:
        print(f"Error: {e}")

if __name__ == "__main__":

```

```
asyncio.run(main())
```

Pistas

- Utiliza la librería `pydantic-settings` o `pydantic` para validar que las API Keys existan. Esto evita errores en tiempo de ejecución cuando el LLM ya está procesando datos.
- Para que la clase sea extensible, define un método abstracto o una interfaz que obligue a implementar un método `generate` asíncrono. Recuerda que en Python 3.12 puedes usar `typing.Self` para retornos de métodos de clase.
- El cliente de Anthropic y el de OpenAI tienen constructores distintos (`AsyncOpenAI` vs `AsyncAnthropic`). Tu Factory debe instanciar el correcto basándose en un Enum de proveedores.

Rubrica: Abstracción de Clientes y Configuración Robusta

Evaluación para el diseño de una capa de conexión de LLMs utilizando patrones de diseño, validación con Pydantic y manejo asíncrono en Python 3.12.

Criterio	Descripción	Peso
Implementación del Factory Pattern y Abstracción	Capacidad para desacoplar la lógica de negocio de los SDKs específicos mediante una interfaz común y una fábrica de clientes.	35%
Gestión de Configuración y Seguridad con Pydantic	Uso de Pydantic y SecretStr para la validación de entornos y protección de API Keys.	25%
Correctitud Asíncrona (Asyncio) y Python 3.12	Uso correcto de la programación asíncrona y características modernas de Python.	20%
Normalización de Diferencias de Proveedores	Capacidad para unificar las respuestas y solicitudes entre OpenAI y Anthropic.	20%

Total: 100 pts | Aprobación: 65 pts

• Autoevaluación

Evaluación: estados, turnos y latencia de LLMs

Valida tus conocimientos sobre gestión de estados conversacionales, métricas de rendimiento (TTFT) y

estrategias de streaming para sistemas de IA escalables.

Evaluación: estados, turnos y latencia de LLMs

¡Bienvenido a la evaluación de la unidad! En las lecciones anteriores, exploramos cómo construir la capa de infraestructura que conecta nuestras aplicaciones con los modelos de lenguaje. Aprendimos que un sistema de IA robusto no es solo un "wrapper" de una API, sino un motor asíncrono que debe gestionar el estado conversacional y optimizar la experiencia del usuario a través del control de la latencia.

¿Qué evaluaremos?

En este cuestionario, pondremos a prueba tu comprensión sobre:

- **Gestión de Estados:** La diferencia entre el estado mutable e inmutable, y cómo escalar la memoria de la conversación fuera del proceso.
- **Ciclos de Turnos:** Cómo estructurar la interacción usuario-sistema siguiendo principios de ingeniería.
- **Métricas de Latencia:** Conceptos críticos como el **Time to First Token (TTFT)** y cómo el streaming impacta la percepción del usuario.
- **Optimización:** Estrategias para reducir la carga cognitiva y técnica mediante el manejo eficiente de tokens.

Recuerda que en el nivel intermedio, no solo buscamos que el código funcione, sino que sea **escalable, predecible y eficiente**. ¡Mucho éxito!

Quiz

Aprobación: 70 · Intentos máximos: 3

1. Estás diseñando un orquestador que maneja miles de usuarios concurrentes. ¿Por qué es una mala práctica mantener la 'base de conversación' (historial de mensajes) únicamente en una variable global de Python?

☐ No hay problema, Python maneja la visibilidad global de forma inmutable por defecto.

☒ **Impide el escalado horizontal, ya que cada instancia del servidor tendría un estado de conversación diferente.**

☐ Provoca un bloqueo del Event Loop de Asyncio al intentar acceder a memoria compartida.

Explicación: El concepto clave aquí es la **escalabilidad horizontal**.

- **La opción correcta** señala que guardar el historial en una lista global (`global messages = []`) impide que las peticiones se distribuyan entre múltiples contenedores o procesos, ya que cada uno tendría una "memoria" distinta y fragmentada. Para escalar, el estado debe ser externo (ej. Redis o una DB).
- El error de "bloqueo de hilos" (opción 3) es más común por usar código síncrono en `asyncio` , pero no es la razón principal por la cual las variables globales fallan en el escalado de memoria.
- Pensar que las variables globales son "inmutables" (opción 1) es un error conceptual básico; en Python son mutables, pero su alcance es el problema.

2. Un cliente se queja de que su chatbot 'tarda mucho en responder'. Al analizar las métricas, notas que el modelo tarda 5 segundos en generar la respuesta completa. ¿Qué métrica deberías optimizar para mejorar la *percepción* de velocidad sin cambiar necesariamente el modelo?

☐ Throughput (tokens/segundo) del modelo.

☒ **TTFT (Time to First Token) mediante streaming.**

☐ RTT (Round Trip Time) de la conexión TCP/IP.

Explicación: El **TTFT (Time to First Token)** es la métrica de 'oro' para la experiencia de usuario en chat.

- **TTFT** mide cuánto tarda el usuario en ver *algo* en pantalla. Aunque el modelo tarde 10 segundos en terminar (latencia total), si el primer token aparece en 200ms, la percepción es de inmediatez.
- El **Throughput** (opción 1) mide volumen (tokens por segundo), pero no indica cuándo empezó la respuesta.
- El **Round Trip Time (RTT)** (opción 3) es una métrica de red pura que no incluye el tiempo de inferencia del LLM.

3. ¿En qué escenarios el uso de streaming de respuestas podría ser contraproducente o difícil de implementar? (Selecciona todas las que apliquen)

☒ **Cuando se requiere una respuesta en formato JSON estructurado para ser procesado por otra función.**

☒ **Cuando se deben aplicar validaciones de seguridad (guardrails) sobre la respuesta completa antes de mostrarla.**

☐ Cuando el sistema utiliza Python 3.12 y Asyncio, ya que estas tecnologías no soportan paquetes parciales.

Explicación: El streaming es una técnica de optimización de UX, pero tiene limitaciones técnicas claras:

1. **Estructura de Datos (JSON):** Los LLMs generan texto secuencial. Si pides un JSON y lo streameas, el receptor recibirá `{"nombre": "An` , lo cual no es un JSON válido hasta que se cierre el string y el objeto. Esto

rompe los parsers tradicionales.

2. Post-procesamiento: Si necesitas analizar la respuesta con otro modelo (ej. para moderación de seguridad severa) antes de que el usuario la vea, no puedes streamear en tiempo real porque necesitas el contenido completo para validarlo.

3. El uso de `asyncio` (distractor) es de hecho un facilitador del streaming, no un impedimento.

4. Tu asistente de viajes necesita recordar el destino y la fecha elegida por el usuario durante 10 turnos de conversación. ¿Cuál es la estrategia más robusta para manejar este estado?

☐ Confiar en que el LLM siempre podrá extraer la información previa del historial completo enviado en cada prompt.

☒ **Implementar una gestión de 'slots' o variables de contexto persistentes independientes del historial de mensajes.**

☐ Reiniciar el estado de la conversación cada vez que el usuario cambia de tema.

Explicación: En un nivel intermedio, debemos distinguir entre **qué recordó el usuario** y **qué necesita el sistema**.

- **El historial de mensajes** es el registro crudo (short-term memory).
- Los **Slots** (como 'fecha_vuelo', 'destino') son el estado estructurado derivado de la conversación.
- Almacenar slots permite que, aunque el historial se borre o se resuma, los datos críticos para la lógica de negocio (hacer una reserva) persistan de forma clara y procesable.
- Pensar que el LLM extraerá siempre los datos del historial crudo (opción 1) es arriesgado por el límite de tokens y posibles alucinaciones.

5. Si notas que el TTFT aumenta linealmente a medida que la conversación avanza, ¿cuál es la causa más probable y su solución?

☐ Incrementar el valor de 'max_tokens' para permitir ráfagas más rápidas de información.

☒ **Reducir el tamaño del System Prompt y aplicar una ventana deslizante (sliding window) al historial de mensajes.**

☐ Desactivar el modo asíncrono para evitar la sobrecarga de gestión de tareas de Python.

Explicación: Un error común en principiantes es pensar que la latencia es solo culpa de "la API lenta".

- **El contexto (System Prompt + Historial)** consume procesamiento. A más tokens de entrada, más tiempo tarda el modelo en procesar el "prompt" antes de emitir el primer token de salida (Prefill phase).
- Optimizar la latencia implica, frecuentemente, **podar el historial** o usar modelos más pequeños para tareas de baja complejidad.

- Aumentar el `max_tokens` (opción 1) solo afecta el límite de salida, no la velocidad de inicio.
- Desactivar Asyncio (opción 3) empeoraría el rendimiento general del servidor bajo carga.

• Pre-entrega

Pre-entrega 1: Cliente de LLM robusto y asíncrono

Guía detallada para la construcción de una capa de abstracción asíncrona para LLMs empleando estándares de ingeniería de software para IA.

Pre-entrega 1: Cliente de LLM robusto y asíncrono

Qué construir

Debes entregar un **repositorio de código** (o un archivo Python estructurado) que contenga la implementación de un **"Unified Async LLM Client"**.

Este artefacto debe ser una clase (o conjunto de clases) en Python 3.12 que permita:

1. **Intercambiabilidad:** Poder instanciar un proveedor (OpenAI o Anthropic) bajo una interfaz común.
2. **Asincronía:** Todas las llamadas a modelos deben ser no bloqueantes (`async/await`).
3. **Streaming:** Implementar un método que devuelva un generador asíncrono de tokens.
4. **Validación:** Uso de Pydantic para definir la estructura de los mensajes de entrada y la configuración del modelo.

Pasos Sugeridos

1. **Estructura de Datos:** Crea un archivo `schemas.py` con Pydantic para definir `ChatMessage` (role, content) y `ModelResponse` . Implementar esto primero evita el "error de diccionarios anidados" tan común en principiantes.
2. **La Base Asíncrona:** Define una clase base abstracta `BaseLLMClient` con un método `async def generate()` .
3. **Implementación del Proveedor:** Crea `OpenAIClient` y `AnthropicClient` heredando de la base. Usa los SDKs oficiales (`openai` y `anthropic`) en sus versiones asíncronas (`AsyncOpenAI` y `AsyncAnthropic`).
4. **Lógica de Streaming:** Implementa el generador usando `yield` dentro de un loop `async for` que recorra el stream del SDK.
5. **** script de validación**:** Crea un archivo `main.py` simple que importe tus clientes, cargue el `.env` y realice una pregunta corta ("¿Qué es la entropía?") tanto en modo normal como en streaming.

Errores Comunes a Evitar

- **Bloqueo del Event Loop:** Un error clásico de nivel intermedio es usar la versión síncrona del cliente dentro de una función `async` . Esto detiene todo el programa mientras el LLM "piensa". Asegúrate de usar `await`

```
client.chat.completions.create(...)
```

- **Fuga de Excepciones:** No dejes que un error de "API Key inválida" o "Límite de cuota" rompa el loop principal. Captura la excepción y devuelve un mensaje estructurado o haz un reintento (retry).
-

Práctica

1. Configura un entorno virtual con Python 3.12 e instala `openai`, `anthropic`, `pydantic` y `python-dotenv`.
2. Crea un esquema de Pydantic para validar los parámetros de entrada del modelo (temperatura de 0 a 2, max_tokens, etc.).
3. Implementa una clase `AsyncLLMManager` que pueda cargar tanto OpenAI como Anthropic basándose en una variable de configuración.
4. El método de generación debe ser capaz de manejar streaming: usa `yield` para retornar fragmentos de texto conforme lleguen de la API.
5. Asegúrate de capturar excepciones de red y de límite de tasa (rate limiting) devolviendo un error controlado en lugar de un crash.
6. Incluye un archivo `README.md` que explique cómo ejecutar el script de prueba y qué variables de entorno son necesarias.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga la implementación de los clientes asíncronos, archivos de configuración (schemas.py), variables de entorno (.env.example) y un script de prueba de streaming.

Pre-entrega: Cliente de LLM Robusto y Asíncrono

Guía detallada para la construcción de una capa de abstracción asíncrona para LLMs empleando estándares de ingeniería de software para IA.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga la implementación de los clientes asíncronos, archivos de configuración (schemas.py), variables de entorno (.env.example) y un script de prueba de streaming.

Instrucciones

1. Configura un entorno virtual con Python 3.12 e instala `openai`, `anthropic`, `pydantic` y `python-dotenv`.

- 2. Crea un esquema de Pydantic para validar los parámetros de entrada del modelo (temperatura de 0 a 2, max_tokens, etc.).
- 3. Implementa una clase AsyncLLMManager que pueda cargar tanto OpenAI como Anthropic basándose en una variable de configuración.
- 4. El método de generación debe ser capaz de manejar streaming: usa yield para retornar fragmentos de texto conforme lleguen de la API.
- 5. Asegúrate de capturar excepciones de red y de límite de tasa (rate limiting) devolviendo un error controlado en lugar de un crash.
- 6. Incluye un archivo README.md que explique cómo ejecutar el script de prueba y qué variables de entorno son necesarias.

Objetivos de aprendizaje

- Construir una clase de cliente asíncrono utilizando los SDKs oficiales de OpenAI y Anthropic.
- Implementar validación de configuraciones y esquemas de mensajes mediante Pydantic.
- Gestionar el streaming de tokens mediante generadores asíncronos en Python 3.12.
- Aplicar patrones de manejo de errores para mejorar la resiliencia de las llamadas a modelos externos.

Rúbrica de Evaluación: Cliente de LLM Robusto y Asíncrono

Esta rúbrica evalúa la capacidad del estudiante para construir una capa de abstracción técnica sólida, utilizando programación asíncrona avanzada y validación de esquemas para interactuar con proveedores líderes de LLM en el marco del programa AI Architect 2026.

Criterio	Descripción	Peso
Abstracción y Gestión de Proveedores (AsyncLLMManager)	Evalúa la implementación de la clase principal para manejar dinámicamente OpenAI y Anthropic mediante configuración.	35%
Implementación de Streaming Asíncrono	Manejo de generadores asíncronos para el procesamiento de tokens en tiempo real.	25%
Validación de Esquemas con Pydantic	Uso de Pydantic para garantizar la integridad y el rango de los parámetros de entrada.	20%
Resiliencia y Manejo de Errores	Capacidad del sistema para manejar fallos de red y Rate Limiting sin interrumpir el flujo del programa.	20%

Total: 100 pts | Aprobación: 65 pts

• Clase en vivo

Sesión en vivo 1. La interfaz base: conexión y abstracción de LLMs

Sesión interactiva de 120 minutos para consolidar la creación de interfaces asíncronas y capas de abstracción para múltiples proveedores de LLMs.

Sesión en Vivo: Consolidación de Interfaces y Abstracción de LLMs

¡Bienvenidos a la sesión de cierre del módulo! En esta clase en vivo, daremos el salto de la teoría a la implementación profesional. Tras haber explorado la programación asíncrona y la configuración de clientes, hoy nos enfocaremos en construir una arquitectura que sea verdaderamente agnóstica al proveedor y escalable para entornos productivos.

¿Qué aprenderás?

En esta sesión, nos enfocaremos en la resolución de problemas complejos que surgen al conectar sistemas reales con modelos de lenguaje. Aprenderás a:

- Estructurar una capa de abstracción robusta que permita cambiar entre OpenAI, Anthropic u otros proveedores sin modificar la lógica de negocio.
- Implementar patrones de manejo de errores específicos para APIs de LLMs (Rate limits, Timeout, Context Window).
- Optimizar la latencia mediante el uso eficiente de `asyncio`.

Temas clave

1. **Refactorización hacia la Abstracción:** Cómo pasar de un script simple a una clase base abstracta.
2. **Estrategias de Retries y Fallbacks:** Qué hacer cuando la API falla o el modelo está saturado.
3. **Benchmarks en Tiempo Real:** Comparación de latencia y tiempos de respuesta (TTFT - Time To First Token).

Preparación previa

Para aprovechar al máximo esta sesión, asegúrate de:

1. Tener instalado **Python 3.12**.
2. Disponer de tus API Keys (OpenAI y/o Anthropic) configuradas en variables de entorno.
3. Haber completado la unidad de "Abstracción de clientes y configuración robusta", ya que trabajaremos sobre esa base de código.

Actividades

Realizaremos un **Live Coding Colaborativo** donde enfrentaremos desafíos de concurrencia y un **Debate de Arquitectura** sobre cuándo es mejor usar un wrapper propio versus una librería de terceros como LangChain.

Clase en vivo

Materiales de apoyo

Puntos clave:

- Abstracción (ABC): Permite definir un contrato que todos los clientes de LLM deben cumplir, facilitando el intercambio de modelos (Interoperabilidad).
- Asyncio Event Loop: Es el motor que permite manejar múltiples llamadas a APIs sin que una espere a la otra, crucial para la escalabilidad.
- Time To First Token (TTFT): Métrica clave de UX que mide cuánto tarda el modelo en empezar a responder.

Ejemplos sugeridos:

- Caso de Uso: Un chatbot de atención masiva que debe procesar 100 tickets por minuto. Sin asyncio, el servidor colapsaría por esperas de red.
- Escenario de Error: API de OpenAI caída globalmente. El sistema de fallback conmuta a Claude 3 en <200ms sin intervención humana.

Herramientas recomendadas:

- Python 3.12+
- Httpx (para peticiones asíncronas)
- Pydantic (para validación de esquemas de respuesta)
- Loguru (para observabilidad de procesos asíncronos)

Actividades

- **Live Refactoring: Del Script al Framework** (40 min, DEMO)
 - El instructor compartirá pantalla y presentará una estructura base de cliente LLM con 'deudas técnicas' (sincrónico, atado a un solo proveedor). De forma interactiva, los estudiantes sugerirán pasos para refactorizarlo usando ABC (Abstract Base Classes) y Asyncio. El instructor implementará los cambios en tiempo real, resolviendo errores de tipado y lógica que surjan.
- **Desafío de Arquitectura: Estrategias de Fallback** (30 min, GROUP_WORK)
 - En grupos pequeños, los estudiantes deben diseñar la lógica para un sistema que, si OpenAI devuelve un RateLimitError, cambie automáticamente a Anthropic. Deben discutir: ¿Cómo mantenemos el prompt consistente? ¿Cómo manejamos los diferentes formatos de respuesta? Al final, un representante de cada grupo comparte su estrategia.

Tiempos

Sección	Minutos
Introducción	10
Contenido principal	20
Actividades	70
Preguntas y respuestas	10
Cierre	10
Total	120

• Podcast

La interfaz base: conexión y abstracción de LLMs – PODCAST

Un episodio conversacional que recapitula los fundamentos técnicos para conectar y abstraer modelos de lenguaje en aplicaciones profesionales.

Resumen del Episodio: El Puente hacia la Escalabilidad

En este episodio de cierre, conversamos sobre por qué no deberías "casarte" con un SDK específico y cómo la ingeniería de software clásica nos salva del caos en el mundo de la IA Generativa. Repasamos todo el camino recorrido en el módulo: desde el poder de **Asyncio** para manejar múltiples peticiones sin bloquear nuestro sistema, pasando por la creación de **clases base** que estandarizan la comunicación con proveedores como OpenAI o Anthropic, hasta la importancia vital de medir la **latencia** para garantizar una buena experiencia de usuario.

Al escuchar este podcast, consolidarás una visión arquitectónica: entenderás que un desarrollador de IA no solo lanza prompts, sino que construye infraestructuras **robustas, asíncronas y agnósticas al proveedor**.

Puntos clave:

- **La potencia de Python 3.12 y Asyncio:** Por qué la concurrencia es el corazón de las aplicaciones de IA modernas.
- **Patrones de Abstracción:** El uso de clases abstractas para crear un "idioma común" entre diferentes LLMs.
- **Métricas que Importan:** Tiempo hasta el primer token (TTFT) y por qué los estados de la conexión determinan la estabilidad.
- **Preparación para el Cambio:** Cómo una interfaz base te permite migrar de proveedor en minutos, no en semanas.

MÓDULO 2

Encadenamiento lógico: orquestación con LangChain

Diseñar y ejecutar cadenas lógicas de procesamiento utilizando LangChain y LCEL para flujos de trabajo predecibles.

• Lectura

Arquitectura de cadenas con LCEL y runnables (Texto)

Aprende a diseñar pipelines de IA profesionales utilizando LangChain Expression Language (LCEL). Esta unidad enseña a componer flujos de trabajo escalables, asíncronos y robustos mediante el protocolo Runnable.

Arquitectura de Cadenas con LCEL y Runnables

Bienvenido a la primera unidad del módulo de **Encadenamiento Lógico**. Hasta ahora, en el módulo anterior, has aprendido a conectar con LLMs como OpenAI y Anthropic utilizando sus SDKs base y gestionando la asincronía con Python 3.12. Has visto cómo fluye el texto y cómo controlar la latencia. Sin embargo, en el desarrollo de software profesional, las aplicaciones de IA rara vez consisten en una única llamada a un modelo.

Imagina que estás construyendo un sistema de análisis de sentimientos que primero debe traducir el texto, luego clasificarlo, extraer entidades clave en formato JSON y finalmente generar un resumen ejecutivo. Si usaras código imperativo tradicional (muchos `if`, `await` y transformaciones manuales), navegarías en un "código espagueti" difícil de testear y escalar.

Aquí es donde entra **LangChain Expression Language (LCEL)**. En esta unidad, aprenderás a transformar esos procesos aislados en tuberías (pipelines) declarativas, robustas y preparadas para producción.

1. El Porqué de LCEL: De Scripts a Tuberías de Ingeniería

El problema de la orquestación manual

Cuando trabajas con SDKs básicos (como el de OpenAI), cada paso de tu lógica requiere manejo manual:

1. **Formateo de variables:** Insertar datos del usuario en un string.
2. **Llamada al modelo:** Gestionar reintentos y sesiones.
3. **Parsing:** Convertir el string de respuesta en algo útil (un objeto Python o un JSON).

Si quieres añadir soporte para **streaming**, **ejecución en paralelo** o **rastreo de errores (tracing)**, el código se vuelve exponencialmente complejo.

La solución: LCEL (LangChain Expression Language)

LCEL es un lenguaje declarativo diseñado para componer cadenas de manera intuitiva. Se basa en una idea fundamental: **cada componente es un "bloque" con una interfaz estándar**.

La ventaja principal de LCEL no es solo la sintaxis elegante (el uso del operador pipe `|`), sino lo que obtienes "gratis" al usarlo:

- **Soporte de primera clase para Async:** Puedes alternar entre síncrono y asíncrono sin cambiar la lógica.
- **Streaming automático:** Si el modelo soporta streaming, la cadena lo expone inmediatamente.
- **Ejecución en paralelo:** Si tienes dos tareas independientes (ej. resumir y extraer keywords), LCEL puede ejecutarlas en paralelo sin configurar `asyncio.gather` manualmente.
- **Observabilidad:** Se integra nativamente con herramientas como LangSmith para ver exactamente qué pasó en cada eslabón de la cadena.

2. El Protocolo Runnable: La Interfaz Universal

Para que LCEL funcione, todos los componentes deben hablar el mismo idioma. En LangChain, ese idioma es el **Protocolo Runnable**.

Casi todos los objetos en LangChain (modelos, prompts, parsers, herramientas) heredan de la clase base `Runnable`. Esto garantiza que todos implementen los mismos métodos estratégicos:

Método	Descripción
<code>invoke()</code>	Llama al componente con una entrada única (síncrono).
<code>ainvoke()</code>	Versión asíncrona de <code>invoke</code> .
<code>batch()</code>	Llama al componente con una lista de entradas (optimiza latencia).
<code>stream()</code>	Devuelve fragmentos de la respuesta a medida que se generan.

Analogía de la Estación de Trabajo: Imagina una fábrica de coches. Cada estación (Runnable) tiene una entrada de piezas y una salida de componentes más complejos. No importa si la estación pone puertas o pinta el chasis; todas aceptan piezas en una cinta transportadora y entregan el resultado a la siguiente. El operador `|` es la cinta transportadora que conecta estas estaciones.

3. Composición de Cadenas: El Operador Pipe |

La sintaxis core de LCEL utiliza el operador `|` (pipe), similar a como se usa en terminales de Unix o en lenguajes funcionales. La salida del componente a la izquierda se convierte en la entrada del componente a la derecha.

Estructura típica de una cadena

Un flujo estándar de IA suele verse así: `PromptTemplate | ChatModel | OutputParser`

- 1. PromptTemplate (Input -> PromptValue):** Toma un diccionario de variables y genera el mensaje formateado.
- 2. ChatModel (PromptValue -> Message):** Toma el mensaje y lo envía al LLM.
- 3. OutputParser (Message -> String/JSON):** Toma la respuesta cruda del modelo y la limpia.

Ejemplo concreto:

Si queremos un traductor inteligente:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Definimos el Prompt
prompt = ChatPromptTemplate.from_template("Translate the following text to {language}: {text}")

# 2. El Modelo
model = ChatOpenAI(model="gpt-4o")

# 3. El Parser
parser = StrOutputParser()

# 4. Composición de la cadena
chain = prompt | model | parser

# 5. Ejecución
# Observa que pasamos un diccionario que matchea las variables del prompt
response = await chain.ainvoke({"language": "Spanish", "text": "Learning LCEL is powerful."})
print(response) # "Aprender LCEL es poderoso."
```

¿Qué está pasando bajo el capó?

Cuando ejecutas `chain.ainvoke(...)`:

- 1.** El diccionario fluye al `prompt`. El `prompt` genera un objeto `ChatPromptValue`.

2. Ese objeto pasa al `model`. El `model` entiende ese tipo de entrada y genera un `BaseMessage`.
 3. El `BaseMessage` pasa al `parser`, que extrae el contenido de texto (el `.content`) y devuelve un string limpio.
-

4. Gestión Avanzada de Prompts en LCEL

En sistemas escalables, los prompts no son simples strings; son **artefactos de software**. Deben ser modulares, versionables y capaces de manejar estructuras complejas.

ChatPromptTemplate: Más que solo texto

A diferencia de los prompts básicos, `ChatPromptTemplate` permite definir roles (System, Human, AI), lo cual es crucial para modelos modernos que dependen del contexto del sistema para seguir instrucciones de seguridad o formato.

```
template = ChatPromptTemplate.from_messages([
    ("system", "Eres un arquitecto de software experto en {topic}."),
    ("human", "Explicame los beneficios de {subtopic} en 3 puntos.")
])
```

Modelado de Entradas (Input Shaping)

Un error común para los desarrolladores es intentar "limpiar" los datos antes de enviarlos a la cadena. LCEL nos permite hacerlo *dentro* de la cadena usando `RunnablePassthrough` o funciones lambda.

Si tu cadena espera un diccionario con `context` y `question`, pero tu base de datos solo te da la `question`, puedes modelar la entrada así:

```
from langchain_core.runnables import RunnablePassthrough

# Esta cadena 'inyecta' datos antes de llegar al prompt
map_chain = (
    {"context": lambda x: "Información secreta sobre " + x["topic"], "question": lambda x:
x["question"]}
    | prompt
    | model
)
```

5. Operaciones por Lote y Paralelismo

Una de las joyas de LCEL es la capacidad de ejecutar múltiples tareas simultáneamente sin complejidad adicional.

Ejecución en Paralelo (`RunnableParallel`)

Imagina que quieres evaluar una idea de negocio desde dos perspectivas: técnica y financiera.

```
from langchain_core.runnables import RunnableParallel

chain_tecnica = prompt_tecnico | model | parser
chain_financiera = prompt_financiero | model | parser

# Ejecuta ambas al mismo tiempo
analisis_total = RunnableParallel(tech=chain_tecnica, finance=chain_financiera)

resultado = await analisis_total.ainvoke({"idea": "Drones que entregan pizza"})
# resultado será: {"tech": "...", "finance": "..."}
```

Esto reduce el tiempo total de respuesta al máximo posible (el tiempo de la rama más lenta), en lugar de sumar ambos tiempos.

Procesamiento por Lote (`batch`)

Si tienes 100 reseñas de clientes para analizar, usar un bucle `for` con `ainvoke` es ineficiente. El método `abatch` optimiza internamente las llamadas para ejecutarlas de la manera más rápida que permita el proveedor.

```
inputs = [
    {"language": "French", "text": "Hello"},
    {"language": "German", "text": "Goodbye"},
]

results = await chain.abatch(inputs)
```

6. Errores Comunes y Mejores Prácticas

Error 1: Mismatch de tipos de datos

El error: Intentar pasar un string a una cadena que espera un diccionario. **La solución:** Si tu `ChatPromptTemplate` tiene variables (ej. `{text}`), la entrada de la cadena **debe** ser un diccionario `{"text": "valor"}`. Si el prompt no tiene variables, puedes pasar el string directamente.

Error 2: Ignorar el Output Parser

El error: No usar un parser y recibir un objeto `AIMessage` en lugar de texto. **La solución:** Siempre termina tus cadenas de texto con `StrOutputParser()`. Esto asegura que el resto de tu aplicación reciba un string estándar de Python.

Mejor Práctica: Usar `ainvoke` por defecto

En aplicaciones web (como FastAPI), siempre prefiere los métodos asíncronos (`ainvoke`, `abatch`, `astream`). Esto evita que el loop de eventos de Python se bloquee mientras esperas la respuesta de la API de OpenAI, permitiendo que tu servidor atienda a otros usuarios.

7. Ejemplo resuelto paso a paso: pipeline de análisis de reseñas (end-to-end)

Hasta acá viste cada pieza por separado (pipe, `RunnableParallel`, parser). Ahora las combinamos en un caso real y resuelto: una cadena que, a partir de una reseña de cliente, **en paralelo** clasifica el sentimiento y extrae el tema, y luego **synetiza** todo en una línea de ticket para el equipo de soporte.

```
import asyncio
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel

model = ChatOpenAI(model="gpt-4o", temperature=0)
parser = StrOutputParser()

# --- Sub-cadena 1: sentimiento ---
sentiment_chain = (
    ChatPromptTemplate.from_template(
        "Clasificá el sentimiento de esta reseña en una palabra "
        "(positivo/negativo/neutro): {review}"
    )
    | model
    | parser
)

# --- Sub-cadena 2: tema principal ---
topic_chain = (
    ChatPromptTemplate.from_template(
        "¿Cuál es el tema principal de esta reseña? Respondé en 3 palabras: {review}"
    )
    | model
    | parser
)
```

```
# --- Paso 1: corremos ambas sub-cadenas EN PARALELO ---
analysis = RunnableParallel(sentiment=sentiment_chain, topic=topic_chain)

# --- Paso 2: síntesis en una línea de ticket ---
ticket_prompt = ChatPromptTemplate.from_template(
    "Sentimiento: {sentiment}\nTema: {topic}\n"
    "Escribí una sola línea de ticket de soporte (máx. 15 palabras) para el equipo."
)

# --- Cadena completa: análisis paralelo -> síntesis ---
full_chain = analysis | ticket_prompt | model | parser

async def main():
    resultado = await full_chain.ainvoke(
        {"review": "Llevo 3 días esperando que respondan mi reclamo y nadie me contesta."}
    )
    print(resultado)

if __name__ == "__main__":
    asyncio.run(main())
```

Salida esperada (ejemplo):

```
Cliente sin respuesta a su reclamo hace 3 días – sentimiento negativo, priorizar.
```

Qué pasó, paso a paso:

1. `ainvoke({"review": ...})` entrega el diccionario a `RunnableParallel`, que se lo pasa **a las dos sub-cadenas a la vez**. Ambas usan la variable `{review}`. No esperamos a que termine una para empezar la otra: el tiempo total es el de la rama más lenta, no la suma.
2. Cada sub-cadena es un `prompt | model | parser` completo, así que devuelve **texto limpio** (gracias a `StrOutputParser`), no un objeto `AIMessage`.
3. `RunnableParallel` arma un diccionario `{"sentiment": "...", "topic": "..."}` . Ese diccionario **fluye directo** a `ticket_prompt`, cuyas variables `{sentiment}` y `{topic}` coinciden con esas claves (acá está la clave de por qué los nombres deben matchear).
4. El paso final vuelve a pasar por `model | parser` para producir la línea de ticket como string listo para usar.

Este patrón —descomponer en sub-cadenas, paralelizar lo independiente y sintetizar al final— es la base de cualquier pipeline LCEL de producción.

Segundo ejemplo resuelto: cadena secuencial + procesamiento por lote

El primer ejemplo mostró ramas en paralelo. Este muestra el patrón opuesto y igual de común: **encadenar pasos en secuencia** (la salida de uno es la entrada del siguiente) y procesar **varios inputs a la vez** con `abatch`. Caso: generar una descripción de producto y luego condensarla a un tweet.

```
import asyncio
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model = ChatOpenAI(model="gpt-4o", temperature=0.7)
parser = StrOutputParser()

# --- Paso 1: generar una descripción de producto ---
descripcion_chain = (
    ChatPromptTemplate.from_template(
        "Escribí una descripción de producto de 2 oraciones para: {producto}"
    )
    | model
    | parser
)

# --- Paso 2: condensar esa descripción a un tweet ---
tweet_chain = (
    ChatPromptTemplate.from_template(
        "Resumí esto en un tweet de menos de 120 caracteres, con 1 emoji: {descripcion}"
    )
    | model
    | parser
)

# --- Cadena secuencial: la salida (string) del paso 1 se "envuelve" en el dict
#      que espera el paso 2. La función lambda se convierte en Runnable automáticamente. ---
full_chain = descripcion_chain | (lambda desc: {"descripcion": desc}) | tweet_chain

async def main():
    # .abatch procesa varios productos a la vez (más eficiente que un for con ainvoke)
    productos = [{"producto": "termo de acero 1L"}, {"producto": "mochila antirrobo"}]
    tweets = await full_chain.abatch(productos)
    for p, t in zip(productos, tweets):
        print(f"{p['producto']}: {t}")
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Salida esperada (ejemplo):

termo de acero 1L: Frío 24h o calor 12h, donde vayas. Tu bebida siempre lista 🧊 mochila antirrobo: Tu laptop y tus cosas, siempre seguras. Salí tranquilo 🗝️

Qué pasó, paso a paso:

1. `descripcion_chain` produce un **string** (gracias a `StrOutputParser`).
2. Ese string no se puede pasar tal cual a `tweet_chain`, porque su prompt espera la variable `{descripcion}`. Por eso intercalamos `(lambda desc: {"descripcion": desc})`: LCEL convierte cualquier función en un `Runnable`, así que **modelás la entrada** sin código extra.
3. `tweet_chain` recibe ese diccionario y genera el tweet final.
4. `abatch([...])` ejecuta toda la cadena para **los dos productos a la vez**, optimizando las llamadas. Para 200 productos, esto es muchísimo más rápido que un `for`.

Mientras el primer ejemplo paralelizaba tareas independientes, este encadena pasos dependientes: son las dos formas básicas de componer en LCEL.

8. Síntesis y Conexiones

LCEL no es magia; es una abstracción poderosa sobre el protocolo **Runnable**. Al usarlo, dejas de escribir "código de pegamento" y empiezas a diseñar **arquitectura de datos para IA**.

Puntos clave para recordar:

1. **Composabilidad:** Usa `|` para unir Prompt -> Modelo -> Parser.
2. **Interfaz Única:** Todos los componentes implementan `invoke`, `batch` y `stream`.
3. **Declarativo:** El código describe el *qué* (el flujo), no el *cómo* (la gestión de hilos/estados).

En la siguiente unidad, daremos un paso más allá aprendiendo cómo forzar al modelo a responder con esquemas de datos estrictos (JSON) y cómo manejar errores cuando el flujo se rompe. Por ahora, es momento de que refactorices tu conocimiento previo hacia este nuevo paradigma.

Práctica

Título del ejercicio: "Refactorización a LCEL Asíncrono"

Propósito del ejercicio

Este ejercicio tiene como objetivo que migres de una implementación imperativa basada en SDKs crudos (OpenAI/Anthropic) a una arquitectura declarativa usando **LCEL (LangChain Expression Language)**. Practicarás la composición de componentes y la gestión de la asincronía en un flujo profesional.

Objetivo de aprendizaje

Al finalizar vas a tener un archivo Python funcional que define y ejecuta **una cadena LCEL asíncrona concreta**: toma una pregunta del usuario como diccionario (ej. `{"pregunta": "..."}`), la procesa con `ChatPromptTemplate → modelo → StrOutputParser` y devuelve la respuesta como **texto plano**.

En concreto, vas a lograr:

1. Reemplazar la llamada manual al SDK del Módulo 1 por un modelo de LangChain (`ChatOpenAI` o `ChatAnthropic`).
2. Componer `prompt | modelo | parser` con el operador `|` .
3. Ejecutar la cadena con `await chain.ainvoke({...})` .

Cómo sabés que lo lograste: tu script corre sin errores, imprime una respuesta en **texto plano** (no un objeto `AIMessage`) y todo el flujo principal está armado con LCEL (el operador `|`), no con lógica imperativa.

Instrucciones paso a paso

1. Preparación del entorno:

- Asegúrate de tener instalados `langchain` , `langchain-openai` (o `langchain-anthropic`) y `python-dotenv` .

2. Configuración del Modelo:

- Sustituye la inicialización del cliente manual por una instancia de `ChatOpenAI` o `ChatAnthropic` .
- Configura parámetros base como `temperature` y `model_name` .

3. Definición del Template:

- Usa `ChatPromptTemplate.from_messages` para crear un prompt que reciba al menos una variable (ej. `contexto` o `pregunta`).

4. Construcción de la Cadena (Pipeline):

- Usa el operador `|` para conectar: `Prompt | Model | StrOutputParser()` .

5. Ejecución Asíncrona:

- Crea una función asíncrona principal (`main`) que ejecute la cadena usando el método `.ainvoke()` .
- Pasa los inputs necesarios como un diccionario.

Criterios de Aceptación

1. El código debe utilizar **exclusivamente** la sintaxis de LCEL (`|`) para el flujo principal.
2. La ejecución de la cadena debe ser **asíncrona** (`await chain.ainvoke(...)`).
3. La salida debe ser texto plano extraído mediante `StrOutputParser` (no el objeto `BaseMessage` completo).
4. El prompt debe utilizar un `ChatPromptTemplate` con roles definidos (System/Human).

Errores comunes a evitar

- **Olvidar el `await`** : Al usar `ainvoke` , si olvidas el `await` , obtendrás una coroutine en lugar del resultado.
- **Manejo de variables**: Asegúrate de que las llaves `{variable}` en tu prompt coincidan exactamente con las llaves en el diccionario que pasas al `ainvoke` .

Entregable

Tipo: REPOSITORY

Un repositorio de GitHub (o estructura de archivos local) que contenga la refactorización del cliente del Módulo 1. Debe incluir un archivo Python que defina y ejecute una cadena LCEL asíncrona usando `ChatPromptTemplate`, un modelo de `LangChain` y `StrOutputParser`.

Arquitectura de Cadenas con LCEL y Runnables (Texto)

Aprende a diseñar pipelines de IA profesionales utilizando LangChain Expression Language (LCEL). Esta unidad enseña a componer flujos de trabajo escalables, asíncronos y robustos mediante el protocolo Runnable.

Entregable

Tipo: REPOSITORY

Un repositorio de GitHub (o estructura de archivos local) que contenga la refactorización del cliente del Módulo 1. Debe incluir un archivo Python que defina y ejecute una cadena LCEL asíncrona usando `ChatPromptTemplate`, un modelo de `LangChain` y `StrOutputParser`.

Instrucciones

Título del ejercicio: "Refactorización a LCEL Asíncrono"

Propósito del ejercicio

Este ejercicio tiene como objetivo que migres de una implementación imperativa basada en SDKs crudos (OpenAI/Anthropic) a una arquitectura declarativa usando **LCEL (LangChain Expression Language)**. Practicarás la composición de componentes y la gestión de la asincronía en un flujo profesional.

Objetivo de aprendizaje

Al finalizar vas a tener un archivo Python funcional que define y ejecuta **una cadena LCEL asíncrona concreta**: toma una pregunta del usuario como diccionario (ej. `{"pregunta": "..."}`), la procesa con `ChatPromptTemplate → modelo → StrOutputParser` y devuelve la respuesta como **texto plano**.

En concreto, vas a lograr:

1. Reemplazar la llamada manual al SDK del Módulo 1 por un modelo de LangChain (`ChatOpenAI` o `ChatAnthropic`).
2. Componer `prompt | modelo | parser` con el operador `|`.
3. Ejecutar la cadena con `await chain.ainvoke({...})`.

Cómo sabés que lo lograste: tu script corre sin errores, imprime una respuesta en **texto plano** (no un objeto `AIMessage`) y todo el flujo principal está armado con LCEL (el operador `|`), no con lógica imperativa.

Instrucciones paso a paso

1. Preparación del entorno:

- Asegurate de tener instalados `langchain`, `langchain-openai` (o `langchain-anthropic`) y `python-dotenv`.

2. Configuración del Modelo:

- Sustituye la inicialización del cliente manual por una instancia de `ChatOpenAI` o `ChatAnthropic`.
- Configura parámetros base como `temperature` y `model_name`.

3. Definición del Template:

- Usa `ChatPromptTemplate.from_messages` para crear un prompt que reciba al menos una variable (ej. `contexto` o `pregunta`).

4. Construcción de la Cadena (Pipeline):

- Usa el operador `|` para conectar: `Prompt | Model | StrOutputParser()`.

5. Ejecución Asíncrona:

- Crea una función asíncrona principal (`main`) que ejecute la cadena usando el método `.ainvoke()`.
- Pasa los inputs necesarios como un diccionario.

Criterios de Aceptación

1. El código debe utilizar **exclusivamente** la sintaxis de LCEL (`|`) para el flujo principal.
2. La ejecución de la cadena debe ser **asíncrona** (`await chain.ainvoke(...)`).
3. La salida debe ser texto plano extraído mediante `StrOutputParser` (no el objeto `BaseMessage` completo).
4. El prompt debe utilizar un `ChatPromptTemplate` con roles definidos (System/Human).

Errores comunes a evitar

- **Olvidar el `await`** : Al usar `ainvoke` , si olvidas el `await`, obtendrás una corrutina en lugar del resultado.
- **Manejo de variables**: Asegúrate de que las llaves `{variable}` en tu prompt coincidan exactamente con las llaves en el diccionario que pasas al `ainvoke` .

Objetivos de aprendizaje

- Componer procesos de IA utilizando la sintaxis de pipe (`|`) de LCEL para crear pipelines legibles y mantenibles.
- Implementar el protocolo `Runnable` para ejecutar operaciones síncronas, asíncronas y por lotes (batching).
- Gestionar `ChatPromptTemplates` dinámicos para desacoplar la lógica del negocio de las instrucciones del modelo.
- Utilizar `StrOutputParser` para estandarizar la salida de los LLMs hacia el resto de la aplicación.

Rúbrica de Evaluación: Refactorización a LCEL Asíncrono

Esta rúbrica evalúa la capacidad del estudiante para migrar de implementaciones imperativas a una arquitectura declarativa utilizando LangChain Expression Language (LCEL). Se centra en la orquestación de componentes, la gestión de la asincronía y el desacoplamiento de la lógica de procesamiento para construir sistemas de IA escalables y predecibles.

Criterio	Descripción	Peso
Implementación de Sintaxis LCEL	Uso correcto del operador pipe (<code> </code>) para la composición de la cadena.	30%
Gestión de Asincronía	Ejecución correcta de la cadena utilizando el paradigma asíncrono de Python.	25%
Configuración de Componentes y Prompting	Estructuración de <code>ChatPromptTemplate</code> con roles y configuración del modelo.	25%
Post-procesamiento y Estructura del Repositorio	Extracción de datos con <code>StrOutputParser</code> y organización profesional del código.	20%

Total: 100 pts | Aprobación: 65 pts

Aprende a garantizar la fiabilidad de tus cadenas de LangChain mediante la validación estricta de esquemas con Pydantic y la implementación de estrategias de reintento para fallos transitorios.

Validación estructurada y resiliencia en cadenas

En la ingeniería de software tradicional, los contratos de datos (como una especificación OpenAPI o una interfaz de TypeScript) aseguran que dos sistemas se comuniquen sin errores de formato. En el mundo de la IA, este principio es aún más crítico: los LLMs son probabilísticos y su salida natural es texto libre, lo cual es veneno para la escalabilidad.

Esta unidad se centra en transformar la "magia" de las respuestas de texto en **sistemas predecibles y resilientes** mediante dos pilares: la validación estricta de esquemas y el manejo proactivo de errores.

1. El Contrato de Datos: Salidas Estructuradas

Cuando un LLM responde en lenguaje natural, corres el riesgo de que cambie el formato de la respuesta, alucine con claves JSON o devuelva tipos de datos incorrectos (como un string donde esperabas un número).

¿Por qué Pydantic?

Pydantic es la librería de validación de datos estándar en Python moderno. Al integrarla con LangChain, definimos un **esquema formal** que el modelo está obligado a seguir.

- **Validación Sintáctica:** El JSON es íntegro y parseable.
- **Validación Estructural:** Todos los campos requeridos están presentes.
- **Validación Semántica:** Los valores cumplen reglas de negocio (ej. un valor de `confianza` debe estar entre 0 y 1).

El método `with_structured_output`

LangChain abstrae la complejidad de parsear manualmente las salidas mediante este método. Por debajo, utiliza técnicas como el **Tool Calling** o **JSON Mode** de los proveedores (OpenAI, Anthropic) para garantizar que el modelo "encaje" su pensamiento en tu molde.

2. Resiliencia: Manejo de Errores y Reintentos

Un sistema de IA en producción no solo debe ser correcto, debe ser **robusto**. Las llamadas a modelos pueden fallar por:

1. **Errores Transitorios:** Caídas de red, latencia o límites de tasa (429 Rate Limit).
2. **Errores de Formato:** El modelo no pudo seguir exactamente el esquema Pydantic.

Estrategias de Recuperación

En lugar de dejar que el sistema falle y devuelva un error al usuario, implementamos:

- **Retries con Backoff Exponencial:** Si falla, esperamos un poco más antes de intentar de nuevo. Esto evita saturar el servicio y permite que fallos momentáneos se resuelvan solos.
- **Fallbacks:** Si un modelo (ej. GPT-4o) falla repetidamente, podemos redirigir la carga a un modelo más ligero o a una respuesta predefinida.

Errores Comunes de Arquitectura

1. **Confiar ciegamente en el JSON del modelo:** Nunca asumas que un `json.loads` funcionará. Usa siempre un validador de esquema.
2. **Reintentar errores permanentes:** Si el prompt tiene un error lógico que causa rechazo, reintentar 10 veces solo desperdicia dinero y latencia. Diferencia entre errores de transporte (red) y errores de lógica.

En el siguiente ejercicio, aplicarás estos conceptos para convertir una cadena frágil en un pipeline de grado producción.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import asyncio
from typing import List, Optional
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# TODO 1: Define la clase Pydantic 'EntityExtraction'
# Debe tener: topic (str), entities (Lista de str), y sentiment_score (float entre 0 y 1)
class EntityExtraction(BaseModel):
    pass

async def run_validated_chain(text: str):
    llm = ChatOpenAI(model="gpt-4o", temperature=0)

    # TODO 2: Configura el modelo para usar la salida estructurada con Pydantic
    # Tip: Usa el método .with_structured_output()
```



```

structured_llm = None

# TODO 3: Agrega una estrategia de reintento con .with_retry()
# para que sea resiliente ante fallos de conexión (máximo 3 intentos).
resilient_llm = None

prompt = ChatPromptTemplate.from_messages([
    ("system", "Analiza el texto y extrae las entidades."),
    ("human", "{input}")
])

# TODO 4: Une el prompt con el resilient_llm y ejecuta asíncronamente
# No olvides manejar excepciones con try/except para capturar fallos de validación
try:
    pass
except Exception as e:
    print(f"Error: {e}")

if __name__ == "__main__":
    sample_text = "LangGraph es una extensión de LangChain para agentes cíclicos."
    asyncio.run(run_validated_chain(sample_text))

```

Código solución

```

import asyncio
from typing import List, Optional
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableConfig

# 1. Definición del Esquema de Datos con Pydantic
class EntityExtraction(BaseModel):
    """Esquema para la extracción de entidades clave de un texto técnico."""
    topic: str = Field(description="El tema principal del texto")
    entities: List[str] = Field(description="Lista de nombres propios, tecnologías o conceptos clave")
    sentiment_score: float = Field(description="Puntuación de 0 a 1 sobre qué tan positivo es el texto", ge=0, le=1)
    complexity_level: Optional[str] = Field(description="Nivel: 'Low', 'Medium', 'High'", pattern="^(Low|Medium|High)$")

    async def run_validated_chain(text: str):

```

```

# Configuración del modelo
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# 2. Configurar Structured Output (Contrato de Datos)
# Esto asegura que el modelo use internamente herramientas para devolver JSON válido
structured_llm = llm.with_structured_output(EntityExtraction)

# 3. Configurar Estrategia de Reintento (Resiliencia)
# Reintentará en caso de errores de red o rate limits del proveedor
resilient_llm = structured_llm.with_retry(
    stop_after_attempt=3,
    wait_exponential_jitter=True
)

prompt = ChatPromptTemplate.from_messages([
    ("system", "Eres un experto en análisis de datos técnicos. Extrae la información solicitada en formato estructurado."),
    ("human", "{input}")
])

# Composición de la cadena LCEL
chain = prompt | resilient_llm

try:
    # Ejecución asíncrona
    result = await chain.ainvoke({"input": text})
    print("--- Extracción Exitosa ---")
    print(result.model_dump_json(indent=2))
    return result

except Exception as e:
    # 4. Manejo de Errores de Validación o SDK
    print(f"Error crítico en la cadena: {type(e).__name__}")
    print(f"Detalles: {str(e)}")
    # Aquí se podría implementar un fallback o alerta de observabilidad
    return None

if __name__ == "__main__":
    sample_text = "Python 3.12 introduce mejoras de rendimiento significativas y un nuevo sistema de tipos."
    asyncio.run(run_validated_chain(sample_text))

```

Pistas

- Utiliza 'with_structured_output(ClasaPydantic)' sobre tu objeto de modelo para forzar al LLM a seguir el esquema. Es más robusto que intentar parsear texto libre.
- Para la resiliencia, LangChain ofrece el método '.with_fallbacks()' o '.with_retry()'. En este caso, '.with_retry()' es ideal para manejar errores transitorios como rate limits o fallos de conexión.
- Recuerda que Pydantic lanzará una 'ValidationError' si el LLM devuelve un JSON válido pero con tipos o valores incorrectos. Envuelve la invocación final en un bloque try/except para manejar este caso específico.

Rúbrica: Validación Estructurada y Resiliencia en Cadenas

Esta evaluación mide la capacidad del estudiante para diseñar flujos de IA que no solo generen respuestas, sino que garanticen la integridad de los datos mediante contratos Pydantic y gestionen fallos mediante estrategias de resiliencia en LangChain.

Criterio	Descripción	Peso
Integridad del Contrato de Datos (Pydantic)	Uso y definición de modelos Pydantic para validar la salida del LLM, incluyendo validaciones semánticas y tipos de datos correctos.	35%
Implementación de Structured Output y LCEL	Capacidad para integrar el esquema de validación dentro de la cadena utilizando el método .with_structured_output() y la sintaxis LCEL.	25%
Estrategias de Resiliencia (Retries y Fallbacks)	Implementación de mecanismos para manejar errores de red o de formato de manera proactiva.	25%
Arquitectura y Calidad de Código Inline	Calidad del código escrito en el editor: legibilidad, modularidad interna y uso de Python 3.12.	15%

Total: 100 pts | Aprobación: 65 pts

• Autoevaluación

Evaluación: Orquestación y contratos de datos

Evaluación de conocimientos sobre orquestación de LLMs con LCEL, uso de Runnables y validación de salidas estructuradas para sistemas de IA escalables.

Evaluación: Orquestación y contratos de datos

¡Bienvenido a la evaluación de la Unidad 3! En esta etapa del módulo, ya hemos explorado cómo **LCEL (LangChain Expression Language)** nos permite declarar flujos de trabajo de IA como si fueran tuberías de datos, y cómo asegurar que esos datos no sean solo texto libre, sino estructuras validadas.

¿Qué estamos evaluando?

Esta evaluación no busca que memorices nombres de funciones, sino que demuestres **criterio arquitectural**. Un arquitecto de IA debe saber:

- Cómo fluye la información a través de un `Runnable`.
- Por qué la composición con `|` es superior a las cadenas imperativas tradicionales.
- Cómo transformar la incertidumbre de un LLM en un **contrato de datos** robusto mediante Pydantic y parsers estructurados.

Recuerda: en producción, que un LLM responda "bien" no es suficiente; necesitamos que responda con la **forma y validación** correctas para que el resto de nuestro sistema (frontend, bases de datos, APIs) pueda operar sin fallos.

¡Mucho éxito!

Quiz

Aprobación: 75 · Intentos máximos: 3

1. ¿Cuál es la ventaja técnica fundamental de utilizar el protocolo Runnable en LCEL frente al encadenamiento manual de funciones?

- ☐ Permite escribir código puramente imperativo para facilitar la depuración paso a paso.
- ☒ **Estandariza una interfaz común (Runnable) que habilita automáticamente streaming, soporte asíncrono y ejecución por lotes (batch).**
- ☐ Se creó exclusivamente para mejorar la legibilidad visual del código mediante el operador pipe.
- ☐ Elimina la necesidad de utilizar modelos de OpenAI, forzando el uso de modelos locales.

Explicación: El protocolo **Runnable** es el corazón de LCEL. Su principal ventaja es que estandariza la interfaz (`invoke`, `batch`, `stream`, `ainvoke`) para todos los componentes. Al usar el operador `|`, LangChain asegura que las capacidades de streaming y ejecución asíncrona se propaguen automáticamente por toda la cadena.

- La opción **A** es incorrecta porque LCEL es declarativo, no imperativo; busca reducir el "código glue".
- La opción **C** es incorrecta porque aunque es legible, la principal ventaja técnica es la interfaz unificada de ejecución (Async/Streaming/Batch).
- La opción **D** es falsa; LCEL es agnóstico al proveedor siempre que el modelo esté envuelto en un wrapper de LangChain.

2. Estás diseñando una cadena para extraer automáticamente datos de facturas. ¿Cuál es la secuencia de composición de Runnables correcta utilizando LCEL?

☐ `model | prompt | parser`

☒ `prompt | model | parser`

☐ `parser | prompt | model`

☐ `prompt | parser | model`

Explicación: En una cadena de extracción, el orden lógico es:

1. **Prompt:** Prepara la instrucción.
2. **Model:** Procesa la instrucción y genera texto.
3. **Parser:** Transforma ese texto libre en un objeto estructurado (como un diccionario o modelo de Pydantic).

Si el parser estuviera antes del modelo, intentaría procesar la instrucción del prompt en lugar de la respuesta de la IA. Si el passthrough estuviera mal ubicado, perderíamos el input original necesario para el prompt. Un error común es olvidar el parser, lo que dejaría a la aplicación con un objeto `AIMessage` en lugar de datos listos para una base de datos.

3. Incluso utilizando la funcionalidad de 'Structured Outputs' de proveedores como OpenAI o Anthropic, ¿qué tipo de validación suele requerir todavía una capa adicional de código en Python?

☒ **Validación semántica:** asegurar que los datos tengan sentido lógico (ej. que la fecha de inicio sea anterior a la de fin).

☐ Validación sintáctica: asegurar que la respuesta sea un JSON parseable.

☐ Validación estructural: asegurar que todos los campos requeridos estén presentes.

☐ Validación de tipos: asegurar que un campo definido como entero no llegue como string.

Explicación: Este es un concepto crítico: los modelos de lenguaje son excelentes siguiendo la *forma* (JSON), pero pueden fallar en la *lógica* (Semántica).

- **A y B** son validaciones técnicas que las APIs modernas de "Structured Outputs" manejan muy bien.
- **D** (Tipado) se resuelve con el esquema.
- **C** es una validación de negocio. Por ejemplo, el LLM puede devolver un `precio_total` que no coincida con la suma de `items`. Esto requiere una validación secundaria en Python (Pydantic `model_validator`) para asegurar que los datos no solo sean "legibles", sino "correctos" para el negocio.

4. ¿En qué escenarios es fundamental el uso de `RunnableParallel` dentro de una cadena LCEL? (Selecciona todas las que apliquen)

☒ Ejecutar múltiples Runnables de forma concurrente para optimizar el tiempo de respuesta.

☒ Mapear o transformar el input para que coincida con las múltiples variables que espera un `ChatPromptTemplate`.

☐ Garantizar que las tareas se ejecuten secuencialmente para evitar condiciones de carrera.

☐ Gestionar automáticamente los reintentos (retries) cuando un modelo falla.

Explicación: `RunnableParallel` tiene dos funciones principales en arquitectura:

1. **Eficiencia:** Ejecutar múltiples tareas independientes (ej. resumen y análisis de sentimiento) al mismo tiempo para reducir la latencia total.
2. **Manipulación de Estado:** Permite "preparar el terreno" para el siguiente paso, convirtiendo un input simple en un diccionario con múltiples claves que el `ChatPromptTemplate` puede consumir directamente.

La opción **C** es incorrecta porque el orden de ejecución en paralelo no está garantizado. La opción **D** es incorrecta porque `RunnableParallel` no maneja reintentos; para eso se usa `.with_fallbacks()`.

5. Si un modelo de Pydantic falla al validar la salida de un LLM durante una ejecución en producción, ¿cuál es la mejor práctica de resiliencia?

☒ Capturar el error de validación y enviar un nuevo prompt al modelo que incluya el error específico para que este lo corrija.

☐ Configurar un valor por defecto (fallback) y continuar el proceso sin notificar al usuario.

☐ Aumentar la temperatura del modelo para que intente ser más creativo en el próximo intento.

☐ Reiniciar la cadena completa desde el primer eslabón sin modificar los parámetros.

Explicación: Un error frecuente es tratar el fallo de validación como un error crítico que detiene el sistema. En ingeniería de software de IA, el patrón recomendado es:

1. Detectar el error de validación (ej. falta un campo).
2. Volver a llamar al modelo incluyendo el error específico en el prompt.
3. Esto permite que el modelo "entienda" qué hizo mal y lo corrija, aumentando drásticamente la tasa de éxito de la cadena.

Ignorar el error (A) o simplemente reintentar lo mismo (D) no suele solucionar problemas causados por falta de claridad en el input o en la instrucción.

• Pre-entrega

Pre-entrega 2: Pipeline de procesamiento validado

Briefing para la pre-entrega del Módulo 2: Construcción de un pipeline de procesamiento de texto utilizando LangChain (LCEL) y validación estructurada con Pydantic.

Pre-entrega 2: Pipeline de procesamiento validado

Qué construir

Debes desarrollar un **Pipeline de Extracción de Entidades Técnicas**. El sistema debe recibir un párrafo de texto sin procesar (por ejemplo, una descripción de arquitectura de software o un log de error) y devolver un objeto validado.

Los componentes requeridos:

1. **Esquema Pydantic:** Un modelo que defina campos como `tecnologias` (lista de strings), `nivel_de_criticidad` (enum: baja, media, alta), y `resumen_tecnico` (string).
2. **Prompt Template:** Un template modular que acepte el texto de entrada y las instrucciones de formato.
3. **Cadena LCEL:** Una composición que una el Prompt + LLM + Output Parser.
4. **Lógica de Resiliencia:** Configuración de al menos un reintento automático si el LLM devuelve un JSON mal formado o incompleto.

Pasos sugeridos

1. **Define tu Contrato:** Crea la clase Pydantic. Piensa en qué restricciones quieres poner (ej. que la lista de tecnologías no esté vacía).
2. **Prepara el Parser:** Utiliza `PydanticOutputParser` o el método `.with_structured_output()` de LangChain (preferido para OpenAI/Anthropic).
3. **Ensambla la Cadena:**

```
chain = prompt | model.with_structured_output(TuEsquema)
```

4. **Añade Resiliencia:** Envuelve la llamada con una estrategia de reintento (`.with_retry()`).
5. **Prueba de Estrés:** Pasa un texto ambiguo y verifica si el validador lanza excepciones o si el modelo se recupera.

Errores comunes a evitar

- **Ignorar el `finish_reason`** : A veces el LLM corta la respuesta por falta de tokens. Tu pipeline debe detectar si el objeto está incompleto antes de intentar transformarlo.
 - **Hardcoding de Prompts**: Evita las F-strings de Python dentro de la cadena. Usa `ChatPromptTemplate` para mantener la modularidad y permitir que LangChain gestione las variables de entrada.
-

Práctica

1. Crea un archivo `schemas.py` donde definas la estructura de salida deseada usando Pydantic.
2. En `chain.py`, configura un cliente de ChatOpenAI o ChatAnthropic (reutilizando la lógica del Módulo 1).
3. Crea un `ChatPromptTemplate` que instruya al modelo para extraer información técnica de un texto.
4. Construye la cadena usando LCEL: `prompt | model.with_structured_output(Schema)`.
5. Implementa una función asíncrona `process_text(text: str)` que ejecute la cadena usando `.ainvoke()`.
6. Asegúrate de incluir logs adecuados para observar el proceso de validación y posibles reintentos.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código del pipeline, incluyendo el modelo Pydantic, el prompt template y la cadena LCEL con lógica de reintentos. Debe incluir un mini-script de prueba asíncrono.

Pre-entrega: Pipeline de Procesamiento Validado

Briefing para la pre-entrega del Módulo 2: Construcción de un pipeline de procesamiento de texto utilizando LangChain (LCEL) y validación estructurada con Pydantic.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código del pipeline, incluyendo el modelo Pydantic, el prompt template y la cadena LCEL con lógica de reintentos. Debe incluir un mini-script de prueba asíncrono.

Instrucciones

1. Crea un archivo `schemas.py` donde definas la estructura de salida deseada usando Pydantic.
2. En `chain.py`, configura un cliente de ChatOpenAI o ChatAnthropic (reutilizando la lógica del Módulo 1).
3. Crea un `ChatPromptTemplate` que instruya al modelo para extraer información técnica de un texto.
4. Construye la cadena usando LCEL: `prompt | model.with_structured_output(Schema)`.

- 5. Implementa una función asíncrona `process_text(text: str)` que ejecute la cadena usando `.ainvoke()`.
- 6. Asegúrate de incluir logs adecuados para observar el proceso de validación y posibles reintentos.

Objetivos de aprendizaje

- Componer cadenas lógicas de procesamiento utilizando LangChain Expression Language (LCEL).
- Implementar contratos de datos estrictos mediante modelos de Pydantic para salidas de LLM.
- Configurar estrategias de reintento y manejo de errores en flujos asíncronos de IA.
- Diseñar prompts modulares que se integren en arquitecturas de software escalables.

Rúbrica de Evaluación: Pipeline de Procesamiento Validado con LCEL

Esta rúbrica evalúa la capacidad del estudiante para orquestar flujos de trabajo de IA utilizando LangChain Expression Language (LCEL), validación de datos con Pydantic y ejecución asíncrona, asegurando la escalabilidad y robustez del sistema.

Criterio	Descripción	Peso
Definición del Esquema y Salida Estructurada	Evaluación del uso de Pydantic para definir contratos de datos y la implementación de <code>with_structured_output</code> .	30%
Implementación de Cadena LCEL y Prompting	Uso de la sintaxis de pipe (<code> </code>) de LangChain y diseño de <code>ChatPromptTemplate</code> para la extracción técnica.	30%
Ejecución Asíncrona y Resiliencia	Uso de <code>.ainvoke()</code> y manejo de logs para observar el flujo y la validación.	25%
Estructura del Repositorio y Script de Prueba	Organización del código en módulos (<code>schemas.py</code> , <code>chain.py</code>) y presencia de un script de prueba.	15%

Total: 100 pts | Aprobación: 65 pts

● Clase en vivo

Sesión en vivo 2. Encadenamiento lógico: orquestación con LangChain

Sesión interactiva para consolidar el diseño de cadenas lógicas avanzadas usando LCEL, enfocada en la resiliencia y la orquestación profesional.

Sesión en Vivo: Orquestación Maestra con LangChain y LCEL

¡Bienvenidos a la sesión de cierre del módulo! En esta clase de dos horas, daremos el salto de la teoría a la práctica profesional. Ya has aprendido los fundamentos de **LCEL (LangChain Expression Language)** y cómo

estructurar datos, pero ahora es el momento de integrar todo en un flujo de trabajo orquestado, resiliente y listo para producción.

¿Qué vamos a aprender?

En esta sesión consolidaremos tu capacidad para diseñar arquitecturas de IA robustas. No nos limitaremos a "pegar" prompts; aprenderás a manejar el flujo de datos de manera profesional, implementando lógica de validación y manejo de errores directamente en el pipeline.

Temas clave

- **Dominio de LCEL:** Composición avanzada de Runnables y manipulación de estados intermedios.
- **Estrategias de Resiliencia:** Implementación de *fallbacks* y reintentos para evitar fallos catastróficos.
- **Contratos de Datos:** Cómo forzar que los LLM respeten esquemas estrictos mediante validación estructurada.
- **Orquestación Compleja:** Diseño de cadenas que se adaptan dinámicamente según la entrada del usuario.

Preparación previa

Para aprovechar al máximo esta sesión, asegúrate de:

1. Tener listo tu entorno de desarrollo (Jupyter Notebook o VS Code).
2. Repasar la sintaxis del operador "pipe" (`|`) en LCEL.
3. Revisar tu código de la pre-entrega, ya que lo usaremos como base para un ejercicio de "Stress Test".

Actividades

- **Análisis de "Cadenas Rotas":** Identificaremos por qué fallan arquitecturas mal diseñadas.
- **Design Sprint Grupal:** Diseñarás en equipo una cadena lógica para un caso de uso empresarial complejo.
- **Live Coding Challenge:** Refactorización en tiempo real de una cadena básica a una profesional con reintentos y validación.

Clase en vivo

Materiales de apoyo

Puntos clave:

- LCEL (LangChain Expression Language): Lenguaje declarativo para encadenar componentes de forma estándar. Permite streaming, ejecución paralela y manejo de lotes de forma nativa.
- Runnables: La unidad atómica de LangChain. Todo paso en una cadena (Prompt, Model, Parser) es un Runnable con una interfaz común (invoke, batch, stream).

- **Fallbacks:** Mecanismo de seguridad que permite definir una 'cadena de respaldo' si la principal falla, crucial para la confiabilidad del sistema.
- **Validación Estructurada:** Uso de Pydantic para garantizar que la salida del LLM cumpla con el contrato de datos necesario para el resto del software.

Ejemplos sugeridos:

- **Pipeline de Atención al Cliente:** Una cadena que detecta el idioma, traduce si es necesario, extrae la intención y consulta una base de conocimientos, todo en un flujo LCEL único.
- **Generador de Reportes Financieros:** Uso de validación estructurada para asegurar que el LLM siempre devuelva montos numéricos y fechas válidas, no solo texto.

Herramientas recomendadas:

- **LangSmith:** Herramienta de observabilidad para trazar cada paso de la cadena. Instrucciones: Crear cuenta en smith.langchain.com y configurar variables de entorno.
- **Pydantic v2:** Para la definición de esquemas de datos robustos.

Actividades

- **Debugging de Cadenas Críticas** (20 min, EXERCISE)
 - **Setup:** Compartir un código de una cadena LCEL que falla silenciosamente o produce resultados inconsistentes. **Ejecución:** Los estudiantes deben identificar individualmente los tres errores de diseño (falta de validación, pérdida de contexto en un RunnablePassthrough y falta de manejo de excepciones). Luego, se discuten las soluciones en plenaria. **Cierre:** El instructor muestra la versión corregida 'Golden Path'. **Troubleshooting:** Si los estudiantes no ven el error, dar una pista sobre cómo se transforman los diccionarios entre pasos.
- **Design Sprint: Orquestación Empresarial** (35 min, GROUP_WORK)
 - **Setup:** División en breakout rooms. Cada grupo recibe un caso de uso (Ej: Sistema de clasificación de tickets de soporte que debe extraer sentimiento, categoría y urgencia). **Ejecución:** Los grupos deben dibujar o esquematizar la arquitectura LCEL: ¿Qué Runnables usan? ¿Dónde validan? ¿Dónde ponen el fallback? **Cierre:** Pitch de 2 minutos por grupo explicando su arquitectura. **Dinámica:** Uso de pizarras virtuales (Miro/Jamboard).

Tiempos

Sección	Minutos
Introducción	10
Contenido principal	30
Actividades	55
Preguntas y respuestas	15
Cierre	10
Total	120

• Podcast

Recapitulación: El arte de orquestar con LangChain

Un resumen conversacional para integrar los conceptos de arquitectura LCEL, validación de datos y orquestación resiliente vistos en el módulo.

Este podcast es el cierre del módulo **Encadenamiento lógico: orquestación con LangChain**. Acompaña a nuestros especialistas en una charla que conecta todos los puntos clave que hemos visto: desde la sintaxis declarativa de LCEL hasta la construcción de pipelines resilientes y validados.

¿Qué aprenderás al escucharlo?

- **La gran imagen:** Cómo LCEL transforma la forma en que construimos flujos de IA, pasando de código imperativo a una arquitectura de piezas encajables.
- **Resiliencia y validación:** Por qué los contratos de datos y las estrategias de recuperación (fallbacks) son la diferencia entre un prototipo y un producto profesional.
- **Integración:** Un repaso por el proyecto de pre-entrega y los desafíos discutidos en la sesión en vivo.
- **Mentalidad de arquitecto:** Cómo pensar en términos de "runnables" para crear sistemas escalables.

Si ya completaste las unidades de este módulo, este episodio te ayudará a consolidar esos conocimientos y entender cómo se aplican en conjunto en el mundo real.

MÓDULO 3

Persistencia de datos y vector DBs

Implementar un sistema de memoria de largo plazo mediante arquitecturas RAG básicas y bases de datos vectoriales locales.

● Lectura

Embeddings y similitud

Explora cómo la IA convierte el lenguaje humano en representaciones vectoriales (embeddings) y cómo las métricas de similitud permiten a las máquinas entender la relevancia semántica para construir sistemas de recuperación de información escalables.

Embeddings y similitud

Imagina que intentas buscar un documento sobre "cómo limitar el tráfico de red" en una base de datos tradicional de palabras clave. Si buscas exactamente "limitar tráfico", pero tu documento usa los términos "restricción de ancho de banda" o "rate-limiting", es muy probable que el sistema no lo encuentre. Este es el talón de Aquiles de la búsqueda por palabras clave: la rigidez terminológica.

En esta unidad, aprenderás cómo la Inteligencia Artificial resuelve este problema transformando el lenguaje humano en **geometría**. Al convertir palabras y conceptos en vectores numéricos (embeddings), podemos medir qué tan "cerca" están dos ideas en un espacio multidimensional, permitiendo que nuestras aplicaciones entiendan el significado (semántica) más allá de las letras exactas.

1. ¿Qué es un Embedding?

Un **embedding** es una representación numérica densa de un objeto (usualmente texto, pero también puede ser imagen o audio) en un espacio vectorial de alta dimensionalidad.

En términos más sencillos: es una lista de números (un vector) que captura el "significado" de un fragmento de texto.

El paso de lo discreto a lo denso

Antes de los embeddings modernos, usábamos representaciones "One-Hot Encoding", donde cada palabra era una posición en un vector gigante lleno de ceros. Si tenías un vocabulario de 50,000 palabras, la palabra "perro" era un vector de 50,000 elementos con un solo "1".

- **Problema:** En este sistema, la distancia matemática entre "perro" y "canino" era la misma que entre "perro" y "microprocesador". No había noción de significado.

Los **embeddings** son vectores "densos" (usualmente de entre 1536 y 3072 dimensiones en modelos modernos como los de OpenAI). En este espacio, cada dimensión representa una característica abstracta aprendida por el modelo durante su entrenamiento.

La analogía del Mapa de Significados

Imagina un mapa 2D. Si colocamos "Madrid" en las coordenadas (40, -3) y "Lisboa" en (38, -9), la distancia física entre los puntos refleja su cercanía geográfica. Un embedding hace exactamente lo mismo pero en miles de dimensiones: coloca "Rey" cerca de "Hombre", y "Reina" cerca de "Mujer", manteniendo distancias y relaciones lógicas entre conceptos.

2. La Geometría de la Similitud

Si los textos ahora son puntos en un espacio, ¿cómo calculamos qué tan parecidos son? Aquí es donde entra la matemática vectorial. Existen tres métricas principales que utilizarás como AI Architect:

A. Similitud Coseno (La más utilizada)

Mide el coseno del ángulo entre dos vectores.

- **Qué captura:** La dirección. No importa si un texto es más largo que otro (vector más grande), lo que importa es que "apunten" hacia el mismo concepto.
- **Rango:** De -1 a 1 (en embeddings de texto suele ser de 0 a 1, donde 1 es identidad total).
- **Caso de uso:** Es el estándar en RAG (Retrieval-Augmented Generation) porque ignora la longitud del texto y se enfoca en el contenido temático.

B. Distancia Euclídea (L2)

Mide la distancia en línea recta entre dos puntos.

- **Problema:** Es muy sensible a la magnitud. Si un vector representa un párrafo y otro una oración, la distancia euclídea puede ser grande aunque traten del mismo tema.
- **Uso:** Común cuando los vectores han sido normalizados previamente.

C. Producto Punto (Dot Product)

Multiplica las magnitudes y el coseno del ángulo.

- **Uso:** Muy común en sistemas de recomendación donde la intensidad o "importancia" de un vector (su magnitud) es relevante.

Regla de oro de ingeniería: La mayoría de las bases de datos vectoriales (como ChromaDB o Pinecone) te permiten elegir la métrica. Si estás trabajando con modelos de OpenAI o HuggingFace, la **Similitud Coseno** es casi siempre la elección correcta si no has normalizado los vectores.

3. Del Contexto a la Representación

No todos los embeddings son iguales. La evolución tecnológica ha cambiado cómo extraemos este "significado":

- 1. Modelos Estáticos (Word2Vec, GloVe):** Cada palabra tenía un único vector. La palabra "banco" (de sentarse) y "banco" (entidad financiera) tenían el mismo embedding. El sistema era "ciego" al contexto.
- 2. Modelos Contextuales (BERT, Ada, Voyage):** La misma palabra genera vectores distintos según las palabras que la rodean. El modelo "entiende" que en "me senté en el banco", la palabra se refiere a mobiliario urbano.

El rol de los modelos modernos (OpenAI, Voyage, Cohere)

Como desarrollador, interactuarás con modelos de embeddings a través de APIs. Por ejemplo, el modelo `text-embedding-3-small` de OpenAI toma un string y te devuelve una lista de 1536 flotantes.

Ejemplo conceptual en Python:

```
import openai

# Texto A: "El sistema de archivos está lleno"
# Texto B: "No queda espacio en el disco duro"
# Texto C: "Me gusta comer manzanas"

# El modelo generará vectores tales que:
# sim_coseno(Vector A, Vector B) ≈ 0.85 (Alta similitud semántica)
# sim_coseno(Vector A, Vector C) ≈ 0.15 (Baja similitud semántica)
```

4. Aplicaciones en el Mundo Real

Entender la geometría del lenguaje abre puertas a funcionalidades que antes requerían reglas complejas:

Búsqueda Semántica (Semantic Search)

Empresas como **Notion** o **GitHub** utilizan embeddings para sus buscadores. Si un usuario busca "problemas de pagos", el sistema puede recuperar documentos que mencionen "error en la pasarela de Stripe" o "transacción declinada", aunque no compartan ninguna palabra clave.

Deduplicación Diferida

En sistemas de soporte técnico (como **Zendesk**), se pueden agrupar tickets que preguntan lo mismo ("¿cómo cambio mi clave?" vs "¿procedimiento para resetear password?") comparando sus embeddings y evitando que los agentes respondan lo mismo varias veces.

Clasificación de Zero-Shot

Puedes clasificar texto sin entrenar un modelo. Si tienes categorías como "URGENTE", "SOLICITUD" y "SPAM", simplemente calculas el embedding de cada etiqueta y el del correo entrante. El correo pertenece a la categoría cuyo embedding sea más cercano.

5. Errores Comunes y Mejores Prácticas

Incluso los arquitectos con experiencia suelen caer en estas trampas:

- 1. Tratar los embeddings como "comprensión total":** Los modelos de embeddings capturan correlaciones estadísticas, no lógica pura. Pueden confundir la negación (ej: "Hay stock" y "No hay stock" pueden quedar cerca porque ambos tratan sobre disponibilidad).
- 2. Ignorar el límite de tokens:** Cada modelo de embedding tiene una ventana de contexto. Si intentas embeber un PDF de 100 páginas de una vez, el modelo truncará el texto o perderá precisión. Aquí entra el concepto de *Chunking* (que veremos en la próxima unidad).
- 3. Mezclar modelos: NUNCA** compares un embedding generado con OpenAI con uno generado con un modelo local de HuggingFace. Viven en "universos" (espacios vectoriales) distintos y con dimensiones diferentes. No hay relación matemática entre ellos.
- 4. No considerar el "Domain Shift":** Un modelo entrenado con textos generales de internet puede fallar miserablemente al medir similitudes en lenguaje médico altamente especializado o en código fuente muy técnico.

6. Síntesis y Próximos Pasos

Hemos pasado de ver el texto como una secuencia de caracteres a verlo como un **punto en un espacio geométrico**.

- Los **embeddings** son la traducción del lenguaje a coordenadas numéricas.
- La **similitud coseno** es nuestra herramienta para medir qué tan cerca están dos ideas.
- Esta capacidad es la base de la "Memoria a largo plazo" de cualquier sistema de IA moderno.

En la arquitectura RAG (Retrieval-Augmented Generation), los embeddings son el motor que busca la información relevante. Sin embargo, antes de poder buscar en miles de documentos, necesitamos aprender a "trocearlos" de forma inteligente. En la siguiente unidad, exploraremos las **Estrategias de Chunking**, el arte de dividir el conocimiento para que no se pierda el sentido en la traducción geométrica.

Práctica

Como arquitecto de IA, debes evaluar la capacidad de un modelo de embeddings para discernir entre relevancia semántica y coincidencia de palabras clave.

1. Selecciona un conjunto de 5 oraciones que traten sobre el mismo concepto técnico (ej: "Despliegue de microservicios") pero usen vocabulario totalmente distinto.
2. Incluye 2 oraciones "trampa" que compartan palabras clave pero tengan significados opuestos o irrelevantes (ej: "El servicio de micro-limpieza es excelente").
3. Investiga cómo usarías la librería `scikit-learn` en Python para calcular la **Similitud Coseno** entre dos vectores numéricos.
4. Diseña un diagrama de flujo simple de un proceso de búsqueda semántica: desde que el usuario ingresa una query hasta que el sistema devuelve el documento más parecido.

Entregable: Un documento PDF que contenga el análisis de las oraciones, el fragmento de código (pseudocódigo o Python) para el cálculo de similitud y el diagrama de flujo del proceso.

Entregable

Tipo: FILE

Un documento PDF que contenga el análisis de las oraciones, el fragmento de código (pseudocódigo o Python) para el cálculo de similitud y el diagrama de flujo del proceso.

Embeddings y Similitud: La Geometría del Lenguaje

Explora cómo la IA convierte el lenguaje humano en representaciones vectoriales (embeddings) y cómo las métricas de similitud permiten a las máquinas entender la relevancia semántica para construir sistemas de recuperación de información escalables.

Entregable

Tipo: FILE

Un documento PDF que contenga el análisis de las oraciones, el fragmento de código (pseudocódigo o Python) para el cálculo de similitud y el diagrama de flujo del proceso.

Instrucciones

Como arquitecto de IA, debes evaluar la capacidad de un modelo de embeddings para discernir entre relevancia semántica y coincidencia de palabras clave.

1. Selecciona un conjunto de 5 oraciones que traten sobre el mismo concepto técnico (ej: "Despliegue de microservicios") pero usen vocabulario totalmente distinto.
2. Incluye 2 oraciones "trampa" que compartan palabras clave pero tengan significados opuestos o irrelevantes (ej: "El servicio de micro-limpieza es excelente").
3. Investiga cómo usarías la librería `scikit-learn` en Python para calcular la **Similitud Coseno** entre dos vectores numéricos.

4. Diseña un diagrama de flujo simple de un proceso de búsqueda semántica: desde que el usuario ingresa una query hasta que el sistema devuelve el documento más parecido.

Entregable: Un documento PDF que contenga el análisis de las oraciones, el fragmento de código (pseudocódigo o Python) para el cálculo de similitud y el diagrama de flujo del proceso.

Objetivos de aprendizaje

- Explicar la diferencia entre representaciones de texto discretas (One-Hot) y densas (Embeddings).
- Identificar y seleccionar la métrica de similitud adecuada (Coseno vs Euclídea) según el caso de uso técnico.
- Describir cómo los modelos de embeddings contextuales resuelven el problema de la polisemia en la búsqueda semántica.
- Analizar las limitaciones prácticas de los embeddings, como los límites de tokens y la sensibilidad al dominio.

Evaluación de Embeddings y Similitud: La Geometría del Lenguaje

Esta rúbrica evalúa la capacidad del estudiante para diferenciar entre proximidad sintáctica y semántica, implementar métricas de similitud de vectores y diseñar flujos de búsqueda semántica para sistemas RAG.

Criterio	Descripción	Peso
Selección y Análisis Semántico	Capacidad para proponer ejemplos que desafíen la técnica de embeddings, diferenciando entre sinonimia léxica y trampas de palabras clave.	30%
Implementación de Similitud Coseno	Uso correcto de librerías estándar de Python para el cálculo de distancias o similitudes entre vectores de embeddings.	35%
Arquitectura del Flujo de Búsqueda	Diseño lógico de un pipeline que transforma texto en vectores para realizar una búsqueda semántica eficiente.	25%
Claridad y Calidad del Entregable	Presentación técnica, coherencia en la redacción y formato del documento final.	10%

Total: 100 pts | Aprobación: 65 pts

Código

Estrategias de Chunking y preprocesamiento de documentos

Implementa pipelines de procesamiento de documentos utilizando técnicas de limpieza y Recursive

Character Splitting para optimizar la ingesta en bases de datos vectoriales.

Estrategias de Chunking y preprocesamiento de documentos

En la arquitectura RAG (Generación Aumentada por Recuperación), la calidad de la respuesta del LLM no depende solo del modelo, sino fundamentalmente de la calidad de los datos que recuperamos. Aquí es donde el **Chunking** se vuelve crítico.

¿Por qué necesitamos fragmentar?

Los modelos de lenguaje y, más importante aún, los modelos de **Embedding**, tienen límites de "ventana de contexto". Si intentas convertir un PDF de 50 páginas en un solo vector, perderás todos los matices y detalles. El resultado será un promedio semántico tan diluido que no servirá para encontrar respuestas específicas.

El objetivo del chunking es crear **unidades de información atómicas y semánticamente coherentes**.

1. El Proceso de Limpieza (Preprocessing)

Antes de dividir, debemos limpiar. Los documentos reales (PDFs, HTMLs) suelen traer ruido:

- Encabezados y pies de página repetidos.
- Saltos de línea dobles o triples.
- Caracteres especiales resultantes de una mala extracción de tablas.
- Rutas de archivos o metadatos basura.

Por qué importa: Si un chunk contiene un pie de página en medio de una frase importante, el modelo de embedding capturará ese ruido, afectando la similitud semántica.

2. Estrategias de Chunking

A. Fixed-size (Tamaño Fijo)

Divide el texto cada X caracteres o tokens. Es el método más simple pero el más peligroso, ya que puede cortar una oración o un concepto por la mitad.

B. Recursive Character Splitting (El estándar de industria)

Es el enfoque preferido en LangChain. Intenta dividir el texto usando una jerarquía de separadores:

1. Párrafos (`\n\n`)
2. Saltos de línea (`\n`)
3. Espacios ()
4. Caracteres individuales (como último recurso)

Esto garantiza que, en la medida de lo posible, los párrafos y oraciones se mantengan unidos.

C. Overlap (Solapamiento)

Al dividir, permitimos que el final del "Chunk 1" sea el inicio del "Chunk 2".

- **Motivación:** Imagina que un concepto clave se explica en una frase que queda justo en el límite entre dos fragmentos. El overlap asegura que el contexto se preserve en ambos, facilitando la recuperación sin importar por dónde se "cortó" el papel virtual.

Errores Comunes

1. **Chunks demasiado pequeños:** Generan ruido y pierden contexto global. El LLM tendrá dificultades para entender de qué trata el fragmento.
2. **Ignorar los límites de tokens:** Muchos desarrolladores miden en *caracteres*, pero los modelos cobran y limitan por *tokens*. Siempre valida tus chunks con un tokenizador real (como `tiktoken`).
3. **No limpiar el texto:** "Basura entra, basura sale". Un chunk mal preprocesado reduce la precisión de la búsqueda vectorial.

Síntesis

Un buen chunking es un equilibrio entre:

1. **Granularidad:** Fragmentos pequeños para precisión.
2. **Contexto:** Fragmentos lo suficientemente grandes (u con overlap) para ser comprensibles.
3. **Limpieza:** Datos normalizados para embeddings precisos.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import re
import tiktoken
from typing import List
from langchain_text_splitters import RecursiveCharacterTextSplitter

class DocumentProcessor:
    def __init__(self, model_encoding: str = "cl100k_base"):
        # TODO: Inicializar el encoding de tiktoken
        self.tokenizer = None

        # TODO: Configurar RecursiveCharacterTextSplitter
        # Debe usar una función personalizada para medir longitud por TOKENS, no caracteres.
```

```

        self.splitter = None

    def clean_text(self, text: str) -> str:
        """Limpia el texto eliminando espacios duplicados y saltos de línea innecesarios."""
        # IMPLEMENTA AQUÍ: Usa regex para limpiar el texto
        return text

    def calculate_tokens(self, text: str) -> int:
        """Calcula la cantidad de tokens usando el tokenizer de tiktoken."""
        # IMPLEMENTA AQUÍ
        return 0

    def process_document(self, raw_text: str) -> List[str]:
        """Pipeline: Limpieza -> Fragmentación."""
        # TODO: 1. Limpiar el texto

        # TODO: 2. Aplicar el splitter

        # TODO: 3. Retornar los chunks procesados
        return []

if __name__ == "__main__":
    # Texto de prueba
    text = "Tu texto largo aquí..."
    processor = DocumentProcessor()
    chunks = processor.process_document(text)
    print(f"Chunks generados: {len(chunks)}")

```

Código solución

```

import re
import tiktoken
from typing import List
from langchain_text_splitters import RecursiveCharacterTextSplitter

class DocumentProcessor:
    def __init__(self, model_encoding: str = "cl100k_base"):
        self.tokenizer = tiktoken.get_encoding(model_encoding)
        # Configuramos el splitter recursivo: intenta dividir por párrafos, luego oraciones,
        # luego palabras.
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=50,

```

```

        length_function=self.calculate_tokens,
        separators=["\n\n", "\n", ".", " ", ""]
    )

def clean_text(self, text: str) -> str:
    """Limpia el texto base eliminando ruido innecesario."""
    # Eliminar saltos de línea múltiples y espacios extra
    text = re.sub(r'\s+', ' ', text)
    # Eliminar caracteres especiales repetitivos (comunes en PDFs mal extraídos)
    text = re.sub(r'(\.\s){2,}', '. ', text)
    return text.strip()

def calculate_tokens(self, text: str) -> int:
    """Calcula la cantidad de tokens exactos usando tiktoken."""
    return len(self.tokenizer.encode(text))

def process_document(self, raw_text: str) -> List[str]:
    """Pipeline principal: Limpieza -> Chunking -> Validación."""
    # 1. Limpieza
    cleaned_text = self.clean_text(raw_text)

    # 2. Fragmentación (Chunking)
    # Nota: El splitter usará calculate_tokens para medir el tamaño
    chunks = self.splitter.split_text(cleaned_text)

    # 3. Validación final
    for i, chunk in enumerate(chunks):
        token_count = self.calculate_tokens(chunk)
        print(f"Chunk {i} creado: {token_count} tokens.")

    return chunks

# Ejemplo de ejecución
if __name__ == "__main__":
    sample_text = """
    La inteligencia artificial (IA) es un campo de la informática que se enfoca en crear
    sistemas
    capaces de realizar tareas que normalmente requieren inteligencia humana.

    Este es un segundo párrafo con información adicional sobre el procesamiento de lenguaje
    natural
    y cómo los modelos de lenguaje transforman el mundo. Necesitamos que este texto se divida
    correctamente para ser procesado por un sistema RAG.
    """

```

```
processor = DocumentProcessor()
final_chunks = processor.process_document(sample_text)
print(f"\nTotal de chunks generados: {len(final_chunks)}")
```

Pistas

- Para la limpieza, considera usar expresiones regulares (`re`) para normalizar espacios en blanco y eliminar caracteres no deseados que puedan ensuciar el embedding.
- Al configurar `RecursiveCharacterTextSplitter` , recuerda que `chunk_overlap` ayuda a que la información que queda en el borde de un fragmento no se pierda, apareciendo también al inicio del siguiente. Esto es vital para mantener la coherencia semántica.
- Para la validación de tokens, utiliza la librería `tiktoken` . Recuerda que el 'encoding' debe coincidir con el modelo que planeas usar (por ejemplo, 'cl100k_base' para modelos de OpenAI como GPT-4).

Rúbrica: Estrategias de Chunking y Preprocesamiento de Documentos

Evaluación del diseño e implementación de un pipeline de procesamiento de texto para sistemas RAG, enfocado en la limpieza de datos y la segmentación semántica eficiente.

Criterio	Descripción	Peso
Calidad del Preprocesamiento y Limpieza	Capacidad para eliminar ruido (encabezados, saltos de línea excesivos, caracteres basura) antes del proceso de segmentación.	25%
Lógica de Segmentación (Chunking)	Implementación de estrategias de división del texto priorizando la coherencia semántica sobre el corte arbitrario.	35%
Gestión de Contexto y Overlap	Uso de solapamiento (overlap) para garantizar que los conceptos divididos por el corte mantengan su contexto en los fragmentos adyacentes.	25%
Prácticas de Ingeniería de IA	Uso de herramientas adecuadas (tokenizadores vs caracteres) y legibilidad del código.	15%

Total: 100 pts | Aprobación: 65 pts

Persistencia local con ChromaDB: operaciones CRUD

Aprende a gestionar el ciclo de vida de los datos en ChromaDB local, implementando persistencia en disco y operaciones CRUD para sistemas de memoria en IA.

Persistencia local con ChromaDB: operaciones CRUD

En el desarrollo de sistemas de IA, los **embeddings** (vectores que representan el significado del texto) son volátiles si solo viven en la memoria RAM. Para aplicaciones reales, necesitamos que el conocimiento del sistema "sobreviva" al reinicio del servidor y pueda actualizarse sin reindexar todo desde cero. Aquí es donde entra **ChromaDB**.

¿Por qué ChromaDB local?

ChromaDB es una base de datos vectorial de código abierto diseñada para ser simple. En un entorno de desarrollo o para aplicaciones de escritorio/privadas, su modo **Local Persistente** es ideal:

- No requiere levantar un contenedor Docker complejo (por defecto).
- Guarda los datos en archivos `.parquet` y `bin` en una carpeta de tu elección.
- Permite operaciones CRUD (Create, Read, Update, Delete) sobre el "conocimiento" de tu agente.

Operaciones CRUD en el Espacio Vectorial

A diferencia de una base de datos SQL tradicional, el CRUD en una DB vectorial tiene matices importantes:

- 1. Create (Añadir/Upsert):** No solo guardamos texto, sino su vector asociado. Si el texto cambia, el vector **debe** recalcularse. El uso de `upsert` es una mejor práctica: si el ID ya existe, actualiza; si no, lo crea.
- 2. Read (Get vs Query):**
 - `get()` : Busca por IDs exactos o filtros de metadatos (como un SELECT tradicional).
 - `query()` : Busca por **similitud semántica**. Es el corazón de RAG.
- 3. Update:** Actualiza los metadatos o el documento (disparando una nueva vectorización).
- 4. Delete:** Elimina vectores que ya no son relevantes para el contexto.

El Rol de los Metadatos

Los metadatos son "etiquetas" que adjuntamos a los vectores (ej: `{"autor": "Juan", "fecha": "2024-05", "source": "pdf_v1"}`). Son cruciales para:

- Filtrar resultados antes de la búsqueda vectorial.
- Rastrear el origen de la información que usa el LLM (citación de fuentes).

Errores Comunes

- **ID no determinista:** Usar IDs aleatorios hace imposible el `update` eficiente. Se recomienda usar hashes del contenido o IDs estructurados.
- **Desajuste de Embeddings:** Intentar consultar una base de datos local con un modelo de embeddings diferente al que se usó para crearla. Los resultados serán ruido aleatorio.
- **Ignorar la persistencia:** No inicializar el cliente con una ruta de disco, causando pérdida de datos al cerrar el script.

En este ejercicio, aprenderás a encapsular estas operaciones en una clase robusta preparada para escalar.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import chromadb
from chromadb.utils import embedding_functions
from typing import List, Dict, Any

class VectorMemoryManager:
    def __init__(self, persist_path: str, collection_name: str):
        # TODO: Inicializar el PersistentClient de ChromaDB con la ruta dada
        self.client = None

        # TODO: Crear u obtener la colección usando self.client.get_or_create_collection
        # No olvides asignar una función de embedding por defecto
        self.collection = None

    def upsert_documents(self, ids: List[str], documents: List[str], metadatas: List[Dict[str, Any]]):
        """
        Implementa el método upsert para evitar errores por IDs duplicados.
        """
        # TODO: Usar self.collection.upsert
        pass

    def semantic_search(self, query_text: str, n_results: int = 3):
        """
        Realiza una búsqueda por similitud.
        Retorna los documentos más cercanos al query_text.
        """
```

```

        # TODO: Implementar query() y retornar los resultados
        pass

# Ejemplo de prueba (puedes descomentar para probar localmente)
# manager = VectorMemoryManager("./my_local_db", "test_collection")
# manager.upsert_documents(["1"], ["Texto de prueba"], [{"cat": "general"}])
# print(manager.semantic_search("prueba"))

```

Código solución

```

import chromadb
from chromadb.utils import embedding_functions
from typing import List, Dict, Any

class VectorMemoryManager:
    def __init__(self, persist_path: str, collection_name: str):
        # 1. Configurar persistencia local
        self.client = chromadb.PersistentClient(path=persist_path)

        # Usamos una función de embedding por defecto (Sentence Transformers)
        self.embedding_fn = embedding_functions.DefaultEmbeddingFunction()

        # 2. Obtener o crear la colección
        self.collection = self.client.get_or_create_collection(
            name=collection_name,
            embedding_function=self.embedding_fn
        )

    def upsert_documents(self, ids: List[str], documents: List[str], metadatas: List[Dict[str, Any]]):
        """
        Inserta o actualiza documentos en la colección.
        """
        try:
            self.collection.upsert(
                ids=ids,
                documents=documents,
                metadatas=metadatas
            )
            print(f"Éxito: {len(ids)} documentos procesados.")
        except Exception as e:
            print(f"Error en upsert: {e}")

```

```

def semantic_search(self, query_text: str, n_results: int = 3):
    """
    Realiza una búsqueda por similitud y retorna resultados estructurados.
    """
    results = self.collection.query(
        query_texts=[query_text],
        n_results=n_results,
        include=["documents", "metadatas", "distances"]
    )

    # Formatear la salida para que sea legible
    formatted_results = []
    for i in range(len(results["ids"][0])):
        formatted_results.append({
            "id": results["ids"][0][i],
            "document": results["documents"][0][i],
            "metadata": results["metadatas"][0][i],
            "distance": results["distances"][0][i]
        })
    return formatted_results

def delete_by_id(self, ids: List[str]):
    """Elimina registros específicos."""
    self.collection.delete(ids=ids)

# --- Ejemplo de uso ---
if __name__ == "__main__":
    manager = VectorMemoryManager(persist_path="./chroma_db", collection_name="knowledge_base")

    # Datos de ejemplo
    sample_ids = ["doc1", "doc2"]
    sample_docs = [
        "Python 3.12 introduce mejoras en asyncio y tipos.",
        "Las bases de datos vectoriales son clave para la memoria de largo plazo en IA."
    ]
    sample_meta = [{"source": "news"}, {"source": "tech_blog"}]

    manager.upsert_documents(sample_ids, sample_docs, sample_meta)

    # Búsqueda
    search_query = "mejoras en python"
    matches = manager.semantic_search(search_query)

    for match in matches:

```

```
print(f"Match encontrado: {match['document']} (Score: {match['distance']})")
```

Pistas

- Para la persistencia, utiliza `chromadb.PersistentClient(path="./tu_ruta")` . Esto asegura que los datos se guarden en disco y no se pierdan al cerrar el proceso.
- El método `upsert` es más robusto que `add` . Si intentas insertar un ID que ya existe, `add` lanzará un error, mientras que `upsert` actualizará el registro existente. Es ideal para pipelines de ingesta incremental.
- Al realizar la búsqueda con `collection.query` , asegúrate de incluir `include=["metadatos", "distances", "documents"]` para obtener toda la información necesaria para el usuario final.

Rúbrica de Evaluación: Persistencia Local con ChromaDB

Esta rúbrica evalúa la capacidad del arquitecto de IA para implementar un sistema de persistencia vectorial utilizando ChromaDB, asegurando que las operaciones CRUD sean robustas, eficientes y sigan las mejores prácticas de arquitectura de datos para sistemas RAG.

Criterio	Descripción	Peso
Implementación del Cliente y Persistencia	Evalúa la correcta inicialización de ChromaDB con almacenamiento en disco y la gestión de la colección.	25%
Operaciones CRUD y Estrategia de IDs	Capacidad para insertar, consultar, actualizar y eliminar vectores de forma determinista.	35%
Gestión de Metadatos y Filtrado	Uso de metadatos para enriquecer la búsqueda y permitir filtrado selectivo.	20%
Calidad del Código y Manejo de Errores	Limpieza del código inline, tipado y captura de excepciones específicas de la base de datos.	20%

Total: 100 pts | Aprobación: 65 pts

• Pre-entrega

Pre-entrega 3: Sistema de recuperación semántica local (RAG)

Tercer checkpoint del programa enfocado en la construcción de un sistema RAG (Retrieval-Augmented

Generation) funcional que integra memoria vectorial local con lógica de generación fundamentada.

Pre-entrega 3: Sistema de recuperación semántica local (RAG)

¿Qué debes construir?

Debes entregar un script o notebook de Python que implemente un flujo **End-to-End de RAG**. El sistema debe ser capaz de recibir una consulta del usuario, buscar la información relevante en una base vectorial previamente poblada y generar una respuesta que utilice *exclusivamente* esa información.

Componentes a entregar:

- Módulo de Ingesta (Setup):** Una función que tome un conjunto de documentos (pueden ser archivos `.txt` o `.md` simples), los fragmente (*chunking*) y los persista en una colección de **ChromaDB**.
- Capa de Recuperación (Retriever):** Una lógica que convierta la pregunta del usuario en un embedding y recupere los fragmentos más relevantes.
- Generación Grounded:** Una cadena de **LangChain (LCEL)** que reciba los documentos recuperados y la pregunta, generando una respuesta. El prompt debe instruir al modelo a decir "No lo sé" si la respuesta no está en el contexto.

Pasos sugeridos

- Puebla tu "Cerebro":** Elige 3 o 4 archivos de texto sobre un tema específico (ej. manuales técnicos, apuntes de clase o normativas). Usa las técnicas de *RecursiveCharacterTextSplitter* que vimos en la Unidad 2 para procesarlos.
- Configura ChromaDB:** Inicializa el cliente persistente en una carpeta local (ej. `./vectorstore`). Asegúrate de usar el mismo modelo de embeddings para indexar y para consultar.
- Construye el Prompt de Sistema:** Diseña un prompt que actúe como un "filtro de veracidad". Ejemplo:
"Eres un asistente técnico. Responde solo basándote en el CONTEXTO proporcionado. Si la respuesta no está allí, di que no tienes acceso a esa información."
- Crea la Cadena LCEL:** Une el retriever con el transformador de documentos y el modelo de lenguaje. Recuerda que el output debe pasar por un `PydanticOutputParser`.

Errores comunes a evitar

- El "Contexto Infinito":** No intentes pasar 50 fragmentos al LLM. Esto causará errores de límite de tokens o degradación de la atención del modelo (*Lost in the Middle*). Mantén un `top_k` de entre 3 y 5.
- Embeddings no coincidentes:** Es el error #1. Si indexas tus documentos con `OpenAIEmbeddings` y consultas con `HuggingFaceEmbeddings`, la distancia vectorial no tendrá sentido y los resultados serán aleatorios.
- Falta de persistencia:** Asegúrate de que el script verifique si la base de datos ya existe antes de volver a indexar todo, optimizando así el tiempo y costo de ejecución.

Práctica

1. Crea un entorno virtual e instala las dependencias necesarias: `langchain` , `chromadb` , `openai` (o el provider seleccionado), y `pydantic` .
2. Implementa un script de carga que lea archivos de una carpeta `/data` , aplique un chunking estratégico (mínimo 500 tokens con 50 de overlap) y los guarde en una instancia local de ChromaDB.
3. Define una clase o función asíncrona `get_rag_response(query: str)` que: a. Realice una búsqueda de similitud en ChromaDB. b. Construya un prompt incluyendo los fragmentos recuperados. c. Llame al LLM de forma asíncrona. d. Parsee la respuesta a un modelo Pydantic que incluya el texto y las referencias.
4. Realiza dos pruebas: una pregunta cuya respuesta esté en los documentos y una "pregunta trampa" cuya respuesta no esté, verificando que el modelo no alucine.
5. Sube tu código asegurándote de no incluir llaves de API (usa variables de entorno `.env`).

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga: 1) Script de ingesta de documentos, 2) Script/Notebook con la cadena RAG asíncrona, 3) Un dataset de ejemplo (txt/md) y 4) Un archivo README explicando cómo ejecutar el sistema.

Pre-entrega: Sistema de Recuperación Semántica Local (RAG)

Tercer checkpoint del programa enfocado en la construcción de un sistema RAG (Retrieval-Augmented Generation) funcional que integra memoria vectorial local con lógica de generación fundamentada.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga: 1) Script de ingesta de documentos, 2) Script/Notebook con la cadena RAG asíncrona, 3) Un dataset de ejemplo (txt/md) y 4) Un archivo README explicando cómo ejecutar el sistema.

Instrucciones

1. Crea un entorno virtual e instala las dependencias necesarias: `langchain` , `chromadb` , `openai` (o el provider seleccionado), y `pydantic` .
2. Implementa un script de carga que lea archivos de una carpeta `/data` , aplique un chunking estratégico (mínimo 500 tokens con 50 de overlap) y los guarde en una instancia local de ChromaDB.
3. Define una clase o función asíncrona `get_rag_response(query: str)` que: a. Realice una búsqueda de similitud en ChromaDB. b. Construya un prompt incluyendo los fragmentos recuperados. c. Llame al LLM de forma asíncrona. d. Parsee la respuesta a un modelo Pydantic que incluya el texto y las referencias.

4. Realiza dos pruebas: una pregunta cuya respuesta esté en los documentos y una "pregunta trampa" cuya respuesta no esté, verificando que el modelo no alucine.
5. Sube tu código asegurándote de no incluir llaves de API (usa variables de entorno `.env`).

Objetivos de aprendizaje

- Diseñar una arquitectura RAG completa integrando componentes de recuperación y generación.
- Implementar persistencia vectorial local utilizando ChromaDB y LangChain.
- Garantizar el 'grounding' de las respuestas del LLM mediante el diseño de prompts y contextos restringidos.
- Validar salidas de sistemas RAG utilizando esquemas de Pydantic para asegurar la trazabilidad de fuentes.

Rúbrica de Evaluación: Sistema de Recuperación Semántica Local (RAG)

Evaluación de la implementación de un flujo RAG completo, desde la ingesta de documentos en ChromaDB hasta la respuesta validada con Pydantic y lógica asíncrona.

Criterio	Descripción	Peso
Estrategia de Chunking e Ingesta	Capacidad para procesar documentos de la carpeta /data aplicando técnicas de fragmentación que preserven el contexto semántico.	25%
Arquitectura Asíncrona y Recuperación	Implementación de la función <code>get_rag_response</code> utilizando patrones asíncronos para la consulta y el LLM.	35%
Validación con Pydantic y Esquemas	Uso de modelos Pydantic para garantizar que la salida del sistema sea estructurada y contenga las fuentes.	20%
Robustez y Evidencia de Pruebas	Verificación del comportamiento del sistema ante preguntas fuera de contexto y seguridad de las credenciales.	20%

Total: 100 pts | Aprobación: 65 pts

Sesión en vivo 3. Persistencia de datos y vector DBs

Sesión interactiva final para consolidar estrategias de chunking, gestión de ChromaDB y optimización de sistemas RAG locales.

Sesión en Vivo: Consolidación de Persistencia y Bases de Datos Vectoriales

¡Bienvenido a la sesión de cierre del módulo de Persistencia de Datos! En esta clase de 2 horas, pasaremos de la teoría y el código local a la resolución de problemas reales y el diseño de arquitecturas robustas de memoria para IA.

Esta es una sesión eminentemente práctica y colaborativa diseñada para asegurar que los conceptos de embeddings, chunking y operaciones CRUD en bases vectoriales funcionen de manera armónica en un flujo de trabajo RAG (Retrieval-Augmented Generation).

¿Qué aprenderás?

- A diagnosticar y solucionar problemas comunes en la recuperación semántica (por qué mi modelo no encuentra lo que le pido).
- A elegir la estrategia de chunking adecuada según la estructura del documento y el caso de uso.
- A optimizar las operaciones de persistencia en ChromaDB para aplicaciones escalables.
- A diseñar un sistema de memoria de largo plazo que integre todos los componentes del módulo.

Temas a cubrir

1. **Revisión Crítica de RAG:** Diagnóstico de fallos en la recuperación (Retrieval).
2. **Duelo de Arquitecturas:** Comparativa de estrategias de chunking aplicadas a retos reales.
3. **Persistencia Avanzada:** Gestión de colecciones, metadatos y filtros en bases vectoriales.
4. **Hands-on Group Challenge:** Diseño y prototipado de un sistema de recuperación para un dataset complejo.

Preparación previa

Para aprovechar al máximo esta sesión, asegúrate de:

1. Tener instalado y funcional el entorno de **ChromaDB** en tu computadora.
2. Haber completado la **Pre-entrega 3**, ya que usaremos esos sistemas como base para mejorar la precisión de búsqueda.
3. Traer una pregunta o "bug" que hayas encontrado durante el desarrollo de las unidades de código anteriores.

Actividades

- **Demo Live Coding:** Optimizando la búsqueda mediante el uso de metadatos.
 - **Debate de Diseño:** ¿Fragmentos grandes o pequeños? Análisis de trade-offs.
 - **Taller Grupal:** "The Fixers" – Identificar errores en flujos de persistencia mal implementados.
-

Clase en vivo

Materiales de apoyo

Puntos clave:

- **Vector Database:** Sistema diseñado para almacenar, indexar y buscar vectores de alta dimensionalidad con eficiencia.
- **Semantic Search:** Búsqueda basada en el significado y contexto del lenguaje en lugar de coincidencia exacta de palabras clave.
- **Chunking Strategy:** El proceso de dividir textos largos en fragmentos manejables para no exceder la ventana de contexto del LLM y mantener la relevancia.
- **Overlap:** Técnica de repetir el final de un fragmento al inicio del siguiente para evitar la fragmentación de conceptos a mitad de una frase.

Ejemplos sugeridos:

- **Sistema de recomendación de soporte técnico:** Usar ChromaDB para encontrar soluciones a tickets anteriores basándose en la descripción del problema.
- **LegalTech:** Análisis de contratos donde el chunking por cláusulas es superior al chunking por caracteres fijos.
- **Memoria de largo plazo para Agentes:** Persistir el historial de conversaciones de un usuario para que la IA 'recuerde' preferencias meses después.

Actividades

- **Troubleshooting de Recuperación Semántica** (20 min, DISCUSSION)
 - Los estudiantes analizan tres escenarios donde un sistema RAG devuelve resultados irrelevantes. Deben identificar si el fallo está en la generación de embeddings, el tamaño del chunk o el método de similitud. El profesor guía el análisis usando visualizaciones de los vectores en el espacio latente.
- **Duelo de Estrategias de Chunking** (30 min, GROUP_WORK)
 - Se divide a los estudiantes en salas pequeñas. Cada grupo recibe un tipo de documento 'rebelde' (un contrato legal largo, un manual técnico con listas de pasos o una serie de logs de chat). Deben proponer y justificar el método de chunking y el tamaño de overlap ideal para ese caso específico. Luego presentan su solución al resto de la clase.

Tiempos

Sección	Minutos
Introducción	10
Contenido principal	45
Actividades	50
Preguntas y respuestas	10
Cierre	5
Total	120

• Podcast

Podcast: Recapitulación de Persistencia y Bases de Datos Vectoriales

Un episodio conversacional que recapitula e integra todos los conceptos del módulo, desde embeddings hasta la implementación de sistemas RAG locales.

Resumen del episodio: Integrando la Memoria Semántica

En este episodio de cierre, recapitulamos todo nuestro viaje por el módulo de **Persistencia de datos y vector DBs**. No se trata solo de guardar información, sino de cómo transformar texto en conocimiento estructurado que una IA pueda recuperar de forma eficiente.

¿Qué aprenderás en este episodio?

- Cómo se conectan los **embeddings** con la arquitectura **RAG**.
- La importancia crítica de un buen **chunking** para la calidad de la respuesta.
- Cómo **ChromaDB** actúa como el cemento que une la persistencia con la búsqueda semántica.
- Una visión integral de cómo todas las piezas del módulo encajan para crear sistemas de IA con memoria de largo plazo.

Puntos clave:

1. **Del Texto al Vector:** Repaso de la transformación de datos.
2. **Estrategia de Fragmentación:** Por qué el preprocesamiento define el éxito.
3. **CRUD en Vector DBs:** Más allá de las consultas tradicionales.
4. **Sistemas RAG:** La culminación práctica del módulo.

MÓDULO 4

Escalabilidad documental: RAG avanzado y pinecone

Optimizar la escalabilidad y precisión de sistemas de recuperación utilizando Pinecone y técnicas avanzadas de procesamiento documental.

• Código

Pinecone Serverless: persistencia vectorial en la nube

Aprende a configurar una infraestructura de base de datos vectorial profesional en la nube con Pinecone Serverless y Python.

Pinecone Serverless: persistencia vectorial en la nube

En el módulo anterior trabajamos con **ChromaDB**, una base de datos vectorial excelente para desarrollo local y prototipado rápido. Sin embargo, cuando movemos sistemas de IA a producción, nos enfrentamos a desafíos de **escalabilidad, persistencia distribuida y alta disponibilidad**.

Aquí es donde entra **Pinecone**.

¿Por qué Pinecone en el Stack de un AI Architect?

Pinecone es una base de datos vectorial nativa de la nube, diseñada específicamente para manejar millones (o miles de millones) de vectores con latencias de milisegundos. A diferencia de las bases de datos locales:

- 1. Arquitectura Serverless:** No necesitas gestionar nodos ni memoria. Pagas por lo que usas y el almacenamiento se escala automáticamente.
- 2. Separación de Computación y Almacenamiento:** Permite que las búsquedas sean rápidas sin importar cuántos datos tengas guardados.
- 3. Persistencia Garantizada:** Tus datos viven en la infraestructura administrada de Pinecone, no en tu contenedor local.

Conceptos Clave de Infraestructura

Antes de programar, debemos entender los objetos fundamentales:

- Index (Índice):** Es el contenedor principal. Define la **dimensión** de los vectores (que debe coincidir con tu modelo de embeddings) y la **métrica de similitud** (Coseno, Producto Punto o Euclidiana).
- Nombre de Espacio (Namespace):** Permite segmentar un solo índice en múltiples particiones lógicas. Es vital para implementar multi-tenancy (separar datos de diferentes clientes) o separar entornos (dev, staging, prod) dentro de un mismo índice.

- **Metadatos:** Información adjunta al vector (JSON) que permite realizar filtrados adicionales durante la búsqueda.

El Flujo de Trabajo (Pipeline)

En un entorno profesional, Pinecone no vive solo. Se integra en un pipeline:

1. **Ingesta:** Los documentos se fragmentan (Chunking), se convierten a vectores (Embedding) y se suben (`upsert`) a Pinecone.
2. **Recuperación (Retrieval):** La consulta del usuario se vectoriza, se busca el "vecino más cercano" en Pinecone filtrando por metadatos, y el resultado alimenta al LLM.

Errores Comunes de Arquitectura

- **Mismatch de Dimensiones:** Intentar insertar un vector de 1536 dimensiones en un índice configurado para 768. Esto lanzará un error inmediato.
- **Ignorar los Namespaces:** Mezclar datos de prueba con producción o datos de diferentes usuarios en el mismo espacio lógico, comprometiendo la seguridad y la claridad de los resultados.
- **Elección Errónea de Métrica:** Usar similitud euclidiana cuando el modelo de embeddings fue entrenado específicamente para optimizar la similitud coseno.

En esta unidad, aprenderás a automatizar la creación de esta infraestructura usando el SDK de Python.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import os
import asyncio
from pinecone import Pinecone, ServerlessSpec

# TODO: Asegúrate de tener tu PINECONE_API_KEY en las variables de entorno
PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")

async def setup_vector_infrastructure(index_name: str, dimension: int):
    """
    Configura la infraestructura de Pinecone Serverless y realiza una carga inicial.
    """
    # 1. Inicializar el cliente de Pinecone
    pc = None # TODO: Implementar
```

```

# 2. Crear el índice si no existe
# Tip: Usa pc.list_indexes().names() para verificar existencia
# Tip: Usa ServerlessSpec para definir la nube (aws) y región (us-east-1)

# 3. Conectar al índice
index = None # TODO: Implementar

# 4. Implementar un Upsert de prueba usando un Namespace
# Crea una pequeña lista de tuplas (id, vector, metadata)

# 5. Retornar las estadísticas del índice usando describe_index_stats()
return None

if __name__ == "__main__":
    # Ejemplo con dimensión 1536 (OpenAI)
    asyncio.run(setup_vector_infrastructure("mi-índice-serverless", 1536))

```

Código solución

```

import os
import asyncio
from pinecone import Pinecone, ServerlessSpec

# Configuración de entorno
PINECONE_API_KEY = os.getenv("PINECONE_API_KEY", "tu_api_key")

async def setup_vector_infrastructure(index_name: str, dimension: int):
    # 1. Inicializar cliente
    pc = Pinecone(api_key=PINECONE_API_KEY)

    # 2. Crear índice Serverless si no existe
    if index_name not in pc.list_indexes().names():
        print(f"Creando índice: {index_name}...")
        pc.create_index(
            name=index_name,
            dimension=dimension,
            metric="cosine",
            spec=ServerlessSpec(
                cloud="aws",
                region="us-east-1"
            )
        )

    # Esperar a que el índice esté listo (en serverless es casi instantáneo,

```

```

        # pero en despliegues reales conviene verificar el status)
        while not pc.describe_index(index_name).status['ready']:
            await asyncio.sleep(1)

# 3. Conectar al índice
index = pc.Index(index_name)

# 4. Operación CRUD: Upsert con Namespace
# Simulamos vectores (en un caso real vendrían de un modelo de embeddings)
sample_vectors = [
    ("vec1", [0.1] * dimension, {"topic": "infraestructura", "priority": "high"}),
    ("vec2", [0.2] * dimension, {"topic": "seguridad", "priority": "medium"})
]

print(f"Insertando vectores en namespace 'dev-environment'...")
index.upsert(vectors=sample_vectors, namespace="dev-environment")

# 5. Verificación
stats = index.describe_index_stats()
print(f"Estado del índice: {stats}")
return index

if __name__ == "__main__":
    # Dimension 1536 es estándar para text-embedding-3-small de OpenAI
    asyncio.run(setup_vector_infraestructure("ai-architect-index", 1536))

```

Pistas

- Para crear un índice serverless, utiliza `pc.create_index()`. Asegúrate de especificar `spec=ServerlessSpec(cloud='aws', region='us-east-1')` (o la región que prefieras). El nombre del índice debe ser único en tu proyecto.
- Al realizar el `upsert`, recuerda que Pinecone espera una lista de tuplas o diccionarios. Cada elemento debe tener un `id` único y los `values` (el vector de embeddings). Opcionalmente, puedes incluir un diccionario de `metadata`.
- Para trabajar con namespaces, obtén la instancia del índice con `pc.Index(index_name)` y luego usa el método `.upsert(vectors=..., namespace='tu-namespace')`. El namespace ayuda a segmentar datos dentro del mismo índice (por ejemplo, por cliente o tipo de documento).

Evaluación de Configuración e Ingesta en Pinecone Serverless

Esta rúbrica evalúa la capacidad del estudiante para configurar infraestructuras vectoriales en la nube, gestionando de forma eficiente la creación de índices, la segmentación mediante namespaces y el manejo de metadatos bajo una arquitectura serverless.

Criterio	Descripción	Peso
Configuración de Infraestructura de Índice	Evaluación de la correcta inicialización del índice de Pinecone, asegurando la correspondencia entre dimensiones, métricas de similitud y el modelo de embeddings.	30%
Estrategia de Namespaces y Segmentación	Capacidad para organizar la información dentro del índice para soportar multi-tenancy o separación de entornos.	25%
Enriquecimiento con Metadatos	Uso de metadatos en los vectores para permitir filtrado estructurado durante la recuperación.	25%
Eficiencia en la Ingesta (Upsert)	Implementación técnica del proceso de carga de datos en Pinecone.	20%

Total: 100 pts | Aprobación: 65 pts

• Código

Ingesta masiva y gestión de metadatos avanzados

Aprende a construir pipelines de datos robustos utilizando procesamiento por lotes y metadatos enriquecidos para búsquedas vectoriales filtradas y seguras.

Ingesta masiva y gestión de metadatos avanzados

En un sistema RAG de producción, no basta con "subir archivos". La capacidad de respuesta y la precisión del sistema dependen crucialmente de cómo estructuramos los datos en el momento de la ingesta. Cuando pasamos de 100 documentos a 1,000,000, el **filtrado por metadatos** y el **procesamiento por lotes (batching)** dejan de ser extras para convertirse en requisitos de arquitectura.

El Corazón de la Escalabilidad: Ingesta Distribuida y Batching

Ingresar datos uno por uno (`vector a vector`) es ineficiente debido al overhead de red y los límites de la API de Pinecone. La estrategia correcta es el **Batch Upsert**.

- **¿Por qué batches?:** Reduce latencia de red y aprovecha el paralelismo interno de la base de datos vectorial.

- **Tamaño ideal:** Generalmente, batches de entre 100 y 200 vectores son el "punto dulce" entre velocidad y estabilidad.

Metadatos: El Índice Selectivo para la IA

Si buscas "factura de Amazon" en un mar de 10 millones de documentos, la búsqueda semántica pura puede darte resultados de "facturas de Azure" o "recibos de Walmart" porque son semánticamente similares. Los **metadatos** permiten aplicar un filtro rígido (Hard Filter) antes o durante la búsqueda semántica.

En Pinecone, los metadatos se almacenan junto al vector y permiten consultas SQL-like. Algunos casos de uso avanzados incluyen:

1. **Multi-tenancy:** Filtrar por `user_id` o `org_id` para asegurar que un usuario nunca vea datos de otro.
2. **Temporalidad:** Filtrar por `year > 2023` para evitar información obsoleta.
3. **Categorización:** Limitar el contexto a `department == 'legal'`.

El Riesgo del "Schema Drift"

Un error común es no estandarizar los metadatos. Si un proceso guarda la fecha como `ingest_date` y otro como `date_created`, tus filtros fallarán. Como arquitecto, debes definir una **interfaz de metadatos** estricta.

Ejemplo de Filtro en Pinecone (Conceptual)

```
# Queremos buscar similitud solo en documentos de la categoría 'legal'
index.query(
    vector=[0.1, 0.2, ...],
    top_k=10,
    filter={
        "category": {"$eq": "legal"},
        "status": {"$in": ["reviewed", "final"]}
    }
)
```

Este enfoque reduce drásticamente el espacio de búsqueda, mejorando la latencia y eliminando el "ruido" de categorías irrelevantes.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import asyncio
import uuid
from datetime import datetime
from typing import List, Dict, Any

# Mock del índice de Pinecone para testing
class MockPineconeIndex:
    async def upsert(self, vectors: List[Dict]):
        # Simula la inserción en la base de datos
        return {"upserted_count": len(vectors)}

    async def query(self, vector: List[float], top_k: int, filter: Dict = None):
        # Simula una consulta con filtro
        return {"matches": []}

class IngestionPipeline:
    def __init__(self, index: MockPineconeIndex):
        self.index = index

    def create_metadata(self, doc_text: str, category: str, author: str) -> Dict[str, Any]:
        """
        TODO: Implementa la creación de un diccionario de metadatos.
        Debe incluir: category, author, ingested_at (ISO format) y el tamaño del texto.
        """
        pass

    async def process_and_upsert_batches(self, documents: List[Dict], batch_size: int = 100):
        """
        TODO: Implementa la lógica de batching.
        1. Divide la lista 'documents' en trozos de tamaño 'batch_size'.
        2. Para cada batch, prepara la lista de vectores (id, values, metadata).
        3. Llama a self.index.upsert() por cada batch.
        """
        pass

    async def search_by_category(self, query_vector: List[float], category: str):
        """
        TODO: Implementa una consulta que use un filtro de metadatos.
        Usa el operador "$eq" de Pinecone para la categoría.
        """
        pass

    async def main():
```

```

index = MockPineconeIndex()
pipeline = IngestionPipeline(index)

# Simulación de documentos
docs = [
    {"text": "Contenido A", "category": "legal", "author": "User1"},
    {"text": "Contenido B", "category": "hr", "author": "User2"}
]

# 1. Ejecutar ingesta
# 2. Ejecutar búsqueda filtrada
pass

if __name__ == "__main__":
    asyncio.run(main())

```

Código solución

```

import asyncio
import uuid
from datetime import datetime
from typing import List, Dict, Any

# Simulamos las dependencias
class MockPineconeIndex:
    async def upsert(self, vectors: List[Dict]):
        print(f"[Pinecone] Insertando batch de {len(vectors)} vectores...")
        return {"upserted_count": len(vectors)}

    async def query(self, vector: List[float], top_k: int, filter: Dict = None):
        print(f"[Pinecone] Consultando con filtro: {filter}")
        return {"matches": [{"id": "doc_1", "score": 0.98, "metadata": filter}]}

class IngestionPipeline:
    def __init__(self, index: MockPineconeIndex):
        self.index = index

    def create_metadata(self, doc_text: str, category: str, author: str) -> Dict[str, Any]:
        """
        Genera metadatos enriquecidos para un documento.
        """
        return {
            "text": doc_text[:100], # Guardamos snippet para referencia rápida

```

```

        "category": category,
        "author": author,
        "ingested_at": datetime.utcnow().isoformat(),
        "char_count": len(doc_text)
    }

async def process_and_upsert_batches(self, documents: List[Dict], batch_size: int = 2):
    """
    Procesa documentos en batches para optimizar la ingesta masiva.
    """
    total_upserted = 0
    for i in range(0, len(documents), batch_size):
        batch = documents[i : i + batch_size]
        vectors_to_upsert = []

        for doc in batch:
            # Simulamos un embedding (dimensión 3 para el ejemplo)
            dummy_embedding = [0.1, 0.2, 0.3]

            vectors_to_upsert.append({
                "id": str(uuid.uuid4()),
                "values": dummy_embedding,
                "metadata": self.create_metadata(doc["text"], doc["category"],
doc["author"])
            })

            result = await self.index.upsert(vectors_to_upsert)
            total_upserted += result["upserted_count"]

        return total_upserted

async def search_by_category(self, query_vector: List[float], category: str):
    """
    Ejemplo de búsqueda con filtrado por metadatos.
    """
    return await self.index.query(
        vector=query_vector,
        top_k=5,
        filter={"category": {"$eq": category}}
    )

# Flujo de ejecución
async def main():
    index = MockPineconeIndex()
    pipeline = IngestionPipeline(index)

```

```

# Datos de ejemplo
raw_docs = [
    {"text": "Reporte financiero Q1", "category": "finance", "author": "Alice"},
    {"text": "Manual de ingeniería React", "category": "tech", "author": "Bob"},
    {"text": "Update de producto Mayo", "category": "tech", "author": "Alice"},
    {"text": "Análisis de riesgo", "category": "finance", "author": "Charlie"}
]

print("--- Iniciando Ingesta Masiva ---")
count = await pipeline.process_and_upsert_batches(raw_docs, batch_size=2)
print(f"Total procesado: {count} documentos.\n")

print("--- Probando Búsqueda Filtrada ---")
search_results = await pipeline.search_by_category([0.1, 0.2, 0.3], "tech")
print(f"Resultados: {search_results}")

if __name__ == "__main__":
    asyncio.run(main())

```

Pistas

- Para la gestión de metadatos, utiliza un diccionario que incluya campos técnicos (como `id` o `timestamp`) y de negocio (como `category`). En Pinecone, estos se pasan como el campo `metadata` en cada vector.
- El proceso de batching es crucial para no saturar la API. Divide tu lista total de documentos en trozos más pequeños (ej. 10 a 100 documentos por vez) y realiza el `upsert` dentro de un bucle.
- Para el filtrado, recuerda que Pinecone permite pasar un argumento `filter` en la función `query`. Este filtro utiliza operadores como `$eq` (igual a) o `$in` (contenido en).

Evaluación: Ingesta Masiva y Gestión de Metadatos en Pinecone

Esta rúbrica evalúa la capacidad del arquitecto para diseñar procesos de ingesta de datos a gran escala hacia Pinecone, enfocándose en la eficiencia del procesamiento por lotes (batching) y la precisión de la recuperación mediante filtros de metadatos robustos.

Criterio	Descripción	Peso
Estrategia de Batching y Escalabilidad	Evalúa la implementación de técnicas de carga por lotes para optimizar la latencia y estabilidad del sistema.	35%
Diseño y Filtrado de Metadatos	Mide la capacidad de estructurar y consultar metadatos para mejorar la precisión de la recuperación semántica.	30%
Arquitectura y Calidad de Código	Evalúa la legibilidad, mantenibilidad y robustez de la solución propuesta en el entorno inline.	20%
Previsión de Casos Borde y Seguridad	Valida si se consideran aspectos como la concurrencia, límites de API o aislamiento de datos.	15%

Total: 100 pts | Aprobación: 65 pts

● Tarjetas

Flashcards: Métricas y recuperación híbrida

Unidad de repaso sobre las métricas críticas de evaluación para sistemas de búsqueda y los conceptos fundamentales de la arquitectura de recuperación híbrida.

Flashcards: Métricas y recuperación híbrida

En sistemas RAG de alta escala, la calidad de la respuesta depende totalmente de la relevancia de los documentos recuperados. Como arquitectos de IA, no basta con "conectar" una base de datos vectorial; debemos entender cómo evaluar y optimizar el proceso de recuperación para garantizar que no estemos entregando "ruido" al LLM.

Esta unidad se centra en los pilares fundamentales para medir y mejorar la recuperación:

- 1. Métricas de Calidad:** Entender cuándo estamos recuperando documentos útiles (Precisión) y cuándo estamos omitiendo documentos clave (Recall).
- 2. Recuperación Híbrida:** Superar las limitaciones de los embeddings (semántica) combinándolos con la robustez de la búsqueda léxica (BM25).
- 3. Fusión y Re-ranking:** Estrategias como RRF para combinar resultados heterogéneos y modelos de re-ranking (cross-encoders) para un refinamiento final de alta precisión.

Cómo utilizar estas tarjetas

- **Intenta responder mentalmente** antes de hacer clic para voltear la tarjeta.
- **Visualiza el escenario:** Si una tarjeta menciona **Precision**, piensa en un sistema de búsqueda donde el usuario espera resultados exactos y no quiere basura.

- **Enfócate en la relación:** No memorices las fórmulas como algo abstracto; entiende qué fenómeno del mundo real intenta capturar cada métrica.

¡Comencemos el repaso!

Flashcards

1. Frente: ¿Cuál es la diferencia fundamental entre Precision y Recall en un sistema RAG y por qué ambos son críticos?

Reverso: La **Precisión** mide cuánto de lo que recuperó el sistema es realmente relevante (evita resultados irrelevantes). El **Recall** mide qué proporción de todos los documentos relevantes existentes fue encontrada (evita dejar información útil fuera). En RAG, un recall bajo causa alucinaciones por falta de contexto, mientras que una precisión baja introduce ruido que confunde al LLM.

2. Frente: ¿Por qué los motores de búsqueda modernos combinan BM25 con búsqueda semántica (embeddings)?

Reverso: **BM25** se basa en coincidencias exactas de términos (frecuencia de palabras, rareza). Es excelente para nombres propios, códigos de error o términos técnicos. Los **Embeddings** capturan el significado conceptual y la intención del usuario, manejando sinónimos y variaciones lingüísticas que BM25 ignoraría. La búsqueda híbrida captura lo mejor de ambos mundos.

3. Frente: ¿Qué es el RRF y por qué es la estrategia preferida para fusionar resultados en una búsqueda híbrida?

Reverso: El **Reciprocal Rank Fusion (RRF)** combina rankings de diferentes fuentes (ej. BM25 y Vectores) basándose en las posiciones (rank) de los documentos en lugar de sus puntuaciones absolutas. Es robusto porque no requiere normalizar puntuaciones que provienen de sistemas con escalas totalmente distintas.

4. Frente: ¿Cuál es el propósito del Re-ranking en un pipeline de recuperación y en qué se diferencia del retrieval inicial?

Reverso: El **Re-ranking** es una segunda etapa donde un modelo (generalmente un *cross-encoder* más pesado) evalúa con mayor precisión el subconjunto de documentos devueltos por la primera etapa de recuperación. Permite mayor exactitud sin sacrificar la latencia de todo el sistema, ya que se aplica solo a un pequeño número de candidatos (top-k).

5. Frente: Defina $\text{Recall}@k$ y explique su importancia específica en el contexto de un sistema RAG.

Reverso: $\text{Recall}@k$ mide si al menos un documento relevante está incluido dentro de los primeros k resultados recuperados. Es la métrica reina para evaluar si nuestro sistema es capaz de "entregar el contexto necesario" al LLM de forma consistente.

6. Frente: ¿Cuál es el error conceptual más común al implementar una búsqueda híbrida "a mano"?

Reverso: Un error frecuente es intentar sumar directamente las puntuaciones (scores) de BM25 y de similitud vectorial. Esto no funciona porque sus escalas son heterogéneas (ej. un score de 20 en BM25 no equivale a una similitud coseno de 0.8). Es indispensable normalizar o usar algoritmos de fusión como RRF.

7. Frente: ¿Qué caracteriza a un Cross-encoder y por qué no se usa para la etapa de retrieval inicial?

Reverso: El **Cross-encoder** procesa el par (query, documento) conjuntamente desde el inicio para calcular la relevancia. Es extremadamente preciso porque entiende interacciones profundas entre palabras, pero es demasiado lento para buscar en millones de documentos, por lo que se reserva estrictamente para la fase de re-ranking.

8. Frente: ¿Qué nos indica el F1-Score en la evaluación de un sistema de recuperación?

Reverso: El **F1-Score** es la media armónica entre precisión y recall. Se utiliza para encontrar un equilibrio único que resuma el rendimiento del sistema. Es especialmente útil en datasets donde no podemos permitirnos ni demasiada relevancia baja ni demasiada omisión de datos.

• Pre-entrega

Pre-entrega 4: Sistema RAG escalable en la nube con Pinecone

Guía detallada para la pre-entrega del Módulo 4, donde el estudiante debe escalar su arquitectura RAG de un entorno local a Pinecone Cloud, implementando búsqueda híbrida y evaluación de métricas.

Pre-entrega 4: Sistema RAG escalable en la nube con Pinecone

Qué construir

Debes entregar un **Módulo de Recuperación Escalable** integrado en un repositorio de código. El artefacto principal es un servicio (o conjunto de scripts organizados) en Python que ejecute el flujo completo de un sistema RAG en la nube.

Componentes obligatorios:

- 1. Pipeline de Ingesta en Pinecone:** Un script que tome un conjunto de documentos (PDFs, Markdown o JSON), los procese y los suba a un índice de Pinecone Serverless utilizando metadatos avanzados (fuente, página, etiquetas de categoría).
- 2. Recuperador Híbrido (Hybrid Retriever):** Una implementación que combine búsqueda por similitud de vectores con búsqueda léxica (BM25) para mejorar la precisión en términos técnicos o nombres propios.
- 3. Script de Evaluación:** Una utilidad que calcule al menos dos métricas fundamentales (Precision@k y Recall@k) utilizando un pequeño "Golden Set" de preguntas y respuestas de prueba.

Pasos sugeridos

- 1. Preparación de Infraestructura:** Crea un índice *Serverless* en Pinecone (usa la dimensión 1536 si usas OpenAI `text-embedding-3-small`).
- 2. Ingesta Inteligente:** No subas el texto plano. Crea un esquema donde guardes el texto original dentro de los metadatos de Pinecone para evitar consultas adicionales a una base de datos relacional.
- 3. Configuración de LangChain:** Utiliza el `PineconeVectorStore` de LangChain o el SDK nativo de Pinecone para configurar el motor de búsqueda.
- 4. Implementación BM25:** Configura un recuperador de LangChain que use `BM25Retriever` y combínalo con el de Pinecone usando un `EnsembleRetriever` .
- 5. Evaluación Local:** Crea un pequeño archivo JSON con pares `{"pregunta": "...", "documento_id_esperado": "..."}` y mide cuántos de esos documentos aparecen efectivamente en el Top-5 recuperado.

Errores comunes a evitar

- **Mismatch de Dimensiones:** Intentar subir embeddings de 1536 dimensiones a un índice configurado con 512 o 768.
 - **Ignorar el Namespace:** En aplicaciones multi-inquilino o con distintos tipos de datos, no usar `namespaces` en Pinecone hará que la búsqueda sea ruidosa y lenta.
 - **Subestimar el Chunking:** Chunks muy pequeños pierden el contexto semántico; muy grandes diluyen la precisión del embedding. Busca un punto medio (~500–800 tokens).
-

Práctica

- 1. Configuración de variables:** Crea un archivo `.env` con `PINECONE_API_KEY` , `OPENAI_API_KEY` (o Anthropic) e `INDEX_NAME` .
- 2. Setup de Pinecone:** Escribe un script de inicialización que verifique si el índice existe y lo cree si es necesario (modo *Serverless*).
- 3. Pipeline de Ingesta:**
 - Carga un dataset de documentos técnicos (puedes usar la documentación de una librería de Python).
 - Divide en chunks usando un `RecursiveCharacterTextSplitter` .
 - Genera embeddings e insértalos en Pinecone incluyendo el contenido y la fuente en la metadata.
- 4. Implementación del Recuperador:**
 - Crea una clase `RAGSystem` que encapsule un `EnsembleRetriever` .
 - El sistema debe recibir una consulta y devolver los top-5 documentos combinando resultados léxicos y semánticos.
- 5. Evaluación:**

- Crea un script `evaluate.py`.
- Define un benchmark de 5 preguntas donde conozcas de antemano el documento fuente.
- Ejecuta las consultas y calcula:
 - **Recall@5**: ¿Está el documento correcto entre los 5 recuperados?
 - **Precision@5**: ¿Qué porcentaje de los 5 recuperados son realmente útiles?

6. Reporte: Imprime en consola un breve resumen de los resultados de evaluación.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga el pipeline de ingesta, el recuperador híbrido y el script de evaluación de métricas. El README debe incluir instrucciones para replicar el índice de Pinecone.

Pre-entrega: Sistema RAG Escalable en la Nube con Pinecone

Guía detallada para la pre-entrega del Módulo 4, donde el estudiante debe escalar su arquitectura RAG de un entorno local a Pinecone Cloud, implementando búsqueda híbrida y evaluación de métricas.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga el pipeline de ingesta, el recuperador híbrido y el script de evaluación de métricas. El README debe incluir instrucciones para replicar el índice de Pinecone.

Instrucciones

- 1. Configuración de variables:** Crea un archivo `.env` con `PINECONE_API_KEY`, `OPENAI_API_KEY` (o Anthropic) e `INDEX_NAME`.
- 2. Setup de Pinecone:** Escribe un script de inicialización que verifique si el índice existe y lo cree si es necesario (modo Serverless).
- 3. Pipeline de Ingesta:**
 - Carga un dataset de documentos técnicos (puedes usar la documentación de una librería de Python).
 - Divide en chunks usando un `RecursiveCharacterTextSplitter`.
 - Genera embeddings e insértalos en Pinecone incluyendo el contenido y la fuente en la metadata.
- 4. Implementación del Recuperador:**
 - Crea una clase `RAGSystem` que encapsule un `EnsembleRetriever`.
 - El sistema debe recibir una consulta y devolver los top-5 documentos combinando resultados léxicos y semánticos.
- 5. Evaluación:**

- Crea un script `evaluate.py`.
- Define un benchmark de 5 preguntas donde conozcas de antemano el documento fuente.
- Ejecuta las consultas y calcula:
 - **Recall@5**: ¿Está el documento correcto entre los 5 recuperados?
 - **Precision@5**: ¿Qué porcentaje de los 5 recuperados son realmente útiles?

6. Reporte: Imprime en consola un breve resumen de los resultados de evaluación.

Objetivos de aprendizaje

- Implementar una base de datos vectorial en la nube con Pinecone Serverless.
- Diseñar estrategias de ingesta masiva con metadatos estructurados para filtrado.
- Construir un sistema de recuperación híbrida combinando BM25 y búsqueda de vectores.
- Evaluar el rendimiento del sistema de recuperación mediante métricas de Precision y Recall.

Rúbrica de Evaluación: Sistema RAG Escalable con Pinecone

Esta evaluación mide la capacidad del estudiante para diseñar e implementar un pipeline de recuperación de información escalable en la nube, integrando técnicas de búsqueda híbrida y evaluando cuantitativamente el rendimiento del sistema mediante métricas de recuperación.

Criterio	Descripción	Peso
Configuración e Infraestructura Cloud (Pinecone)	Evaluación de la correcta inicialización del índice serverless y la gestión de variables de entorno de forma segura.	15%
Pipeline de Ingesta y Gestión de Metadatos	Capacidad para procesar documentos técnicos, fragmentarlos estratégicamente y persistirlos con metadatos útiles.	25%
Implementación del Recuperador Híbrido	Diseño de la clase RAGSystem utilizando EnsembleRetriever para combinar búsqueda semántica y léxica.	35%
Evaluación Cuantitativa de Métricas	Implementación del script evaluate.py para calcular Recall@5 y Precision@5 basado en un benchmark.	25%

Total: 100 pts | Aprobación: 65 pts

• Clase en vivo

Sesión en vivo 4. Escalabilidad documental: RAG avanzado y pinecone

Sesión práctica e interactiva para consolidar el diseño de sistemas RAG escalables, optimizar la recuperación híbrida y resolver desafíos reales de producción con Pinecone.

Sesión en Vivo: Masterclass de RAG Avanzado con Pinecone

¡Bienvenidos a la sesión de cierre del módulo! En esta clase en vivo, daremos el salto de la teoría y los scripts locales a la construcción de arquitecturas de recuperación de grado industrial.

A lo largo del módulo, has explorado cómo **Pinecone Serverless** permite manejar millones de vectores y cómo la **recuperación híbrida** mejora la precisión. Hoy, pondremos todo ese conocimiento en práctica enfrentando un escenario real de escalabilidad.

¿Qué aprenderás hoy?

- **Arquitectura de Producción:** Cómo diseñar flujos que no se rompan cuando pasamos de 1,000 a 1,000,000 de documentos.
- **Optimización de Query Pipeline:** Técnicas avanzadas como *Query Expansion* y *Re-ranking* para superar las limitaciones de la búsqueda semántica simple.
- **Debugging de Relevancia:** Aprenderás a identificar por qué un sistema RAG "miente" o no encuentra la información, y cómo arreglarlo usando metadatos.

Temas clave a cubrir

1. **Repaso Estratégico:** Por qué *Serverless* cambia las reglas del juego en costos y latencia.
2. **Filtrado con Metadatos:** El arte de particionar el índice para búsquedas ultra-precisas.
3. **Evaluación de Recuperación:** Métricas prácticas más allá del "parece que funciona".
4. **Hands-on:** Resolución de un caso de cuello de botella en ingesta masiva.

Preparación previa

Para aprovechar al máximo esta sesión, asegúrate de:

1. Tener acceso a tu consola de **Pinecone** y tu API Key a la mano.
2. Haber completado la **Pre-entrega del Sistema RAG**, ya que usaremos esa base para los ejercicios de optimización.
3. Traer una duda o "dolor" puntual que hayas encontrado al trabajar con la persistencia vectorial.

Actividades principales

- **Desafío de Arquitectura:** En grupos, diseñarán una solución para un cliente con 50TB de documentación técnica.

- **Live Debugging:** Analizaremos un flujo RAG "roto" y lo optimizaremos en tiempo real.
 - **Q&A de Cierre:** Espacio abierto para resolver dudas técnicas sobre tu proyecto final y tendencias del sector.
-

Clase en vivo

Materiales de apoyo

Puntos clave:

- **Namespaces vs Metadata:** Los Namespaces son para aislar datos (multitenancy); los Metadatos son para filtrar dentro de un segmento de datos.
- **Pinecone Serverless:** Elimina la necesidad de dimensionar pods, cobrando por lectura (read units) y escritura (write units). Es crucial para la escalabilidad.
- **Re-ranking:** El proceso de tomar el Top 20 de Pinecone y re-ordenarlo con un modelo más potente (pero lento) para quedarse con el Top 5.

Ejemplos sugeridos:

- Caso LegalTech: Filtrado de millones de documentos por 'Año de Sentencia' y 'Juzgado' antes de hacer la búsqueda vectorial.
- Efecto Alucinación: Cómo un mal retrieval confunde al LLM enviándole el Manual de Usuario equivocado pero semánticamente parecido.

Herramientas recomendadas:

- Pinecone Python Client (última versión)
- LangChain o LlamaIndex para orquestación de RAG avanzado
- Cross-Encoders de HuggingFace para re-ranking funcional

Actividades

- **Live Debugging: El Problema de la Relevancia** (25 min, DEMO)
 - **Setup:** El instructor presenta un caso de 'Recuperación Fallida': el sistema devuelve documentos semánticamente similares pero contextualmente irrelevantes. **Ejecución:** Los estudiantes proponen en el chat por qué creen que falla (bad chunking, falta de metadatos, etc.). El instructor muestra cómo aplicar un filtro de metadatos complejo y un paso de Cross-Encoder Re-ranking para 'limpiar' los resultados. **Cierre:** Comparativa de 'Antes vs Después' en términos de precisión. **Troubleshooting:** Si el código falla, centrarse en la explicación conceptual de los metadatos.
- **Desafío de Arquitectura: Millones de Vectores** (35 min, GROUP_WORK)

- **Setup:** División en breakout rooms. Se entrega un escenario: 'Empresa de LegalTech con 10M de contratos necesita búsqueda por jurisdicción, fecha y tema con latencia <200ms'. **Dinámica:** Los estudiantes deben proponer: 1) Estrategia de Namespaces vs Metadatos en Pinecone. 2) Modelo de embeddings. 3) Plan de ingesta masiva. **Cierre:** Una persona por grupo comparte la decisión clave de su arquitectura. **Troubleshooting:** El instructor debe circular por las salas para evitar que se pierdan en detalles de código y se enfoquen en la arquitectura.

Tiempos

Sección	Minutos
Introducción	10
Contenido principal	30
Actividades	60
Preguntas y respuestas	15
Cierre	5
Total	120

• Podcast

Resumen del módulo: Escalabilidad documental, RAG avanzado y Pinecone

Un repaso conversacional profundo sobre cómo construir y optimizar sistemas RAG a gran escala utilizando Pinecone y técnicas avanzadas de procesamiento documental.

En este episodio final del módulo, hacemos un recorrido integral por las piezas que componen un sistema de Inteligencia Artificial capaz de manejar millones de documentos sin perder velocidad ni precisión. Analizamos la arquitectura de **Pinecone Serverless**, la importancia de una ingesta masiva bien estructurada y cómo las métricas de recuperación determinan el éxito de un RAG en producción.

Escuchando este podcast, consolidarás tu visión sobre cómo pasar de un prototipo local a una infraestructura documental robusta, entendiendo que la escalabilidad no es solo "hacerlo más grande", sino diseñar con eficiencia y modularidad.

MÓDULO 5

Razonamiento autónomo: introducción de agentes con LangGraph

Desarrollar agentes capaces de razonar cíclicamente y utilizar herramientas externas mediante el paradigma de grafos de LangGraph.

• Lectura

Arquitectura de grafos: nodos, aristas y estado en LangGraph

Aprende a estructurar flujos de IA avanzados utilizando LangGraph, dominando los pilares de nodos, aristas y estados tipados para crear sistemas autónomos con ciclos de razonamiento.

Arquitectura de grafos: nodos, aristas y estado en LangGraph

Bienvenido al Módulo 5. Hasta ahora, has construido cadenas lineales con LCEL y has implementado arquitecturas RAG para dotar a tus modelos de una base de conocimientos. Sin embargo, en el mundo real, los problemas rara vez son lineales. Los sistemas de IA más avanzados no solo "responden", sino que **razonan, iteran y corrigen sus propios errores**.

Imagina un sistema de soporte técnico. No es una simple secuencia de pasos; es un flujo donde, dependiendo de la complejidad, el caso puede resolverse, pedir más información al usuario o escalar a un humano. Intentar programar esto con cadenas `if/else` interminables o loops de chat básicos resulta en un código frágil, difícil de depurar y casi imposible de escalar.

Aquí es donde entra **LangGraph**. En esta unidad, aprenderás a pensar en términos de **Grafos de Estado**, una evolución necesaria para construir agentes autónomos con principios de ingeniería de software.

1. ¿Por qué LangGraph? El paso de Cadenas a Grafos

En los módulos anteriores, utilizamos LangChain para crear secuencias de pasos (cadenas). Estas cadenas son excelentes para flujos de trabajo predecibles: *Input -> Prompt -> LLM -> Output*.

Sin embargo, los **agentes autónomos** requieren ciclos. Un agente necesita observar el resultado de una herramienta y decidir: "¿He terminado?" o "¿Debo intentar otra cosa?". Las cadenas de LangChain (DAGs por naturaleza en su versión básica) dificultan la implementación de estos ciclos de forma limpia.

LangGraph extiende LangChain permitiendo la creación de grafos que pueden contener ciclos. Pero lo más importante no es el ciclo en sí, sino la **gestión del estado**. En un sistema complejo, necesitamos una "memoria de trabajo" que persista mientras el agente navega por diferentes procesos.

2. El Estado: La Memoria Compartida del Grafo

El concepto más fundamental en LangGraph es el **Estado (State)**. Si visualizamos el grafo como una línea de ensamblaje en una fábrica, el Estado es el producto que se mueve por la cinta transportadora. Cada estación (nodo) puede leer el producto, modificarlo o añadirle piezas antes de pasarlo a la siguiente.

Definición mediante TypedDict

En Python 3.12+, definimos el estado utilizando `TypedDict`. Esto nos permite tener un esquema claro y tipado de qué datos están "vivos" en nuestra aplicación.

```
from typing import Annotated, TypedDict, List
from langchain_core.messages import BaseMessage

# Definimos el esquema de nuestra "memoria de trabajo"
class AgentState(TypedDict):
    # El historial de mensajes es el estado más común
    # Usamos Annotated y un "reducer" para decidir cómo se actualiza este campo
    messages: Annotated[List[BaseMessage], add_messages]
    # Podemos añadir campos personalizados para lógica de negocio
    is_escalated: bool
    current_step: str
```

El concepto de Reducers

Un error común es pensar que un nodo sobrescribe todo el estado. En realidad, los nodos devuelven **actualizaciones**. Si el estado tiene 10 campos y un nodo devuelve un diccionario con 1 campo, solo ese campo se actualiza.

¿Qué pasa si queremos que una lista de mensajes crezca en lugar de sobrescribirse? Usamos un **Reducer**. En el ejemplo de arriba, `add_messages` es una función que le dice a LangGraph: "No borres los mensajes anteriores, anexa los nuevos a la lista".

3. Nodos: Las Unidades de Trabajo

Los **Nodos** son simplemente funciones de Python (síncronas o `async`) que toman el estado actual como entrada y devuelven una actualización del estado.

Un nodo no tiene por qué ser un LLM. Puede ser:

1. **Un llamado a un LLM**: Procesa el historial de mensajes y decide qué hacer.
2. **Una herramienta (Tool)**: Realiza una consulta a una base de datos o API.

3. Lógica de negocio: Valida un JSON, formatea un texto o cambia una bandera en el estado.

Anatomía de un Nodo

```
async def call_model(state: AgentState):  
    # 1. Recuperamos información del estado  
    messages = state['messages']  
  
    # 2. Realizamos la operación (ej: llamado al LLM)  
    response = await model.ainvoke(messages)  
  
    # 3. Devolvemos la actualización  
    # LangGraph se encarga de fusionar esto con el estado global  
    return {"messages": [response]}
```

Principio de diseño: Mantén los nodos pequeños y con una única responsabilidad. Esto facilita las pruebas unitarias y la observabilidad. En LangGraph, un nodo es una unidad de software testeable, no una "caja negra mágica".

4. Aristas: El Control de Flujo

Si los nodos son las estaciones de trabajo, las **Aristas (Edges)** son las reglas que definen hacia dónde se mueve el producto después de salir de una estación.

Existen dos tipos principales de aristas:

A. Aristas Directas (Normal Edges)

Simplemente conectan el Nodo A con el Nodo B. "Siempre que termines de procesar en A, ve a B". Ejemplo:

```
workflow.add_edge("limpiar_texto", "analizar_sentimiento") .
```

B. Aristas Condicionales (Conditional Edges)

Permiten que el flujo se bifurque basándose en la lógica. Se apoyan en una **función de enrutamiento**.

```
def route_after_analysis(state: AgentState):  
    # Si el sentimiento es negativo, escalamos a un humano  
    if state["sentiment"] == "negative":  
        return "human_support"  
    # Si no, terminamos  
    return "end"  
  
# En la definición del grafo:
```



```
workflow.add_conditional_edges(  
    "analizar_sentimiento",  
    route_after_analysis,  
    {  
        "human_support": "nodo_humano",  
        "end": END  
    }  
)
```

Aquí es donde ocurre la "magia" de los agentes autónomos. Una arista condicional puede devolver el flujo a un nodo anterior, creando un **ciclo de reflexión**. Por ejemplo, un nodo "Validador" podría decidir que la respuesta del LLM no es lo suficientemente buena y enviarla de vuelta al nodo "Escritor" para una corrección.

5. El Flujo de Trabajo (Compiled Graph)

Una vez definidos los nodos y las aristas, el grafo debe ser compilado. Al compilarlo, LangGraph valida que:

1. Haya un punto de entrada (`START`).
2. Todos los caminos tengan una salida posible o lleguen al nodo especial `END` .
3. No existan nodos aislados.

Metáfora del "Libro de Elige tu propia aventura"

Imagina que cada nodo es una página del libro.

- El **Estado (State)** es lo que el lector recuerda (su inventario, su salud).
- Los **Nodos** son el contenido de la página que el lector lee.
- Las **Aristas** son las instrucciones al final de la página: "Si decides abrir la puerta, ve a la página 45; si decides correr, ve a la página 12".
- Las **Aristas Condicionales** son instrucciones basadas en el inventario: "Si tienes la llave dorada (estado), ve a la página 100; si no, ve a la página 20".

6. Errores Comunes y Mejores Prácticas

Error 1: Cargar el Estado con datos innecesarios

Un error frecuente es intentar meter objetos pesados (como clientes de bases de datos o conexiones de red) dentro del objeto `State` . **Mejor práctica:** El estado debe contener solo datos que necesiten persistir o ser compartidos entre nodos. Los objetos de infraestructura deben pasarse a través de la configuración o ser parte de la clase que envuelve al grafo.

Error 2: Nodos demasiado grandes

Si un nodo hace un llamado al LLM, procesa el resultado, limpia los datos y guarda en la base de datos, estás perdiendo el beneficio de la granularidad de los grafos. **Mejor práctica:** Divide las operaciones. Un nodo para el LLM y un nodo para el post-procesamiento. Esto permite insertar aristas condicionales en medio (ej. "¿Logró el LLM extraer los datos correctamente o necesitamos reintentar?").

Error 3: Olvidar el Reducer en listas de mensajes

Si defines `messages: List[BaseMessage]` en tu `TypedDict` sin un reducer, cada vez que un nodo devuelva `{"messages": [new_msg]}`, la lista anterior desaparecerá. **Mejor práctica:** Usa siempre `Annotated[List[BaseMessage], add_messages]` para flujos de chat convencionales.

7. Ejemplo resuelto paso a paso: un grafo de soporte con ciclo y escalado

El entregable de esta unidad te pide *diseñar el diagrama*. Acá ves su **contraparte en código, completa y ejecutable**: un grafo mínimo que busca una solución, reintentará si no la encuentra (ciclo) y escala a un humano tras 3 intentos. Corre sin API keys, porque simulamos el nodo de búsqueda con lógica simple para que puedas ver la mecánica del grafo.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. El Estado: la "memoria de trabajo" que fluye por el grafo
class SupportState(TypedDict):
    query: str
    attempts: int
    solved: bool
    escalated: bool

# 2. Nodo: busca en la base de conocimiento (simulado)
def search_knowledge(state: SupportState) -> dict:
    intento = state["attempts"] + 1
    encontrado = intento >= 2          # simulamos que recién resuelve en el 2° intento
    print(f"[search] intento {intento} -> {'resuelto' if encontrado else 'sin resultado'}")
    return {"attempts": intento, "solved": encontrado}

# 3. Nodo: escala a un humano
def human_support(state: SupportState) -> dict:
    print("[human] caso escalado a soporte humano")
    return {"escalated": True}

# 4. Arista condicional: decide el camino según el estado
def route(state: SupportState) -> str:
```

```

    if state["solved"]:
        return "end"
    if state["attempts"] >= 3:          # corta el ciclo para no quedar infinito
        return "escalate"
    return "retry"                    # vuelve a 'search' -> esto crea el CICLO

# 5. Construcción del grafo
workflow = StateGraph(SupportState)
workflow.add_node("search", search_knowledge)
workflow.add_node("human", human_support)

workflow.add_edge(START, "search")
workflow.add_conditional_edges(
    "search",
    route,
    {"end": END, "escalate": "human", "retry": "search"},
)
workflow.add_edge("human", END)

app = workflow.compile()

# 6. Ejecución
final = app.invoke(
    {"query": "No puedo iniciar sesión", "attempts": 0, "solved": False, "escalated": False}
)
print(final)

```

Salida esperada:

[search] intento 1 -> sin resultado [search] intento 2 -> resuelto {'query': 'No puedo iniciar sesión', 'attempts': 2, 'solved': True, 'escalated': False}

Qué pasó, paso a paso:

- 1. El Estado (`SupportState`)** arranca con `attempts=0` . Cada nodo devuelve solo los campos que cambia (una *actualización*), no el estado completo; LangGraph se encarga de fusionarlo.
- 2. `START` → `search`** : el primer intento no encuentra solución (`solved=False`).
- 3. La arista condicional `route`** lee el estado: como no está resuelto y aún no llegó a 3 intentos, devuelve `"retry"` , que vuelve al nodo `search` . **Ese retorno a un nodo anterior es el ciclo** que diferencia a LangGraph de una cadena lineal.
- 4.** En el segundo intento `solved=True` , así que `route` devuelve `"end"` y el grafo termina sin escalar.
- 5.** Si la búsqueda nunca resolviera, al llegar a `attempts >= 3` la ruta sería `"escalate"` → nodo `human` → `escalated=True` → END. Probalo cambiando la condición `encontrado` a `False` para ver ese camino.

Cuando armes el diagrama del entregable, este código es el "esqueleto" que estás representando: nodos = cajas, `add_conditional_edges` = la bifurcación, y el `"retry"` = la flecha que vuelve hacia atrás.

Segundo ejemplo resuelto: bucle de reflexión (escritor ↔ crítico)

El primer ejemplo usó un ciclo de reintento. Este muestra el otro patrón cíclico clásico de los agentes: el **bucle de reflexión**, donde un nodo "escritor" genera y un nodo "crítico" decide si está lo bastante bueno o si hay que revisar otra vez. Corre sin API keys (simulamos que la calidad mejora en cada vuelta).

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. Estado: el borrador, su puntaje de calidad y cuántas veces lo revisamos
class WriteState(TypedDict):
    tema: str
    borrador: str
    score: int
    revisiones: int

# 2. Nodo escritor: genera/mejora el borrador (simulado: suma 3 puntos por vuelta)
def escribir(state: WriteState) -> dict:
    nuevo_score = state["score"] + 3
    print(f"[escritor] revisión {state['revisiones'] + 1} -> score {nuevo_score}")
    return {
        "borrador": f"Borrador sobre '{state['tema']}' (calidad {nuevo_score}/10)",
        "score": nuevo_score,
        "revisiones": state["revisiones"] + 1,
    }

# 3. Nodo crítico: no modifica el estado; la decisión la toma la arista condicional
def criticar(state: WriteState) -> dict:
    return {}

# 4. Arista condicional: ¿aprobamos, nos rendimos o seguimos revisando?
def route(state: WriteState) -> str:
    if state["score"] >= 8:
        return "aprobado"
    if state["revisiones"] >= 4:          # tope para no iterar infinito
        return "rendirse"
    return "revisar"                    # vuelve al escritor -> CICLO de reflexión

# 5. Construcción del grafo
g = StateGraph(WriteState)
g.add_node("escritor", escribir)
```

```

g.add_node("critico", criticar)

g.add_edge(START, "escritor")
g.add_edge("escritor", "critico")
g.add_conditional_edges(
    "critico",
    route,
    {"aprobado": END, "rendirse": END, "revisar": "escritor"},
)

app = g.compile()

# 6. Ejecución
final = app.invoke({"tema": "onboarding de usuarios", "borrador": "", "score": 0, "revisiones":
0})
print(f"Final tras {final['revisiones']} revisiones | score {final['score']}")
print(final["borrador"])

```

Salida esperada:

[escritor] revisión 1 -> score 3 [escritor] revisión 2 -> score 6 [escritor] revisión 3 -> score 9 Final tras 3 revisiones | score 9 Borrador sobre 'onboarding de usuarios' (calidad 9/10)

Qué pasó, paso a paso:

1. El flujo es `escritor → crítico → (decisión)`. El crítico no cambia el estado; solo es el punto donde la arista condicional evalúa.
2. Tras cada vuelta, `route` mira el `score`: si llega a 8 termina (`aprobado`); si no, vuelve al escritor (`revisar`).
Ese retorno es el ciclo de reflexión.
3. La condición `revisiones >= 4` es un **freno de seguridad**: sin él, un borrador que nunca llega a 8 generaría un loop infinito.
4. En 3 vueltas el score pasa 3 → 6 → 9 y se aprueba.

A diferencia del primer ejemplo (ciclo de reintento ante un fallo), este es un ciclo de **mejora iterativa**: el sistema se autocritica hasta alcanzar un umbral de calidad. Es la base de los agentes "reflexivos".

8. Síntesis y Conexiones

En esta unidad hemos sentado las bases arquitectónicas de los sistemas de IA del futuro. En lugar de scripts lineales, ahora diseñamos **sistemas dinámicos**.

Puntos clave:

- **Estado:** La única fuente de verdad tipada que fluye por el sistema.
- **Nodos:** Funciones aisladas que transforman el estado.
- **Aristas:** Reglas de navegación que permiten ciclos y bifurcaciones.

En la siguiente unidad, daremos vida a estos nodos integrando **Tool Calling**, permitiendo que el estado no solo guarde texto, sino también los resultados de acciones ejecutadas en el mundo real. ¡Prepárate para conectar tu grafo con herramientas externas!

Práctica

Propósito del ejercicio

Practicar el pensamiento arquitectónico necesario para diseñar agentes autónomos. Antes de escribir código, un arquitecto de IA debe ser capaz de visualizar cómo fluye la información y bajo qué condiciones el sistema debe iterar.

Escenario

Vas a diseñar un **Asistente de Soporte Técnico Multinivel**. El sistema recibe una duda del usuario, intenta resolverla buscando en una base de conocimientos, valida si la solución es útil y, si no lo es o es muy compleja, escala el caso a un agente humano.

Tarea

- Definición del Estado (State):** Escribe (en el mismo diagrama o en un cuadro de texto adjunto) la estructura del `TypedDict` que usará tu grafo. Debe incluir al menos: * `messages` : Para el historial de conversación. * `is_escalated` : Un booleano para marcar si el caso pasó a un humano. * `attempts` : Un contador de cuántas veces el agente ha intentado resolver el problema.
- Diseño del Grafo:** Crea un diagrama que contenga: * **Nodo START:** El punto de entrada. * **Nodo "Triage":** Clasifica la duda del usuario. * **Nodo "Search_Knowledge":** Busca soluciones (Simula un RAG). * **Nodo "Evaluator":** Revisa si la solución encontrada responde a la duda. * **Arista Condicional:** Desde "Evaluator" hacia "END" (si la solución es buena) o hacia "Human_Support" (si ha fallado 3 veces) o de vuelta a "Search_Knowledge" (para intentar otra búsqueda). * **Nodo "Human_Support":** Marca `is_escalated=True`. * **Nodo END:** Cierre del flujo.

Criterios de Aceptación

- El diagrama utiliza una notación clara (nodos como cajas, aristas como flechas).
- Se incluye explícitamente el nodo `START` y el nodo `END`.
- Existe al menos un **ciclo** (el flujo vuelve a un nodo anterior).
- Se especifica la lógica de la **arista condicional** (bajo qué condición se toma cada camino).

- El esquema del Estado (State) está correctamente definido con tipos de datos de Python.

Herramientas sugeridas

- [Mermaid.js Live Editor](#) (Recomendado para desarrolladores).
- Excalidraw.
- Lucidchart / draw.io.

Errores comunes a evitar

- **Grafos lineales:** Si tu diagrama es una línea recta sin bifurcaciones ni ciclos, no estás aprovechando la potencia de LangGraph.
- **Olvidar la salida:** Asegúrate de que todos los caminos tengan una ruta hacia el nodo **END** para evitar loops infinitos.

Entregable

Tipo: DIAGRAM

Un diagrama que ilustre la arquitectura del grafo de estado para el sistema de soporte técnico. El diagrama debe incluir los nodos, las aristas (incluyendo al menos una condicional), la lógica del ciclo de reintento/revisión y una descripción del esquema del Estado (State). Puede entregarse como un archivo de imagen (PNG) o PDF generado con herramientas como Mermaid.js, Excalidraw, draw.io o similares.

Arquitectura de Grafos: Nodos, Aristas y Estado en LangGraph

Aprende a estructurar flujos de IA avanzados utilizando LangGraph, dominando los pilares de nodos, aristas y estados tipados para crear sistemas autónomos con ciclos de razonamiento.

Entregable

Tipo: DIAGRAM

Un diagrama que ilustre la arquitectura del grafo de estado para el sistema de soporte técnico. El diagrama debe incluir los nodos, las aristas (incluyendo al menos una condicional), la lógica del ciclo de reintento/revisión y una descripción del esquema del Estado (State). Puede entregarse como un archivo de imagen (PNG) o PDF generado con herramientas como Mermaid.js, Excalidraw, draw.io o similares.

Instrucciones

Propósito del ejercicio

Practicar el pensamiento arquitectónico necesario para diseñar agentes autónomos. Antes de escribir código, un arquitecto de IA debe ser capaz de visualizar cómo fluye la información y bajo qué condiciones el sistema

debe iterar.

Escenario

Vas a diseñar un **Asistente de Soporte Técnico Multinivel**. El sistema recibe una duda del usuario, intenta resolverla buscando en una base de conocimientos, valida si la solución es útil y, si no lo es o es muy compleja, escala el caso a un agente humano.

Tarea

- 1. Definición del Estado (State):** Escribe (en el mismo diagrama o en un cuadro de texto adjunto) la estructura del `TypedDict` que usará tu grafo. Debe incluir al menos: * `messages` : Para el historial de conversación. * `is_escalated` : Un booleano para marcar si el caso pasó a un humano. * `attempts` : Un contador de cuántas veces el agente ha intentado resolver el problema.
- 2. Diseño del Grafo:** Crea un diagrama que contenga: * **Nodo START:** El punto de entrada. * **Nodo "Triage":** Clasifica la duda del usuario. * **Nodo "Search_Knowledge":** Busca soluciones (Simula un RAG). * **Nodo "Evaluator":** Revisa si la solución encontrada responde a la duda. * **Arista Condicional:** Desde "Evaluator" hacia "END" (si la solución es buena) o hacia "Human_Support" (si ha fallado 3 veces) o de vuelta a "Search_Knowledge" (para intentar otra búsqueda). * **Nodo "Human_Support":** Marca `is_escalated=True`. * **Nodo END:** Cierre del flujo.

Criterios de Aceptación

- El diagrama utiliza una notación clara (nodos como cajas, aristas como flechas).
- Se incluye explícitamente el nodo `START` y el nodo `END`.
- Existe al menos un **ciclo** (el flujo vuelve a un nodo anterior).
- Se especifica la lógica de la **arista condicional** (bajo qué condición se toma cada camino).
- El esquema del Estado (State) está correctamente definido con tipos de datos de Python.

Herramientas sugeridas

- [Mermaid.js Live Editor](#) (Recomendado para desarrolladores).
- Excalidraw.
- Lucidchart / draw.io.

Errores comunes a evitar

- **Grafos lineales:** Si tu diagrama es una línea recta sin bifurcaciones ni ciclos, no estás aprovechando la potencia de LangGraph.
- **Olvidar la salida:** Asegúrate de que todos los caminos tengan una ruta hacia el nodo `END` para evitar loops infinitos.

Objetivos de aprendizaje

- Estructurar flujos de trabajo de IA utilizando grafos dirigidos en LangGraph.
- Definir estados tipados utilizando TypedDict y comprender el funcionamiento de los reducers para la gestión de memoria compartida.
- Diseñar nodos modulares que operen como unidades de trabajo independientes.
- Implementar lógica de control de flujo con aristas directas y condicionales para habilitar ciclos de razonamiento.

Rúbrica de Evaluación: Arquitectura de Grafos con LangGraph

Esta rúbrica evalúa la capacidad del estudiante para diseñar arquitecturas de agentes autónomos utilizando grafos de estado. Se centra en la definición de estados complejos, la lógica de control de flujo mediante aristas condicionales y la implementación de ciclos de razonamiento para la resolución de problemas técnicos.

Criterio	Descripción	Peso
Definición del Estado (TypedDict)	Evaluación de la estructura de datos que persiste a través del grafo, asegurando que contenga las variables necesarias para el control del flujo.	25%
Lógica de Diseño del Grafo y Nodos	Capacidad para descomponer un problema complejo en unidades de trabajo lógicas (nodos) y definir un flujo de inicio a fin.	35%
Implementación de Aristas Condicionales y Ciclos	Evaluación de la capacidad del sistema para tomar decisiones dinámicas y ejecutar procesos iterativos basados en el estado.	30%
Claridad de Notación y Documentación	Uso de herramientas visuales y claridad en la representación técnica del sistema.	10%

Total: 100 pts | Aprobación: 65 pts

• Código

Tool Calling: Definición y Ejecución de Herramientas con LangChain

Aprende a definir, validar y ejecutar funciones externas (tools) que permiten a los LLMs interactuar con bases de datos y APIs externas de forma segura utilizando LangChain y Pydantic.

Tool Calling: Definición y Ejecución Segura

En el desarrollo de sistemas de IA, un LLM "desnudo" es como un cerebro brillante en una habitación sin ventanas: puede razonar, pero no puede interactuar con el mundo exterior. El **Tool Calling** (Invocación de

Herramientas) es el mecanismo que le da "manos" a ese cerebro.

¿Qué es Tool Calling?

No es que el modelo ejecute código Python directamente. En su lugar, el modelo **genera una solicitud estructurada** (usualmente JSON) indicando qué función quiere ejecutar y con qué parámetros. Tu aplicación intercepta esa solicitud, ejecuta el código real, y le devuelve el resultado al modelo.

El "Contrato" de la Herramienta

Para que un modelo como GPT-4 o Claude 3.5 use una herramienta con éxito, necesita un contrato claro. Este contrato tiene cuatro pilares:

1. **Nombre:** Identificador único (ej. `get_stock_price`).
2. **Descripción:** Explicación semántica. El LLM decide si usar la herramienta basándose en esto.
3. **Esquema de Entrada (Input Schema):** Definición estricta de variables (tipos, obligatoriedad).
4. **Esquema de Salida:** Qué esperar de vuelta.

Definición Segura con Pydantic y LangChain

En entornos escalables, no podemos confiar en que el LLM envíe siempre los datos correctos. Usamos **Pydantic** para validar los tipos antes de que el código toque nuestra infraestructura.

```
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class CalculatorInput(BaseModel):
    a: int = Field(description="Primer número")
    b: int = Field(description="Segundo día")

@tool(args_schema=CalculatorInput)
def multiply(a: int, b: int) -> int:
    """Multiplica dos números enteros."""
    return a * b
```

El Ciclo de Ejecución (Reason-Act Loop)

1. **Reasoning:** El LLM recibe el prompt y las definiciones de herramientas. Decide que necesita `multiply`.
2. **Tool Call:** El LLM responde con un objeto especial: `tool_calls: [{name: 'multiply', args: {a: 5, b: 2}}]`.
3. **Execution (Local):** Tu servidor detecta esta intención, valida los argumentos con Pydantic y ejecuta la función `multiply(5, 2)`.

4. **Observation:** El resultado (`10`) se envía de vuelta al LLM en un nuevo mensaje con el rol `tool` .

5. **Final Response:** El LLM procesa el `10` y responde al usuario: "El resultado es 10".

Errores Comunes y Seguridad

- **Confundir Invocación con Ejecución:** El LLM **no** ejecuta el código. Tu arquitectura debe garantizar que las herramientas tengan permisos limitados (Least Privilege). Nunca permitas que una herramienta ejecute SQL raw generado por el LLM sin validación.
- **Descripciones Ambiguas:** Si tu descripción es "Busca datos", el modelo fallará. Sé específico: "Busca precios de acciones en tiempo real para empresas del S&P 500".
- **Ignorar Errores:** Si la herramienta falla (ej. timeout de API), debes capturar el error y enviarlo como un string al LLM para que este pueda explicarle al usuario qué salió mal o intentar otra estrategia.

En la siguiente práctica, conectarás este concepto con el sistema RAG que construiste anteriormente, permitiendo que tu agente decida cuándo consultar la base de conocimientos.

Ejercicio de código

Código inicial

```
from typing import List
from pydantic import BaseModel, Field
from langchain_core.tools import tool

# TODO: Define una clase Pydantic llamada 'SearchInput'
# con los campos 'query' (str) y 'limit' (int).

# Simulación de base de datos vectorial
class MockVectorDB:
    async def similarity_search(self, query: str, k: int):
        return [f"Resultado para {query}"]

db = MockVectorDB()

# TODO: Implementa la herramienta 'search_knowledge_base'
# 1. Usa el decorador @tool y asocia el esquema de Pydantic.
# 2. Asegúrate de que la función sea asíncrona (async def).
# 3. Agrega un docstring descriptivo para que el LLM sepa qué hace la herramienta.

async def search_knowledge_base(query: str, limit: int = 3):
    """
    Descripción de la herramienta aquí.
```

```

"""
# Implementar llamada a db.similarity_search
pass

```

Código solución

```

from typing import List, Type
from pydantic import BaseModel, Field
from langchain_core.tools import tool

# 1. Definimos el esquema de entrada con Pydantic para asegurar tipos estrictos
class SearchInput(BaseModel):
    query: str = Field(description="La consulta en lenguaje natural para buscar en la base de conocimientos")
    limit: int = Field(default=3, description="Número máximo de documentos a recuperar")

# Simulamos la conexión con el módulo 3 (ChromaDB)
# En un entorno real, aquí importarías tu vectorstore
class MockVectorDB:
    async def similarity_search(self, query: str, k: int) -> List[str]:
        # Simulación de recuperación de datos
        return [f"Documento sobre {query} #" + str(i) for i in range(k)]

vector_db = MockVectorDB()

# 2. Implementación de la herramienta con decorador y validación
@tool(args_schema=SearchInput)
async def search_knowledge_base(query: str, limit: int = 3) -> List[str]:
    """
    Busca información técnica relevante en la base de conocimientos local de la empresa.
    Úsala cuando el usuario haga preguntas sobre procedimientos internos o documentación
    técnica.
    """
    try:
        # 3. Invocación asíncrona
        results = await vector_db.similarity_search(query, k=limit)
        return results
    except Exception as e:
        # 4. Manejo de errores: devolvemos un mensaje que el LLM pueda entender
        return [f"Error al acceder a la base de conocimientos: {str(e)}"]

# Ejemplo de uso (para validación interna):

```

```
# result = await search_knowledge_base.ainvoke({"query": "política de seguridad", "limit": 2})
```

Pistas

- Para definir una herramienta que LangChain reconozca automáticamente, utiliza el decorador `@tool` . No olvides incluir un 'docstring' detallado en la función, ya que LangGraph lo utiliza como la descripción que se le envía al LLM para que sepa cuándo usar la herramienta.
- Pydantic es excelente para la validación. Define una clase que herede de `BaseModel` para estructurar los argumentos de entrada. Luego, pasa esta clase al decorador: `@tool(args_schema=MiEsquema)` . Esto garantiza que el LLM no envíe basura a tu función.
- Como estamos trabajando con un sistema RAG, la función de búsqueda debe ser `async` . Esto permite que tu servidor (FastAPI) maneje múltiples peticiones mientras esperas la respuesta de la base de datos vectorial.

Rúbrica de Evaluación: Tool Calling y Definición Segura con LangChain

Evalúa la capacidad del estudiante para definir contratos de herramientas (tools) utilizando Pydantic y LangChain, garantizando la seguridad, la precisión semántica y la estructura necesaria para agentes autónomos.

Criterio	Descripción	Peso
Definición del Contrato con Pydantic	Evaluación del esquema de entrada (args_schema) para asegurar tipos de datos correctos y validación robusta.	35%
Calidad de la Descripción Semántica	Claridad y especificidad del docstring de la función, que actúa como instrucción para que el LLM decida cuándo usar la herramienta.	25%
Implementación del Decorador y Lógica de Función	Correcta aplicación del decorador @tool de LangChain y eficiencia del código Python interno.	20%
Manejo de Errores y Seguridad	Capacidad de la herramienta para gestionar datos inesperados o fallos de ejecución de forma segura.	20%

Total: 100 pts | Aprobación: 65 pts

Ciclos ReAct y Persistencia de Checkpoints en Agentes Autónomos

Implementa un agente autónomo con el ciclo ReAct y utiliza Checkpointers para persistir el estado de la ejecución y el historial de conversación.

Ciclos ReAct y Persistencia de Checkpoints en Agentes Autónomos

Como arquitecto de IA, tu objetivo no es solo que el sistema responda, sino que sea **resiliente**. En sistemas complejos, un agente puede fallar a mitad de una ejecución de 10 pasos debido a un error de red o un timeout. Sin persistencia, perderías todo el razonamiento y los resultados parciales (y el dinero gastado en tokens).

1. El Ciclo ReAct (Reasoning + Acting)

El patrón ReAct es el estándar de oro para agentes autónomos. A diferencia de una cadena lineal, ReAct permite al LLM:

1. **Pensar (Reason):** Analizar la solicitud del usuario y decidir qué necesita.
2. **Actuar (Act):** Seleccionar una herramienta y generar los parámetros de llamada.
3. **Observar (Observe):** Recibir la salida de la herramienta y re-evaluar el estado.

Este proceso es **cíclico**. El LLM puede decidir que necesita usar tres herramientas diferentes antes de dar una respuesta final. LangGraph facilita esto permitiendo que una arista (edge) vuelva al mismo nodo o a un nodo anterior, creando un historial de mensajes que crece con cada "observación".

2. Checkpoints: La Caja Negra de tu Agente

En LangGraph, un **Checkpoint** es un mecanismo de almacenamiento que guarda una "fotografía" del estado del grafo después de cada nodo ejecutado.

- **Para qué sirve:** Si el proceso se detiene, puedes reanudarlo simplemente pasando el mismo `thread_id`. El sistema buscará en la base de datos (SQLite, Redis o Memoria) y cargará el último estado conocido.
- **Time Travel:** Los checkpoints permiten no solo reanudar, sino "volver atrás" en el tiempo (debugging) o requerir aprobación humana antes de continuar con un nodo crítico (Human-in-the-loop).

Enunciado del Problema

Vas a construir un agente climatológico capaz de razonar si necesita herramientas y que no pierda la memoria incluso si interrumpimos la ejecución del script.

Objetivo:

1. Implementar un ciclo ReAct completo.
2. Configurar persistencia en memoria.

3. Demostrar que el agente recuerda la ubicación mencionada en el primer mensaje al recibir un segundo mensaje, gracias al `thread_id`.

Restricciones:

- Usa `Annotated[list[BaseMessage], add_messages]` en tu estado para asegurar que los mensajes no se sobrescriban, sino que se concatenen.
- El grafo debe terminar solo cuando el LLM decida que no hay más herramientas que llamar.
- No olvides que el nodo de herramientas (`ToolNode`) debe mapear sus resultados de vuelta al agente.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import asyncio
from typing import Annotated, TypedDict
from langchain_openai import ChatOpenAI
from langchain_core.messages import BaseMessage, HumanMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.memory import MemorySaver

# 1. Define el Estado (State) del agente
class AgentState(TypedDict):
    # TODO: Usa add_messages para que el historial sea acumulativo
    messages: Annotated[list[BaseMessage], add_messages]

# 2. Herramientas
def get_weather(location: str):
    """Consulta el clima actual."""
    return "Soleado, 22°C"

tools = [get_weather]
# TODO: Crea el ToolNode con la lista de herramientas
tool_node = None

# 3. LLM con Herramientas vinculadas
# TODO: Instancia ChatOpenAI y usa .bind_tools(tools)
model = None
```

```

# 4. Nodos
async def call_model(state: AgentState):
    # TODO: Invoca al modelo con los mensajes actuales
    # Retorna un diccionario con la nueva clave "messages"
    pass

# 5. Orquestación del Grafo
workflow = StateGraph(AgentState)

# TODO: Añade el nodo 'agent' y el nodo 'tools'
# TODO: Define la lógica de aristas:
# - START -> agent
# - agent -> conditional_edges (usa tools_condition)
# - tools -> agent

# 6. Persistencia
# TODO: Instancia MemorySaver()
memory = None

# TODO: Compila el grafo pasando el checkpointer
app = None

async def main():
    # Identificador de sesión para persistencia
    thread_config = {"configurable": {"thread_id": "dev_test_01"}}

    # Primera interacción
    print("--- Paso 1 ---")
    # app.ainvoke(...)

    # Segunda interacción (reanudando desde checkpoint)
    print("--- Paso 2 ---")
    # app.ainvoke(...)

if __name__ == "__main__":
    asyncio.run(main())

```

Código solución

```

import asyncio
from typing import Annotated, TypedDict, Union
from langchain_openai import ChatOpenAI
from langchain_core.messages import BaseMessage, HumanMessage

```



```

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.memory import MemorySaver

# 1. Definición del Estado del Grafo
class AgentState(TypedDict):
    # 'add_messages' permite que LangGraph gestione el historial automáticamente
    messages: Annotated[list[BaseMessage], add_messages]

# 2. Herramientas de ejemplo
def get_weather(location: str):
    """Consulta el clima actual en una ubicación dada."""
    if "madrid" in location.lower():
        return "Soleado, 25°C"
    return "Nublado, 15°C"

tools = [get_weather]
tool_node = ToolNode(tools)

# 3. Configuración del Modelo con Herramientas
model = ChatOpenAI(model="gpt-4o", temperature=0).bind_tools(tools)

# 4. Definición de Nodos
async def call_model(state: AgentState):
    """Nodo del agente que decide el siguiente paso."""
    response = await model.ainvoke(state["messages"])
    return {"messages": [response]}

# 5. Construcción del Grafo ReAct
workflow = StateGraph(AgentState)

# Añadimos los nodos
workflow.add_node("agent", call_model)
workflow.add_node("tools", tool_node)

# Definimos las aristas (edges)
workflow.add_edge(START, "agent")

# Arista condicional: ReAct decide si seguir usando herramientas o terminar
workflow.add_conditional_edges(
    "agent",
    tools_condition,
)

```

```

# El nodo de herramientas siempre vuelve al agente para procesar el resultado
workflow.add_edge("tools", "agent")

# 6. Configuración de la Persistencia (Checkpoint)
memory = MemorySaver()

# Compilación del grafo con el checkpoint
app = workflow.compile(checkpointer=memory)

async def main():
    # ID de hilo para persistir la conversación
    config = {"configurable": {"thread_id": "session_123"}}

    print("--- Iniciando ciclo ReAct ---")
    inputs = {"messages": [HumanMessage(content="¿Qué tiempo hace en Madrid?")]}

    async for event in app.astream(inputs, config=config, stream_mode="values"):
        message = event["messages"][-1]
        message.pretty_print()

    print("\n--- Simulando reanudación con el mismo thread_id ---")
    # Notar que no enviamos el historial anterior, LangGraph lo recupera del checkpoint
    resume_inputs = {"messages": [HumanMessage(content="¿Y en Londres?")]}

    async for event in app.astream(resume_inputs, config=config, stream_mode="values"):
        message = event["messages"][-1]
        message.pretty_print()

if __name__ == "__main__":
    asyncio.run(main())

```

Pistas

- Para que el agente decida si debe usar una herramienta o responder directamente, el nodo del 'agente' debe invocar al LLM y pasarle las definiciones de las herramientas mediante el método 'bind_tools'. Automáticamente, el LLM decidirá si el 'tool_calls' del mensaje de salida contiene datos o no.
- LangGraph requiere un objeto 'Checkpoint' para habilitar la persistencia. Para desarrollo local, 'MemorySaver' es ideal ya que guarda el hilo en memoria RAM. Recuerda que para que la persistencia funcione, al invocar el grafo (.invoke o .stream), debes pasar un diccionario de configuración que incluya un 'thread_id' único.
- El ciclo ReAct se cierra conectando el nodo de herramientas de vuelta al nodo del agente. Usa la función predefinida 'tools_condition' como una arista condicional: si hay llamadas a herramientas pendientes, irá al nodo de herramientas; si no, finalizará el grafo.

Evaluación del Agente ReAct con Persistencia en LangGraph

Evalúa la capacidad de construir un agente autónomo resiliente capaz de razonar mediante ciclos ReAct y mantener el estado de la conversación utilizando checkpoints de LangGraph.

Criterio	Descripción	Peso
Implementación del Ciclo ReAct	Capacidad del grafo para alternar entre el razonamiento del LLM, la ejecución de herramientas y la observación de resultados de forma cíclica.	35%
Gestión de Estado y Persistencia	Uso correcto de Annotated y checkpoints para garantizar que la memoria del agente persista entre diferentes sesiones de ejecución.	30%
Configuración del Grafo y Nodos	Definición técnica del grafo, incluyendo la creación de nodos de herramientas (ToolNode) y rutas condicionales.	20%
Prueba de Concepto y Razonamiento	Capacidad del código para demostrar que el agente recuerda información previa y toma decisiones climáticas lógicas.	15%

Total: 100 pts | Aprobación: 65 pts

● Pre-entrega

Pre-entrega 5: Agente de razonamiento cíclico con memoria persistente

Especificación de la pre-entrega para el diseño de un agente autónomo cíclico con LangGraph, integrando persistencia y ejecución de herramientas externas.

Pre-entrega 5: Agente de razonamiento cíclico con memoria persistente

Criterios de Aceptación

Para que este checkpoint se considere aprobado, el entregable debe cumplir con lo siguiente:

- Autonomía:** El agente debe ser capaz de determinar por sí mismo cuándo llamar a una herramienta basándose en el prompt del usuario (sin rutas manuales `if/else`).
- Ciclo de Retorno:** Si una herramienta devuelve un error o información incompleta, el agente debe ser capaz de realizar un segundo intento o pedir aclaraciones.
- Resiliencia de Estado:** Al proporcionar un `thread_id`, el agente debe ser capaz de recordar interacciones previas dentro de una misma sesión de razonamiento.
- Código Limpio:** Uso de Python 3.12, tipado estático (Type Hints) y gestión asíncrona (`asyncio`).

Guía de Implementación Sugerida

Fase 1: El Contrato de Herramientas

Define tus funciones utilizando el decorador `@tool` de LangChain. Asegúrate de incluir docstrings extremadamente descriptivos; recuerda que el LLM decide qué herramienta usar basándose únicamente en esa descripción.

Fase 2: Definición del Estado y el Grafo

Crea el esquema de tu estado. En LangGraph, el estado es inmutable y se actualiza mediante reducers (usualmente `operator.add` para la lista de mensajes). Configura el `StateGraph` conectando el nodo del modelo con el nodo de ejecución de herramientas mediante una arista condicional (`tools_condition`).

Fase 3: Persistencia

Configura un Checkpointer. Esto es lo que permite que el agente sea "arquitectónicamente escalable". Sin persistencia, tu agente es efímero; con ella, es capaz de manejar flujos de trabajo largos que requieren intervención humana o esperar procesos externos.

Errores Comunes a Evitar

- **Descripciones Vagaces:** Si el agente no usa la herramienta que esperas, el error suele estar en el docstring de la función, no en la lógica del grafo.
- **Bucles Infinitos:** No establecer un límite de recursión (`recursion_limit`) al invocar el grafo. Define siempre un techo (ej. 10 pasos) para evitar costos inesperados en la API.
- **Estado Sucio:** Olvidar que el estado se acumula. Asegúrate de limpiar o resumir mensajes si el contexto se vuelve demasiado grande.

Práctica

1. Inicializa un proyecto Python 3.12+ con Poetry o venv.
2. Define un `StateGraph` que herede de `MessagesState` .
3. Crea al menos una herramienta personalizada que simule una operación de base de datos o búsqueda técnica.
4. Configura un LLM (OpenAI o Anthropic) vinculado a las herramientas (`llm.bind_tools()`).
5. Implementa la lógica de persistencia usando `SqliteSaver` (para desarrollo local).
6. Realiza una prueba de ejecución donde el agente deba llamar a la herramienta al menos dos veces para llegar a la conclusión (razonamiento multi-paso).
7. Sube el código a un repositorio público y asegúrate de no incluir las API Keys (usa variables de entorno).

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga el código del agente en un entorno asíncrono, incluyendo la definición del grafo, las herramientas y la configuración del checkpointer. Debe incluir un archivo README explicando cómo levantar el entorno y un ejemplo de traza de ejecución (.json o log).

Pre-entrega: Agente de Razonamiento Cíclico con Memoria Persistente

Especificación de la pre-entrega para el diseño de un agente autónomo cíclico con LangGraph, integrando persistencia y ejecución de herramientas externas.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub que contenga el código del agente en un entorno asíncrono, incluyendo la definición del grafo, las herramientas y la configuración del checkpointer. Debe incluir un archivo README explicando cómo levantar el entorno y un ejemplo de traza de ejecución (.json o log).

Instrucciones

1. Inicializa un proyecto Python 3.12+ con Poetry o venv.
2. Define un `StateGraph` que herede de `MessagesState`.
3. Crea al menos una herramienta personalizada que simule una operación de base de datos o búsqueda técnica.
4. Configura un LLM (OpenAI o Anthropic) vinculado a las herramientas (`llm.bind_tools()`).
5. Implementa la lógica de persistencia usando `SqliteSaver` (para desarrollo local).
6. Realiza una prueba de ejecución donde el agente deba llamar a la herramienta al menos dos veces para llegar a la conclusión (razonamiento multi-paso).
7. Sube el código a un repositorio público y asegúrate de no incluir las API Keys (usa variables de entorno).

Objetivos de aprendizaje

- Diseñar e implementar un flujo de trabajo cíclico (ReAct) utilizando LangGraph.
 - Configurar la persistencia de estado mediante Checkpointers para garantizar la continuidad de la ejecución.
 - Definir herramientas técnicas robustas que los LLMs puedan invocar de forma autónoma mediante esquemas estructurados.
 - Gestionar estados complejos en sistemas de IA asíncronos siguiendo principios de ingeniería de software.
-

Rúbrica de Evaluación: Agente de Razonamiento Cíclico con Memoria Persistente

Esta rúbrica evalúa la capacidad del estudiante para diseñar e implementar un agente autónomo utilizando LangGraph, integrando ciclos de razonamiento ReAct, herramientas personalizadas y persistencia de estado para sistemas de IA escalables profesionales.

Criterio	Descripción	Peso
Arquitectura del Grafo y Gestión de Estado	Evaluación de la correcta implementación del StateGraph y el uso de MessagesState para mantener la coherencia del diálogo.	30%
Integración de Herramientas y Razonamiento Multi-paso	Capacidad del agente para invocar herramientas externas y procesar información en múltiples iteraciones.	35%
Persistencia con SqliteSaver	Implementación de mecanismos de memoria para permitir la continuidad de las conversaciones.	20%
Calidad del Repositorio y Entorno Profesional	Uso de estándares de la industria en la organización del código, gestión de dependencias y seguridad de credenciales.	15%

Total: 100 pts | Aprobación: 65 pts

• Clase en vivo

Sesión en vivo 5. Razonamiento autónomo: introducción de agentes con LangGraph

Sesión interactiva para consolidar el diseño y ejecución de agentes autónomos utilizando grafos de estado en LangGraph, enfocada en robustez y toma de decisiones cíclica.

Sesión en Vivo: Dominando el Razonamiento Autónomo con LangGraph

¡Bienvenidos a la sesión de cierre del módulo! En esta clase de dos horas, transformaremos el conocimiento teórico y el código base que han desarrollado en una capacidad estratégica para diseñar sistemas de IA que no solo "responden", sino que **razonan y actúan** de forma autónoma y controlada.

Pasaremos de ver el código de forma aislada a entender la arquitectura de agentes como un sistema vivo de toma de decisiones.

¿Qué aprenderás hoy?

- A conectar conceptos de **grafos de estado** con flujos de trabajo de nivel empresarial.
- Estrategias para diseñar **ciclos de razonamiento (ReAct)** que se recuperan de errores.
- Cómo estructurar el **Estado (State)** para que tu agente tenga "memoria de trabajo" efectiva.

- Patrones de diseño para integrar herramientas externas de forma segura.

Temas clave

1. **El salto al Agente:** Diferencia real entre una cadena lineal y un ciclo de razonamiento.
2. **Diseño de Grafos:** Nodos de decisión vs. Nodos de ejecución.
3. **Gestión de Errores:** Cómo evitar que un agente entre en bucles infinitos.
4. **Persistencia y Memoria:** El uso de checkpoints para la continuidad del razonamiento.

Preparación previa

Para aprovechar al máximo esta sesión, asegúrate de:

1. Haber completado la **Pre-entrega** del agente de razonamiento cíclico.
2. Tener a mano el diagrama de flujo de tu agente actual.
3. Listar al menos dos problemas que encontraste durante el desarrollo de tus unidades anteriores (ej: "mi agente repite la búsqueda tres veces").

Beneficios

Al finalizar, tendrás una visión arquitectónica clara para decidir cuándo un proyecto requiere la complejidad de LangGraph y cómo implementar agentes que sean robustos, predecibles y escalables.

Clase en vivo

Materiales de apoyo

Puntos clave:

- **Estado (State):** Diccionario compartido que los nodos leen y escriben. Es la memoria de corto plazo del agente.
- **Nodos:** Funciones de Python que realizan una acción y actualizan el estado.
- **Aristas Condicionales:** La lógica que decide qué nodo se ejecuta a continuación basada en el estado actual.
- **Ciclo ReAct:** Reasoning + Acting. Un patrón donde el LLM piensa qué herramienta usar, la ejecuta y observa el resultado antes de concluir.

Ejemplos sugeridos:

- **Agente de Soporte:** Clasifica el ticket -> Consulta DB -> Si no hay solución, busca en documentación externa -> Responde al cliente.

- **Traductor con Crítico:** Traduce texto -> Un nodo 'crítico' evalúa la gramática -> Si hay errores, vuelve a traducir con sugerencias.

Herramientas recomendadas:

- LangGraph Studio (para visualizar grafos en tiempo real)
- Tavily Search API (como herramienta de búsqueda estándar para agentes)
- Python 3.10+ con LangChain-Community

Actividades

- **Del Prompt Único al Grafo de Estado** (20 min, DISCUSSION)
 - **Setup:** Presentar un caso de uso (ej: Investigador de Mercado). El instructor muestra un prompt largo que intenta hacer todo a la vez (zero-shot). **Ejecución:** Los estudiantes proponen cómo dividir esa tarea única en 3 nodos y una arista condicional. **Dinámica:** Votación en tiempo real sobre qué nodo es el más crítico para el éxito de la tarea. **Cierre:** Reflexión sobre por qué el control explícito supera al prompt único.
- **Depuración de Bucles Infinitos** (25 min, EXERCISE)
 - **Setup:** Presentar un bloque de código o diagrama que contiene un error de diseño (ej: un agente de búsqueda que nunca llega al nodo 'END' porque la condición de salida siempre es falsa). **Ejecución:** Los estudiantes deben analizar el flujo e identificar el 'loop de la muerte'. **Dinámica:** Chat abierto para proponer la solución (¿cambiar el prompt? ¿añadir un contador de pasos?). **Cierre:** El instructor muestra cómo un contador de pasos en el 'state' de LangGraph soluciona el problema.
- **Role-play: El Cerebro del Agente** (25 min, GROUP_WORK)
 - **Setup:** Dividir a la clase en grupos de 3 (Planner, Executor, Reviewer). Caso: Agente de soporte técnico. **Ejecución:** Se da una entrada compleja. El Planner dice qué herramienta usar, el Executor simula el resultado, y el Reviewer decide si se termina o se vuelve a empezar. **Dinámica:** Interacción verbal rápida simulando la latencia del agente. **Cierre:** Debatir cómo la separación de responsabilidades facilita la depuración en LangGraph.

Tiempos

Sección	Minutos
Introducción	10
Contenido principal	25
Actividades	70
Preguntas y respuestas	10
Cierre	5
Total	120

• Podcast

Recapitulación: El camino hacia los agentes autónomos con LangGraph

Un cierre conversacional profundo que integra todos los conceptos del módulo, desde la arquitectura de grafos hasta la persistencia de agentes autónomos.

Este episodio final es una inmersión completa en el ecosistema de **LangGraph**. Acompañanos en esta recapitulación donde conectamos todos los puntos: desde la arquitectura base de nodos y aristas hasta la implementación de ciclos ReAct y sistemas de persistencia.

Al escucharlo, podrás consolidar tu comprensión sobre cómo los agentes no son solo "prompts inteligentes", sino sistemas estructurados que gestionan estados, ejecutan herramientas y aprenden de sus propios procesos cíclicos. Es el cierre perfecto para integrar la teoría y la práctica de este módulo.

Lo que aprenderás:

- Cómo se integra la arquitectura de grafos en el razonamiento de la IA.
 - La importancia del **Tool Calling** como puente hacia el mundo exterior.
 - El funcionamiento del patrón **ReAct** y por qué la persistencia es vital para agentes de producción.
 - Una visión holística sobre el diseño de sistemas de agentes autónomos.
-

Sistemas multi-agente: colaboración y especialización

Diseñar sistemas complejos donde múltiples agentes autónomos colaboran y se especializan para resolver tareas de alto nivel.

• **Lectura**

Topologías Multi-Agente: Colaboración vs Jerarquía

Explora las estructuras fundamentales de organización en sistemas multi-agente, analizando cuándo centralizar el control mediante jerarquías o descentralizarlo mediante colaboración para maximizar la eficiencia y modularidad.

Topologías Multi-Agente: Colaboración vs Jerarquía

Imagina que estás diseñando un sistema de software para gestionar un hospital inteligente. Tienes diferentes necesidades: diagnosticar enfermedades raras, gestionar el inventario de medicamentos, optimizar las rondas de las enfermeras y responder a emergencias en tiempo real. ¿Deberías crear un único programa "todopoderoso" que intente resolverlo todo? ¿O sería mejor tener especialistas que hablen entre sí?

Si optas por especialistas, acabas de entrar en el mundo de los **Sistemas Multi-Agente (MAS)**. Pero aquí surge el verdadero reto de ingeniería: ¿cómo deben relacionarse estos agentes? ¿Necesitan un "director de orquesta" que les asigne tareas (Jerarquía), o pueden negociar entre ellos como un equipo de expertos en una sala de juntas (Colaboración)?

En esta unidad, exploraremos las topologías que definen el flujo de control y comunicación en arquitecturas de IA escalables, sentando las bases para construir sistemas que no solo "funcionen", sino que sean mantenibles y resilientes.

1. El Salto de la Cadena al Sistema

Hasta ahora, en los módulos anteriores, hemos trabajado principalmente con **cadena lineales** o **ciclos ReAct dirigidos**. Un usuario pregunta, un agente razona, usa una herramienta y responde.

Sin embargo, cuando el problema es demasiado grande para un solo prompt o un solo bucle de razonamiento (por ejemplo, escribir un libro entero, auditar un sistema financiero o simular una cadena de suministro), la complejidad cognitiva desborda a un único agente. Aquí es donde la **topología** se vuelve crítica.

La **topología multi-agente** es la estructura de red que define quién tiene la autoridad, cómo se distribuye el estado y qué canales de comunicación existen entre los agentes. No es solo un diagrama; determina cómo

escalará tu sistema bajo carga y qué tan fácil será depurarlo.

2. Topología Jerárquica: El Modelo del Supervisor

La arquitectura jerárquica es, quizás, la más intuitiva para un desarrollador de software. Se basa en una estructura de mando clara donde un agente de nivel superior (Supervisor o Coordinador) gestiona a un grupo de agentes especialistas (Workers).

¿Cómo funciona?

- 1. Delegación:** El usuario envía una petición al Supervisor.
- 2. Descomposición:** El Supervisor analiza la tarea y la divide en subtareas más pequeñas.
- 3. Asignación:** El Supervisor "llama" a los agentes específicos (ej. Agente de Búsqueda, Agente de Redacción, Agente de Validación).
- 4. Agregación:** Los especialistas devuelven sus resultados al Supervisor, quien sintetiza la respuesta final.

Ejemplo Real: Agencia de Viajes Autónoma

Imagina un sistema para planificar viajes complejos:

- Agente Supervisor (El Planner):** Recibe "Quiero un viaje a Japón de 10 días". Decide que necesita un vuelo, hoteles y un itinerario.
- Agente Especialista A (SkyScanner Expert):** Solo sabe buscar vuelos y manejar fechas.
- Agente Especialista B (Booking Specialist):** Solo sabe de alojamientos y disponibilidad.
- Agente Especialista C (Cultural Guide):** Genera descripciones de lugares basadas en intereses.

El Planner coordina. Si el Agente de Vuelos falla, el Planner decide si intenta de nuevo o busca una alternativa, sin que el Agente de Hoteles se entere siquiera del problema.

Ventajas y Desventajas

Ventaja	Explicación
Control Centralizado	Es fácil predecir el flujo y auditar las decisiones en un solo punto (el supervisor).
Modularidad	Puedes cambiar el Agente de Vuelos por uno mejor sin tocar el resto del sistema.
Reducción de Ruido	Los especialistas no reciben información irrelevante; solo su tarea específica.

El Gran Riesgo: El supervisor es un **cuello de botella**. Si el LLM que actúa como supervisor se confunde o falla, todo el sistema queda paralizado. Además, a medida que añades más especialistas, la "carga cognitiva" del supervisor aumenta, lo que puede requerir modelos más costosos (como GPT-4o o Claude 3.5 Sonnet) solo para la coordinación.

3. Topología Cooperativa: El Modelo de Colaboración (Peer-to-Peer)

En una topología cooperativa o descentralizada, no hay un jefe. Los agentes son "pares" (peers) que se comunican entre sí para resolver un problema. La coordinación surge de un **estado compartido** o de un protocolo de comunicación acordado.

¿Cómo funciona?

En lugar de esperar órdenes, los agentes observan el estado del sistema y actúan cuando ven que pueden aportar algo.

- **Pizarrón (Blackboard):** Un patrón común donde todos los agentes escriben en un espacio compartido. Si el "Agente Traductor" ve un texto en inglés en el pizarrón, lo traduce automáticamente al español sin que nadie se lo pida explícitamente.
- **Coreografía:** Cada agente sabe qué hacer cuando recibe un mensaje de otro, creando una reacción en cadena.

Ejemplo Real: Desarrollo de Software Colaborativo

Imagina tres agentes trabajando en un repositorio de código:

1. **Agente Arquitecto:** Escribe la especificación de una función en el estado compartido.
2. **Agente Programador:** Al detectar una nueva especificación, escribe el código.
3. **Agente Tester:** Al detectar código nuevo, escribe y ejecuta pruebas.
4. **Agente Programador (de nuevo):** Si el Tester publica errores, el Programador los corrige automáticamente.

Aquí no hay un supervisor diciendo "ahora tú, ahora tú". El flujo es reactivo y dinámico.

Ventajas y Desventajas

Ventaja	Explicación
Resiliencia	No hay un punto único de fallo. Si un agente se cae, otros pueden seguir trabajando en tareas paralelas.
Escalabilidad Orgánica	Es más fácil añadir agentes que "escuchen" eventos sin reconfigurar un orquestador central.
Adaptabilidad	Ideal para entornos donde el orden de las tareas no es predecible.

El Gran Riesgo: El **caos emergente**. Sin una supervisión estricta, los agentes pueden entrar en bucles infinitos (el programador cambia algo, el tester falla, el programador vuelve a cambiar, y así para siempre) o producir

resultados inconsistentes. La depuración (debugging) se vuelve significativamente más compleja al no haber un rastro lineal de ejecución.

4. Comparativa Técnica para el Arquitecto de IA

Como desarrollador Python, tu elección de topología suele depender de la **impureza del problema**.

¿Cuándo elegir Jerarquía?

- Cuando tienes un **workflow determinista** o semi-determinista.
- Cuando la seguridad es crítica y necesitas un "validador" final (el supervisor) que revise todo antes de mostrarlo al usuario.
- Cuando usas modelos de diferentes tamaños (ej. un Supervisor "inteligente" GPT-4o controlando a muchos Workers "rápidos" y baratos GPT-4o-mini).

¿Cuándo elegir Colaboración?

- Cuando el problema es **explorativo** (ej. investigación de mercado donde no sabes qué vas a encontrar).
 - Cuando los agentes necesitan trabajar en paralelo masivo sobre un mismo conjunto de datos.
 - En sistemas de tiempo real donde la latencia de un supervisor centralizado sería inaceptable.
-

5. Errores Comunes y Mejores Prácticas

Error 1: El "Supervisor Todólogo"

Un error frecuente es diseñar un supervisor que intenta hacer parte del trabajo de los agentes. Esto infla el prompt del supervisor, gasta tokens innecesarios y aumenta la probabilidad de alucinaciones.

- **Solución:** El supervisor debe ser un **director de tráfico**, no un ejecutor. Su prompt debe enfocarse exclusivamente en lógica de división de tareas y evaluación de resultados.

Error 2: Falta de Contratos de Interfaz

En topologías cooperativas, los desarrolladores a menudo olvidan definir *exactamente* cómo debe verse el dato que un agente pasa al siguiente.

- **Solución:** Utiliza **Pydantic** para definir esquemas robustos. Si el Agente A produce un JSON, el Agente B debe saber exactamente qué llaves esperar. En LangGraph, esto se gestiona mediante el `State` compartido.

Error 3: El Bucle Infinito de Corrección

En arquitecturas tipo Critic-Refiner (una variante de colaboración), es común que el Crítico siempre encuentre algo mal y el Refinador siempre intente arreglarlo, gastando presupuesto de API sin fin.

- **Solución:** Implementa un **contador de intentos** o una condición de parada basada en una métrica de calidad (ej. "detener si la puntuación de calidad es > 0.8 o si llevamos 3 iteraciones").
-

6. Síntesis: El camino hacia sistemas complejos

Diseñar sistemas multi-agente es elevarse un nivel por encima del código tradicional. Ya no solo programas instrucciones; estás diseñando una **organización digital**.

- La **Jerarquía** te da orden, seguridad y trazabilidad, pero introduce cuellos de botella.
- La **Colaboración** te da flexibilidad, velocidad y resiliencia, pero a costa de una mayor complejidad de depuración.

En las próximas unidades, veremos cómo implementar el **Patrón Supervisor** en código usando LangGraph, donde aprenderás a gestionar el estado entre estos agentes y a evitar que tu "director de orquesta" se sature. Recuerda: la mejor topología no es la más compleja, sino la más simple que logre resolver el problema de negocio con la menor latencia y costo posibles.

Conexión con lo que viene: Ahora que entiendes la teoría de cómo se organizan los agentes, en la siguiente unidad práctica meteremos las manos en la masa para construir un **Supervisor Agent**. Veremos cómo LangGraph facilita la delegación dinámica y cómo podemos usar nodos para representar a estos especialistas que acabamos de describir.

Práctica

Escenario: Sistema de Análisis de Sentimiento y Respuesta Automatizada

Eres el arquitecto de IA de una empresa de e-commerce. Debes proponer el diseño de una **topología jerárquica** para un sistema que recibe quejas de clientes por Twitter, analiza el sentimiento, verifica el estado del pedido en una base de datos y genera una respuesta personalizada.

1. **Identificación de Agentes:** Define al menos 3 agentes especialistas y un supervisor.
2. **Definición de Responsabilidades:** Para cada agente, describe brevemente qué hace y qué herramientas (tools) necesitaría.
3. **Diagrama de Flujo Lógico:** Describe el paso a paso de cómo fluye la información desde que llega el Tweet hasta que se envía la respuesta.

- 4. Justificación de Topología:** Explica por qué elegiste una topología jerárquica en lugar de una cooperativa para este caso específico de servicio al cliente.

Entrega: Un documento PDF o Markdown con la propuesta arquitectónica detallada.

Entregable

Tipo: FILE

Entrega: Un documento PDF con la propuesta arquitectónica detallada.

Topologías Multi-Agente: Colaboración vs Jerarquía

Explora las estructuras fundamentales de organización en sistemas multi-agente, analizando cuándo centralizar el control mediante jerarquías o descentralizarlo mediante colaboración para maximizar la eficiencia y modularidad.

Entregable

Tipo: FILE

Entrega: Un documento PDF con la propuesta arquitectónica detallada.

Instrucciones

Escenario: Sistema de Análisis de Sentimiento y Respuesta Automatizada

Eres el arquitecto de IA de una empresa de e-commerce. Debes proponer el diseño de una **topología jerárquica** para un sistema que recibe quejas de clientes por Twitter, analiza el sentimiento, verifica el estado del pedido en una base de datos y genera una respuesta personalizada.

- 1. Identificación de Agentes:** Define al menos 3 agentes especialistas y un supervisor.
- 2. Definición de Responsabilidades:** Para cada agente, describe brevemente qué hace y qué herramientas (tools) necesitaría.
- 3. Diagrama de Flujo Lógico:** Describe el paso a paso de cómo fluye la información desde que llega el Tweet hasta que se envía la respuesta.
- 4. Justificación de Topología:** Explica por qué elegiste una topología jerárquica en lugar de una cooperativa para este caso específico de servicio al cliente.

Entrega: Un documento PDF o Markdown con la propuesta arquitectónica detallada.

Objetivos de aprendizaje

- Diferenciar entre topologías jerárquicas y cooperativas en el diseño de sistemas multi-agente.

- Identificar los casos de uso ideales para cada topología basándose en la complejidad del problema y requisitos de control.
- Evaluar las implicaciones de escalabilidad, costo y latencia al elegir una estructura de coordinación de agentes.
- Reconocer patrones comunes como el Supervisor, Blackboard y Critic-Refiner en arquitecturas modernas de IA.

Diseño de Arquitecturas Multi-Agente: Jerarquía y Especialización

Esta rúbrica evalúa la capacidad del estudiante para diseñar sistemas de IA complejos utilizando topologías jerárquicas, definiendo roles especializados y flujos de control robustos en un escenario de atención al cliente.

Criterio	Descripción	Peso
Especialización y Rol del Supervisor	Capacidad para definir un supervisor que orqueste la tarea y agentes especialistas con roles diferenciados.	30%
Configuración de Herramientas (Tooling)	Selección y descripción de herramientas técnicas necesarias para que cada agente cumpla su función.	20%
Flujo Lógico y Control de Estado	Coherencia en la secuencia de pasos y transferencia de información entre nodos del sistema.	25%
Justificación Arquitectónica	Razonamiento crítico sobre la elección de una topología jerárquica frente a una colaborativa.	25%

Total: 100 pts | Aprobación: 65 pts

• Código

El Patrón Supervisor para Delegación Dinámica

Aprende a implementar el patrón Supervisor para coordinar múltiples agentes especialistas de forma dinámica utilizando LangGraph.

El Patrón Supervisor: Delegación Dinámica

En sistemas multi-agente, la complejidad crece exponencialmente con cada nuevo "agente" que añadimos. Si conectamos todos con todos (una topología de red), el ruido y la falta de control hacen que el sistema sea impredecible. Aquí es donde entra el **Patrón Supervisor**.

¿Qué es el Patrón Supervisor?

Imagina una redacción de noticias: tienes reporteros (investigadores) y redactores. No se mandan correos entre ellos de forma caótica; hay un **Editor Jefe** (el Supervisor) que recibe la noticia, decide quién debe investigar, revisa el borrador y finalmente aprueba la publicación.

En **LangGraph**, el Supervisor es un nodo central que:

1. **Analiza el estado actual:** Lee el historial de mensajes para entender qué se ha hecho.
2. **Toma una decisión de ruteo:** Decide qué especialista es el más apto para el siguiente paso.
3. **Valida el progreso:** Actúa como un "control de calidad", pudiendo pedir a un agente que repita su tarea si no es satisfactoria.

¿Por qué usar este patrón en lugar de una cadena simple?

- **Especialización de Modelos:** Puedes usar un modelo costoso (GPT-4o) para el Supervisor y modelos ligeros (Llama-3-8B) para los agentes especialistas.
- **Escalabilidad:** Añadir una nueva funcionalidad es tan sencillo como crear un nodo y registrarlo en las opciones del Supervisor.
- **Recuperación de Errores:** Si un agente falla, el Supervisor puede decidir reintentar o cambiar de estrategia.

Implementación en LangGraph

Para implementar este patrón, dependemos de las **Aristas Condicionales**. El supervisor no tiene una ruta fija; su salida determina el flujo dinámicamente.

Un error común es intentar que el Supervisor haga el trabajo de los especialistas. **Recuerda:** El Supervisor coordina, el Especialista ejecuta. Si tu Supervisor está escribiendo código, tu arquitectura está perdiendo la ventaja de la especialización.

En esta unidad, construiremos un orquestador que delegará tareas de análisis y escritura de forma inteligente.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
import asyncio
from typing import Annotated, Literal, TypedDict
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END
from langchain_core.messages import BaseMessage, HumanMessage
from langgraph.graph.message import add_messages
```

```

# TODO: Define el estado global del agente (AgentState)
# Debe incluir una lista de mensajes y una variable 'next' para la delegación.
class AgentState(TypedDict):
    pass

# TODO: Implementa el nodo Supervisor
# Debe usar el LLM para decidir si el siguiente paso es "Analista", "Escritor" o "FINALIZAR".
def supervisor_node(state: AgentState):
    pass

# Nodos especialistas (ya implementados para este ejercicio)
def analyst_node(state: AgentState):
    return {"messages": [HumanMessage(content="Datos procesados: +20% incremento.",
name="Analista")]}

def writer_node(state: AgentState):
    return {"messages": [HumanMessage(content="El informe está listo: Crecimiento positivo.",
name="Escritor")]}

# TODO: Configura el Grafo
# 1. Crea el StateGraph
# 2. Añade los nodos: 'supervisor', 'Analista', 'Escritor'
# 3. Añade aristas: Analista -> supervisor, Escritor -> supervisor
# 4. Añade la arista condicional desde 'supervisor' basada en el estado['next']
builder = StateGraph(AgentState)

# Implementa aquí...

# Compilar y ejecutar
# graph = builder.compile()

```

Código solución

```

import asyncio
from typing import Annotated, Literal, TypedDict
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END
from langchain_core.messages import BaseMessage, HumanMessage
from langgraph.graph.message import add_messages

# 1. Definición del Estado
class AgentState(TypedDict):

```

```

# Usamos add_messages para que los nuevos mensajes se concatenen al historial
messages: Annotated[list[BaseMessage], add_messages]
# 'next' indicará cuál es el siguiente nodo a ejecutar
next: str

# 2. Configuración del Supervisor
llm = ChatOpenAI(model="gpt-4o", temperature=0)

def supervisor_node(state: AgentState):
    """
    El Supervisor decide quién trabaja a continuación o si hemos terminado.
    """
    system_prompt = (
        "Eres un supervisor encargado de gestionar un flujo de trabajo entre los siguientes trabajadores: "
        "Analista, Escritor. Dada la petición del usuario, responde con el nombre del trabajador "
        "que debe actuar a continuación. Si la tarea está completa, responde FINALIZAR."
    )

    messages = [{"role": "system", "content": system_prompt}] + state["messages"]

    # Forzamos una salida estructurada para facilitar la delegación
    class Router(TypedDict):
        next_agent: Literal["Analista", "Escritor", "FINALIZAR"]

    response = llm.with_structured_output(Router).invoke(messages)
    return {"next": response["next_agent"]}

# 3. Especialistas
def analyst_node(state: AgentState):
    # Simulación de análisis
    last_msg = state["messages"][-1].content
    return {
        "messages": [HumanMessage(content=f"Análisis completado para: '{last_msg}'. Datos: [Ventas +20%]", name="Analista")]
    }

def writer_node(state: AgentState):
    # Simulación de redacción
    return {
        "messages": [HumanMessage(content="Informe redactado: El mercado muestra un crecimiento del 20% anual.", name="Escritor")]
    }

```

```

# 4. Construcción del Grafo
builder = StateGraph(AgentState)

builder.add_node("supervisor", supervisor_node)
builder.add_node("Analista", analyst_node)
builder.add_node("Escritor", writer_node)

# Aristas: Todos los especialistas regresan al supervisor para validación/nueva delegación
builder.add_edge("Analista", "supervisor")
builder.add_edge("Escritor", "supervisor")

# Arista condicional centrada en el Supervisor
builder.add_conditional_edges(
    "supervisor",
    lambda x: x["next"],
    {
        "Analista": "Analista",
        "Escritor": "Escritor",
        "FINALIZAR": END
    }
)

builder.set_entry_point("supervisor")
graph = builder.compile()

async def main():
    inputs = {"messages": [HumanMessage(content="Necesito un reporte sobre ventas.")]}
    async for output in graph.astream(inputs):
        for key, value in output.items():
            print(f"Nodo '{key}': {value.get('next', 'Procesando...')}")

if __name__ == "__main__":
    asyncio.run(main())

```

Pistas

- Para el Supervisor, utiliza el método `bind_tools` o `with_structured_output` para forzar al modelo a elegir entre una lista de agentes disponibles (ej. 'Analista', 'Escritor') o finalizar con 'FIN'.
- En la lógica de las aristas condicionales (`add_conditional_edges`), mapea el contenido del campo `next` del estado hacia los nombres de los nodos correspondientes.
- Recuerda que cada nodo especialista debe devolver un mensaje que se agregue al historial. El Supervisor usará ese historial para decidir si la tarea está completa.

Evaluación: Implementación del Patrón Supervisor en LangGraph

Esta rúbrica evalúa la capacidad del estudiante para diseñar y codificar un sistema multi-agente jerárquico. Se centra en la correcta delegación de tareas, la gestión del estado del grafo y la separación de responsabilidades entre el nodo supervisor y los nodos especialistas.

Criterio	Descripción	Peso
Lógica del Nodo Supervisor	Diseño y ejecución de la lógica de decisión del supervisor, incluyendo el ruteo dinámico basado en el estado.	35%
Configuración del Grafo y Aristas Condicionales	Uso correcto de StateGraph y la implementación de rutas dinámicas desde el supervisor hacia los agentes.	25%
Especialización de Nodos (Agentes)	Definición y comportamiento de los nodos especialistas encargados de tareas de análisis y escritura.	20%
Manejo de Estado y Concurrencia	Gestión del historial de mensajes y la integridad de los datos a través de los diversos nodos del sistema.	20%

Total: 100 pts | Aprobación: 65 pts

• Código

Estado Compartido y Comunicación Asíncrona

Aprende a gestionar el estado compartido y la comunicación no bloqueante entre agentes utilizando Asyncio, LangGraph y Pydantic para asegurar la integridad de los datos.

Estado Compartido y Comunicación Asíncrona

En sistemas multi-agente, la comunicación no es simplemente "enviar un mensaje". Se trata de gestionar la **coherencia del conocimiento** a medida que este evoluciona. Cuando trabajamos con agentes especializados, el mayor desafío no es que el LLM genere texto, sino que todos los componentes del sistema operen sobre una versión válida y actualizada de la realidad.

Por qué Asyncio es vital en Arquitecturas de IA

Los LLMs son intrínsecamente lentos. Una llamada a GPT-4 o Claude 3.5 puede tardar varios segundos. Si tienes 5 agentes trabajando de forma síncrona, el tiempo de respuesta total será la suma de todos los retardos.

Utilizar `asyncio` permite que mientras un agente espera la respuesta de una API, el orquestador pueda:

1. Procesar resultados de otros agentes.
2. Realizar escrituras en bases de datos vectoriales.

3. Mantener la interfaz de usuario receptiva (streaming).

El Estado Compartido: De Diccionarios a Esquemas Validados

En unidades anteriores usamos diccionarios simples. Sin embargo, en un sistema de grado industrial, necesitamos **contratos**.

Un error común es permitir que cualquier agente modifique cualquier parte del estado. Esto conduce a **condiciones de carrera** (race conditions) donde el Agente B sobrescribe lo que el Agente A acaba de calcular basándose en datos obsoletos.

Estrategia Recomendada:

- **Aislamiento de Escritura:** Cada agente es dueño de una "parcela" del estado. Por ejemplo, el agente `researcher` solo escribe en `state["research_results"]`.
- **Validación con Pydantic:** Antes de "inyectar" datos en el estado compartido, los validamos contra un modelo de Pydantic. Si el LLM alucina un formato de fecha incorrecto o un score fuera de rango, el error se captura inmediatamente.

Consistencia Eventual

En sistemas asíncronos, aceptamos que por unos milisegundos, algunos agentes podrían ver una versión del estado ligeramente antigua. Esto se llama **Consistencia Eventual**. En LangGraph, gestionamos esto mediante el flujo de nodos: un nodo solo se ejecuta cuando tiene la garantía de que sus dependencias (aristas de entrada) han completado su escritura.

Errores Comunes

1. **Shared Mutable State:** Intentar usar una lista global de Python y hacer `.append()` desde múltiples funciones `async`. Esto fallará en entornos distribuidos o bajo alta concurrencia. Usa siempre el mecanismo de retorno de estados del grafo.
2. **No manejar excepciones en Nodos:** Si un nodo asíncrono falla y no se captura, puede dejar el estado del sistema en un limbo ("bloqueado"). Siempre valida los outputs del LLM antes de retornarlos al grafo.

Ejercicio de código

Lenguaje: plaintext

Código inicial

```
from typing import Annotated, TypedDict, List, Optional
from pydantic import BaseModel, Field, ValidationError
from langgraph.graph import StateGraph, END
import asyncio
```

```

# TODO: Define ResearchArtifact y ReviewArtifact usando Pydantic
# Asegúrate de incluir validaciones (ej. score entre 0 y 1)

class AgentState(TypedDict):
    topic: str
    # TODO: Añadir campos para researcher_output y reviewer_output
    pass

# TODO: Implementar el nodo de investigación asíncrono
async def researcher_node(state: AgentState):
    # 1. Crear un ResearchArtifact
    # 2. Retornar la actualización del estado para 'researcher_output'
    return {}

# TODO: Implementar el nodo de revisión asíncrono
async def reviewer_node(state: AgentState):
    # 1. Leer researcher_output del estado
    # 2. Validar y crear un ReviewArtifact
    # 3. Retornar la actualización para 'reviewer_output'
    return {}

# Configuración del Grafo
workflow = StateGraph(AgentState)

# TODO: Añadir nodos y definir el flujo (entry point, edges)

app = workflow.compile()

async def run_example():
    initial_state = {"topic": "Sistemas Multi-Agente"}
    # TODO: Ejecutar el grafo usando app.astream
    pass

if __name__ == "__main__":
    asyncio.run(run_example())

```

Código solución

```

from typing import Annotated, TypedDict, List, Optional
from pydantic import BaseModel, Field, ValidationError
from langgraph.graph import StateGraph, END

```

```

# 1. Definición del Esquema de Validación con Pydantic
class ResearchArtifact(BaseModel):
    """Esquema para los datos de investigación."""
    topic: str
    findings: List[str] = Field(default_factory=list)
    confidence_score: float = Field(ge=0.0, le=1.0)

class ReviewArtifact(BaseModel):
    """Esquema para la revisión de calidad."""
    is_approved: bool
    feedback: Optional[str] = None

# 2. Definición del Estado del Grafo (TypedDict)
class AgentState(TypedDict):
    # Usamos anotaciones para definir cómo se actualiza el estado si es necesario
    # En este caso, cada agente escribe en su propia sección
    topic: str
    researcher_output: Optional[ResearchArtifact]
    reviewer_output: Optional[ReviewArtifact]

# 3. Implementación de Nodos Asíncronos
async def researcher_node(state: AgentState):
    print(f"--- Investigando: {state['topic']} ---")

    # Simulación de procesamiento asíncrono y validación
    try:
        data = ResearchArtifact(
            topic=state['topic'],
            findings=["Descubrimiento A", "Descubrimiento B"],
            confidence_score=0.95
        )
        # Solo modificamos nuestra sección correspondiente
        return {"researcher_output": data}
    except ValidationError as e:
        print(f"Error de validación en Researcher: {e}")
        raise

async def reviewer_node(state: AgentState):
    print("--- Revisando resultados ---")

    research_data = state.get("researcher_output")
    if not research_data:
        return {"reviewer_output": ReviewArtifact(is_approved=False, feedback="No hay datos")}

    # Simulación de lógica de revisión

```



```

    approved = research_data.confidence_score > 0.8
    review = ReviewArtifact(
        is_approved=approved,
        feedback="Excelente profundidad" if approved else "Baja confianza"
    )

    return {"reviewer_output": review}

# 4. Orquestación del Grafo
workflow = StateGraph(AgentState)

workflow.add_node("investigador", researcher_node)
workflow.add_node("revisor", reviewer_node)

workflow.set_entry_point("investigador")
workflow.add_edge("investigador", "revisor")
workflow.add_edge("revisor", END)

# Compilar el grafo
app = workflow.compile()

# Ejemplo de ejecución asíncrona
import asyncio

async def main():
    inputs = {"topic": "Computación Cuántica"}
    async for event in app.astream(inputs):
        for node, value in event.items():
            print(f"Nodo '{node}' finalizado.")

if __name__ == "__main__":
    asyncio.run(main())

```

Pistas

- Usa `TypedDict` para definir la estructura del estado, pero utiliza la clase `BaseModel` de Pydantic para validar los datos que llegan desde el exterior o antes de guardarlos. LangGraph pasará este TypedDict entre nodos.
- Asegúrate de que cada función sea `async def`. En sistemas de IA escalables, las llamadas a LLMs y bases de datos son el cuello de botella; usar async permite que el sistema maneje otras tareas mientras espera la respuesta del modelo.
- Para evitar que los agentes se sobrescriban entre sí (conflictos de escritura), diseña el estado de forma que cada agente tenga su propia 'clave' o sub-objeto dentro del estado principal. Por ejemplo,

`state['research_data']` para el investigador y `state['review_feedback']` para el revisor.

Rúbrica: Estado Compartido y Comunicación Asíncrona en Sistemas Multi-Agente

Evaluación del diseño de sistemas multi-agente enfocada en la gestión concurrente del estado, validación de datos con Pydantic y el aprovechamiento de la programación asíncrona para optimizar la latencia en arquitecturas de IA.

Criterio	Descripción	Peso
Gestión Asíncrona con asyncio	Capacidad para implementar llamadas concurrentes a modelos y procesos, optimizando los tiempos de espera del sistema.	30%
Validación de Estado con Pydantic	Uso de modelos de datos para garantizar que la información compartida entre agentes sea coherente y libre de alucinaciones de formato.	30%
Aislamiento y Coherencia del Estado	Diseño de la lógica de escritura en el estado compartido para evitar condiciones de carrera y sobreescrituras accidentales.	25%
Robustez y Manejo de Excepciones	Capacidad del sistema para recuperarse de errores en nodos asíncronos y evitar estados de bloqueo.	15%

Total: 100 pts | Aprobación: 65 pts

Autoevaluación

Cuestionario: Validación y Resolución de Conflictos

Evaluación de estrategias para asegurar la fiabilidad en sistemas multi-agente mediante validación multi-paso y resolución de conflictos.

Introducción a la Validación y Resolución de Conflictos

En la construcción de sistemas multi-agente, la orquestación no termina cuando un agente entrega una respuesta. La naturaleza probabilística de los LLMs y la complejidad de los flujos de trabajo multi-paso introducen un riesgo crítico: el **error compuesto**. Una pequeña alucinación o una inconsistencia en el primer paso del razonamiento puede descarrilar toda la colaboración.

Como arquitectos de IA, nuestro rol es implementar mecanismos de **validación multi-paso** y **resolución de conflictos**. Esto implica diseñar "puertas de calidad" que verifiquen no solo la sintaxis del JSON, sino la coherencia semántica entre lo que un agente recuperó y lo que otro sintetizó.

Este cuestionario evaluará tu capacidad para:

- 1. Identificar estrategias efectivas para resolver contradicciones entre agentes.

2. Diseñar bucles de reparación (*repair loops*) que no dependan únicamente de la confianza del modelo.
3. Aplicar principios de ingeniería como la validación determinística y la observabilidad en flujos complejos.

¡Mucha suerte!

Quiz

Aprobación: 70 · Intentos máximos: 3

1. Estás diseñando un agente de extracción de datos financieros. El modelo suele extraer fechas inexistentes (ej. 31 de noviembre). ¿Cuál es la estrategia de validación más robusta según los principios de ingeniería de software?

- ☐ Aumentar la temperatura del modelo para que sea más creativo al corregir.
- ☐ Asumir que si el texto es fluido y profesional, la fecha es correcta.
- ☒ Implementar una capa de validación determinística con Pydantic o lógica de Python.
- ☐ Preguntar a un segundo agente si la fecha le "parece" razonable.

Explicación: El concepto clave aquí es la **Validación Determinística**. El error de "alucinación de fechas" (como el 30 de febrero) es un fallo de lógica factual que un LLM puede cometer repetidamente si solo se le pide que "revise". La solución más robusta y escalable es usar un validador determinístico (como Pydantic o una función de Python con `datetime`) para verificar la regla de negocio antes de permitir que el flujo continúe.

- **Pydantic/Regex (Correcto):** Garantiza que la fecha cumpla con reglas lógicas inamovibles.
- **Aumentar la temperatura (Incorrecto):** Esto incrementa la aleatoriedad, haciendo que el error sea más probable o impredecible.
- **Confiar en la fluidez (Incorrecto):** Un modelo puede ser muy persuasivo y fluido mientras afirma algo físicamente imposible.
- **Consultar a otro agente (Menos eficiente):** Aunque es posible, es costoso e innecesario para reglas que el código puede resolver con 100% de certeza.

2. Cuando una validación falla en un sistema multi-agente, ¿cómo se debería estructurar idealmente el bucle de reparación (recovery loop)?

- ☒ Re-enviar el output fallido junto con el mensaje de error específico del validador para que el agente corrija solo la parte afectada.

- ☐ Reiniciar el turno del agente con el mismo prompt original para ver si el azar soluciona el problema.
- ☐ Limpiar la memoria de la conversación para evitar que el error anterior "contamine" el nuevo intento.
- ☐ Aumentar el valor de top-p para que el modelo ignore sus probabilidades previas.

Explicación: En un **bucle de reflexión o reparación**, la clave es la especificidad. Si simplemente le pides al agente que "intente de nuevo", es muy probable que repita el mismo error (el sesgo del modelo). Al enviarle un mensaje de error detallado extraído de la validación (ej. el stacktrace de Pydantic), el agente recibe una señal clara de *qué* debe cambiar, lo que aumenta drásticamente la tasa de éxito de la reparación.

- **Error específico (Correcto):** Proporciona contexto de ingeniería al modelo probabilístico.
- **Prompt idéntico (Incorrecto):** Sin información nueva, el modelo tiende a generar el mismo resultado.
- **Limpiar memoria (Incorrecto):** Elimina el contexto del error, impidiendo que el agente aprenda de la falla anterior.
- **Aumentar top-p (Tangencial):** No aborda el problema de la falta de información sobre el fallo.

3. Un sistema tiene dos agentes: un 'Recuperador' que extrae datos de una base vectorial y un 'Analista' que genera conclusiones. Si el Analista contradice los datos recuperados, ¿qué técnicas serían adecuadas para resolver el conflicto? (Selecciona todas las que apliquen)

- ☐ Seleccionar siempre la respuesta del agente con la salida de texto más larga.

☒ **Implementar trazabilidad de proveniencia (provenance) para verificar qué fuentes respaldan cada afirmación en conflicto.**

☒ **Utilizar un agente 'Árbitro' o 'Crítico' especializado en detectar contradicciones y solicitar evidencia a los agentes implicados.**

- ☐ Implementar una votación simple y elegir la opción que la mayoría de los agentes propongan.

Explicación: Resolver conflictos no se trata de elegir la respuesta más "larga" o popular, sino la más fundamentada.

- **Trazabilidad/Provenza (Correcto):** Es vital saber de dónde viene cada afirmación para decidir cuál es verídica. Si el Agente A cita un PDF y el Agente B cita un tweet, la procedencia ayuda a priorizar.
- **Agente Árbitro/Crítico (Correcto):** Un patrón común es usar un agente cuya única función sea detectar inconsistencias entre sus pares y pedir aclaraciones.
- **Promedio/Mayoría (Incorrectos):** En IA, dos agentes pueden alucinar lo mismo si usan el mismo modelo previo. El consenso no garantiza la verdad factual.

- **Longitud (Incorrecto):** Un error detallado sigue siendo un error.

4. Para asegurar que una validación multi-paso sea escalable y no introduzca latencia excesiva, ¿cuál es una mejor práctica en la gestión del estado del grafo?

☒ **Diseñar el estado (State) para que solo pase el contexto mínimo necesario entre nodos, reduciendo la confusión del modelo.**

- ☐ Pasar siempre el historial completo de la conversación a todos los agentes para que no pierdan contexto.
- ☐ Evitar el uso de clases typed (TypedDict/Dataclasses) y preferir cadenas de texto planas para mayor flexibilidad.
- ☐ Asegurar que todos los agentes se ejecuten en paralelo sin importar si uno depende del output del otro.

Explicación: En arquitecturas como LangGraph, la **gestión de estados** es el corazón de la confiabilidad. Pasar toda la conversación ("ruido") a cada paso diluye las instrucciones y aumenta el riesgo de que el modelo ignore las restricciones de validación.

- **Mínimo estado necesario (Correcto):** Al pasar solo los datos relevantes para la tarea actual, reducimos la carga cognitiva del modelo y aumentamos la precisión del cumplimiento de esquemas.
- **History completo (Incorrecto):** Provoca el fenómeno de 'distracción' del modelo, donde las instrucciones de validación pierden peso frente al texto previo.
- **Strings planos (Incorrecto):** Pierden la estructura necesaria para las validaciones automáticas.
- **Simultaneidad (Tangencial):** No aborda el problema de la consistencia entre pasos.

5. Si un sistema multi-agente llega al máximo de reintentos en una validación semántica (ej. el agente no logra citar fuentes reales), ¿cuál es la acción recomendada en un entorno de producción?

☒ **Detener el proceso, registrar el fallo en el sistema de observabilidad y escalar el caso a una revisión humana (Human-in-the-loop).**

- ☐ Configurar un bucle infinito hasta que el modelo finalmente entregue un formato válido.
- ☐ Asumir que el último agente ejecutado tiene la razón y descartar los errores de validación previos.
- ☐ Pedirle a un agente generativo que "invente" los datos faltantes para completar el esquema y no romper el flujo.

Explicación: En aplicaciones críticas (legal, salud, finanzas), el sistema debe ser capaz de **abstenerse** si la validación falla repetidamente o si hay conflictos irresolubles.

- **Escalamiento humano/Fallo seguro (Correcto):** Es preferible decir "No puedo procesar esto con seguridad" que entregar una respuesta potencialmente peligrosa.
- **Loop infinito (Incorrecto):** Consume tokens indefinidamente (costo y latencia) sin garantía de éxito.
- **Asumir verdad (Incorrecto):** Es la definición de permitir una alucinación en el sistema.
- **Reescribir automáticamente (Incorrecto):** Si el conflicto es profundo, obligar a una reescritura suele resultar en una alucinación más creativa pero igual de falsa.

• Pre-entrega

Pre-entrega 6: Orquestador multi-agente especializado

Briefing de la pre-entrega final del módulo, enfocada en la construcción de un sistema jerárquico de supervisor y agentes especializados utilizando LangGraph.

Pre-entrega 6: Orquestador multi-agente especializado

¿Qué debes construir?

Debes entregar un prototipo funcional de un **Orquestador Multi-Agente de Análisis e Investigación**. El sistema debe ser capaz de procesar una solicitud de usuario que requiera al menos dos dominios de especialización distintos y una fase de síntesis final.

Requerimientos Técnicos:

1. **Topología Jerárquica:** Un nodo `Supervisor` que actúa como router inteligente y controlador de flujo.
2. **Mínimo de 2 Agentes Especialistas:**
 - **Agente de Búsqueda/Investigación:** Con herramientas para consultar fuentes externas (puedes usar Tavily o una búsqueda simulada sobre tu Vector DB de pre-entregas anteriores).
 - **Agente de Análisis/Cómputo:** Especializado en procesar los datos obtenidos (análisis de sentimiento, cálculos matemáticos o validación de esquemas).
3. **Estado Compartido Estructurado:** Un esquema de `State` en LangGraph que permita rastrear qué agente ha contribuido con qué información, evitando la pérdida de contexto en la comunicación asíncrona.
4. **Flujo de Supervisión:** El supervisor debe decidir si la tarea está completa o si algún especialista debe refinar su output antes de dar la respuesta final.

Pasos Sugeridos

1. **Define tu Estado:** Diseña una clase `TypedDict` que herede de `MessagesState`. Piensa si necesitas campos extra como `next_agent` o `task_completed`.
2. **Crea los Agentes Especialistas:** Define funciones que utilicen `create_react_agent` de LangGraph o prompts específicos para cada rol. Asegúrate de que cada uno tenga herramientas acotadas.

3. **Implementa el Supervisor:** Este es el cerebro. Su prompt debe ser claro: "Dada la conversación actual, ¿quién debe intervenir ahora o es momento de finalizar?". Debe mapear sus respuestas a los nombres de los nodos del grafo.
4. **Construye el Grafo:** Une los puntos. Usa `add_node` para cada agente y para el supervisor. Define las `Conditional Edges` que conecten al supervisor con los especialistas.
5. **Prueba la Interacción:** Lanza una consulta que obligue al supervisor a enviar la tarea al Investigador, recibir el dato, enviarlo al Analista y finalmente cerrar la conversación.

Errores Comunes a Evitar

- **El "Supervisor Infinito":** No definir una condición de parada clara, haciendo que el supervisor y los agentes entren en un bucle de correcciones eternas. Tip: Implementa un contador de pasos o un criterio de "Suficiencia" estricto.
 - **Contaminación de Contexto:** Pasar todo el historial a todos los agentes en cada turno. Para sistemas escalables, a veces es mejor que el especialista solo reciba la instrucción específica y el contexto necesario, no toda la metadata del sistema.
-

Práctica

Instrucciones de la Práctica

1. Estructura del Repositorio:

- Crea un archivo `state.py` para definir el esquema de datos compartido.
- Crea un directorio `agents/` con los especialistas (ej. `research_agent.py`, `analyst_agent.py`).
- Define el grafo principal en `main.py` o `graph.py`.

2. Implementación del Grafo:

- Utiliza `StateGraph` de LangGraph.
- Define al menos un nodo de "Supervisor" que decida dinámicamente el flujo usando `Literal` en el retorno para las aristas condicionales.

3. Herramientas (Tools):

- Los agentes deben tener al menos una herramienta funcional (puedes usar `TavilySearchResults` o una función propia que consulte una base de datos).

4. Validación:

- Implementa un nodo de `Validation` o asegura que el `Supervisor` tenga una rúbrica personalizada en su prompt para validar los resultados de los especialistas antes de dar el `END`.

5. Documentación:

- Tu README debe incluir un diagrama del grafo (puedes generarlo con `graph.get_graph().draw_mermaid_png()`).
- Explica brevemente por qué elegiste esa topología y cómo manejas los posibles conflictos entre agentes.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código del orquestador, un archivo README explicando la topología elegida y un video corto o notebook demostrando el flujo de delegación.

Pre-entrega: Orquestador Multi-Agente Especializado

Briefing de la pre-entrega final del módulo, enfocada en la construcción de un sistema jerárquico de supervisor y agentes especializados utilizando LangGraph.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código del orquestador, un archivo README explicando la topología elegida y un video corto o notebook demostrando el flujo de delegación.

Instrucciones

Instrucciones de la Práctica

1. Estructura del Repositorio:

- Crea un archivo `state.py` para definir el esquema de datos compartido.
- Crea un directorio `agents/` con los especialistas (ej. `research_agent.py` , `analyst_agent.py`).
- Define el grafo principal en `main.py` o `graph.py` .

2. Implementación del Grafo:

- Utiliza `StateGraph` de LangGraph.
- Define al menos un nodo de "Supervisor" que decida dinámicamente el flujo usando `Literal` en el retorno para las aristas condicionales.

3. Herramientas (Tools):

- Los agentes deben tener al menos una herramienta funcional (puedes usar `TavilySearchResults` o una función propia que consulte una base de datos).

4. Validación:

- Implementa un nodo de `Validation` o asegura que el `Supervisor` tenga una rúbrica personalizada en su prompt para validar los resultados de los especialistas antes de dar el `END`.

5. Documentación:

- Tu README debe incluir un diagrama del grafo (puedes generarlo con `graph.get_graph().draw_mermaid_png()`).
- Explica brevemente por qué elegiste esa topología y cómo manejas los posibles conflictos entre agentes.

Objetivos de aprendizaje

- Diseñar arquitecturas de supervisor agent para la coordinación de tareas complejas.
- Implementar comunicación asíncrona y estado compartido entre agentes mediante LangGraph.
- Configurar lógica de validación de outputs en flujos multi-paso para garantizar la calidad de la respuesta.
- Aplicar principios de ingeniería como tipado estático y manejo de errores asíncronos en sistemas multi-agente.

Rúbrica de Evaluación: Orquestador Multi-Agente Especializado

Evaluación de la capacidad para diseñar y ejecutar arquitecturas multi-agente jerárquicas utilizando LangGraph, con énfasis en la gestión de estado, delegación dinámica y validación de resultados.

Criterio	Descripción	Peso
Documentación y Visualización	Claridad en la explicación de la topología y representación visual del grafo en el README.	20%
Arquitectura y Flujo del Grafo (Supervisor)	Evaluación del diseño del StateGraph y la implementación del nodo Supervisor para la delegación dinámica.	35%
Especialización de Agentes y Herramientas	Capacidad de los agentes para ejecutar tareas específicas mediante el uso de herramientas funcionales.	25%
Gestión de Estado y Validación	Uso de state.py para la coherencia de datos y presencia de mecanismos de validación de resultados.	20%

Total: 100 pts | Aprobación: 65 pts

• Clase en vivo

Sesión en vivo 6. Sistemas multi-agente: colaboración y especialización

Sesión interactiva final para consolidar el diseño de sistemas multi-agente, enfocada en la especialización de roles, protocolos de comunicación y orquestación de flujos complejos.

Sesión en Vivo: Maestría en Sistemas Multi-Agente

¡Bienvenidos a la sesión de cierre del módulo! En esta clase en vivo, daremos el salto de la teoría del diseño de agentes individuales a la orquestación de ecosistemas complejos. No solo repasaremos lo aprendido, sino que pondremos a prueba nuestra capacidad para diseñar colaboración real.

¿Qué aprenderás hoy?

En esta sesión de 2 horas, consolidarás tu visión sobre cómo los sistemas de IA pueden superar las limitaciones de un modelo único mediante la **especialización operativa**. Aprenderás a identificar cuándo un sistema necesita una jerarquía rígida y cuándo una red de colaboración fluida.

Temas clave

- **Especialización Funcional:** ¿Por qué un "experto en todo" suele ser menos eficiente que un "equipo de especialistas"?
- **Mecanismos de Coordinación:** Desde supervisores centrales hasta protocolos de comunicación asíncrona.
- **Resolución de Conflictos:** Estrategias para cuando los agentes tienen objetivos locales contrapuestos.
- **Analogías de la Industria:** Analizaremos cómo sistemas reales (como las redes eléctricas inteligentes) operan bajo principios multi-agente.

Preparación Previa

Para aprovechar al máximo esta sesión, es fundamental que:

1. Hayas completado la **Pre-entrega del Orquestador Multi-Agente**.
2. Tengas a mano el diagrama de tu arquitectura diseñada en la unidad anterior.
3. Reflexiones sobre esta pregunta: *¿Qué es más difícil de programar: la inteligencia del agente o su capacidad de comunicarse?*

Actividades

- **Desafío "Agente Colectivo":** Un ejercicio de simulación donde los estudiantes asumen roles de agentes para resolver un puzzle de recursos limitados.
- **Clínica de Arquitectura:** Revisión en vivo de diseños propuestos por los estudiantes para identificar cuellos de botella en la comunicación.

- **Debate: Autonomía vs. Control:** ¿Cuánta libertad debe tener un agente antes de que el sistema se vuelva impredecible?
-

Clase en vivo

Materiales de apoyo

Puntos clave:

- **Especialización:** División del trabajo donde cada agente domina un dominio limitado, reduciendo la ventana de contexto necesaria y aumentando la precisión.
- **Estado Compartido:** Mecanismo centralizado o distribuido para que los agentes conozcan el progreso global sin saturar sus mensajes.
- **Supervisión vs. Coreografía:** En la supervisión, un agente manda. En la coreografía, los agentes reaccionan a eventos de forma autónoma.

Ejemplos sugeridos:

- **El ONS Eléctrico:** Un coordinador central que gestiona agentes heterogéneos (hidroeléctricas, eólicas) que tienen restricciones y capacidades distintas para mantener la red estable.
- **Redacción Editorial:** Un agente busca noticias, otro verifica hechos, un tercero escribe y un cuarto genera imágenes. Ninguno sabe hacer el trabajo del otro.

Herramientas recomendadas:

- LangGraph (para orquestación de ciclos)
- CrewAI (para definiciones de roles fáciles)
- AutoGen (para agentes conversacionales complejos)

Actividades

- **El Puzzle de la Colaboración Interrumpida** (30 min, GROUP_WORK)
 - **Dinámica de Especialistas**

Setup: Dividir a la clase en grupos de 4 personas. Cada integrante recibe un "Role Card" virtual:

1. **Investigador:** Solo puede buscar datos crudos.
2. **Analista:** Solo puede comparar datos provistos por el Investigador.
3. **Escritor:** Solo puede redactar basándose en el análisis.
4. **Crítico:** Solo puede pedir correcciones o aprobar.

Ejecución: Se les asigna una tarea compleja (ej. "Diseñar un plan de mitigación para un corte eléctrico nacional"). Los estudiantes deben comunicarse a través del chat siguiendo estrictamente sus roles. **Cierre:**

Reflexión sobre qué pasó cuando un rol intentó hacer el trabajo de otro (sobrecarga vs. eficiencia).

Troubleshooting: Si un grupo se queda estancado, el instructor actúa como 'Supervisor' forzando la delegación.

- **Demo: Orquestador vs. Generalista** (25 min, DEMO)

- **Análisis Comparativo**

Setup: Se presenta un problema de optimización de rutas logísticas. **Ejecución:**

1. El instructor muestra cómo un único agente (LLM generalista) resuelve el problema (posibles errores de contexto o alucinaciones en datos finos).
2. Se presenta el mismo problema resuelto por un enjambre de 3 agentes (Logística, Clima, Inventario) coordinados por un Supervisor. **Dinámica:** Los estudiantes deben identificar 3 puntos exactos donde la especialización evitó un error común del modelo generalista.

Tiempos

Sección	Minutos
Introducción	10
Contenido principal	25
Actividades	55
Preguntas y respuestas	20
Cierre	10
Total	120

- **Podcast**

**Sistemas multi-agente: colaboración y especialización –
PODCAST**

Un episodio de repaso integral que conecta las topologías, los patrones de diseño y los desafíos de comunicación en sistemas multi-agente para consolidar el aprendizaje del módulo.

En este episodio de cierre, recopilamos todo lo aprendido sobre el diseño y la implementación de sistemas multi-agente. A lo largo del módulo, hemos pasado de la teoría de las topologías a la práctica del código con patrones de supervisión y comunicación asíncrona.

Al escuchar este podcast, podrás conectar los puntos entre:

- Las **topologías de colaboración y jerarquía** y cuándo elegir cada una según la complejidad de la tarea.

- La implementación del **patrón Supervisor** para centralizar la toma de decisiones y la delegación dinámica.
- Los retos técnicos del **estado compartido** y por qué la comunicación asíncrona es vital para la resiliencia del sistema.
- La importancia de la **resolución de conflictos** y la validación en sistemas donde los agentes pueden tener visiones parciales o contradictorias.

Este es un repaso diseñado para consolidar tu conocimiento técnico y ofrecerte una visión arquitectónica integral antes de finalizar el módulo.

MÓDULO 7

Producción y robustez: observabilidad, costos y despliegue

Asegurar la robustez del sistema mediante observabilidad detallada, control de costos y despliegue asíncrono bajo estándares de ingeniería.

• Lectura

Observabilidad y Trazado: Arize Phoenix y LangSmith

Aprende a implementar sistemas de observabilidad y trazado avanzado utilizando Arize Phoenix y LangSmith para monitorear latencia, costos y calidad en aplicaciones de IA de producción.

Observabilidad y Trazado: Arize Phoenix y LangSmith

Bienvenido al Módulo 7. Hasta este punto, has construido sistemas de IA capaces de razonar, recuperar información y colaborar en arquitecturas multi-agente. Sin embargo, en el mundo del software de producción, **"funciona en mi computadora"** es una frase peligrosa. Los sistemas basados en LLMs son intrínsecamente probabilísticos y "opacos": pueden fallar sin lanzar una excepción `RuntimeError`, simplemente entregando una respuesta que suena convincente pero es fácticamente incorrecta (alucinación) o tardando 30 segundos en un paso de recuperación que debería tomar 200ms.

En esta unidad, aprenderás a implementar **observabilidad de nivel empresarial** utilizando dos de las herramientas más potentes del ecosistema: **Arize Phoenix** y **LangSmith**. Pasaremos de "adivinar" qué sucede en tus cadenas a tener una visibilidad total sobre cada span, cada token y cada milisegundo.

1. El Problema: El "Éxito Silencioso" y la Opacidad

En el desarrollo de software tradicional, un error suele ser binario: el código rompe o funciona. En la ingeniería de IA, enfrentamos el fenómeno del **"éxito silencioso"**:

- El usuario recibe una respuesta bien formateada.
- El status code de la API es 200 OK.
- **Pero:** El agente decidió ignorar las herramientas, usó un contexto recuperado irrelevante o alucinó una cifra financiera crítica.

Sin observabilidad, estás volando a ciegas. La observabilidad no es solo "loggear" texto; es la capacidad de reconstruir el estado interno del sistema a partir de sus datos externos (trazas, métricas y evaluaciones).

2. Conceptos Fundamentales: Trazas, Spans y Runs

Para dominar herramientas como Arize Phoenix o LangSmith, primero debemos entender la anatomía de una ejecución de IA:

- 1. Trace (Trazo):** Es el viaje completo de una solicitud, desde que el usuario envía un mensaje hasta que recibe la respuesta final. Por ejemplo, una traza de un agente multi-paso incluiría todas las iteraciones de razonamiento.
- 2. Span (Lapso):** Es una unidad individual de trabajo dentro de una traza. Un span puede ser una llamada a una base de datos vectorial, una invocación al modelo (LLM Call), o la ejecución de una herramienta (Tool Call). Los spans son jerárquicos: una traza es un árbol de spans.
- 3. Run (Ejecución):** En el contexto de LangSmith, se refiere a la instancia de ejecución de un `Runnable` de LangChain.

¿Por qué separar en Spans?

Imagina que tu sistema tarda 10 segundos en responder. Si solo tienes un log del inicio y el final, no sabes si el problema fue la latencia de OpenAI, una búsqueda lenta en Pinecone o un bucle infinito en tu lógica de LangGraph. Los spans te permiten aislar el cuello de botella instantáneamente.

3. Arize Phoenix vs. LangSmith: ¿Cuál elegir?

Aunque ambas herramientas resuelven problemas similares, sus filosofías y casos de uso difieren.

Arize Phoenix: El Estándar Abierto y Local-First

Phoenix es una plataforma de observabilidad de código abierto que se integra profundamente con el estándar **OpenInference** (una extensión de OpenTelemetry para IA).

- **Ventaja Principal:** Es *framework-agnostic*. No importa si usas LangChain, LlamaIndex o el SDK puro de OpenAI; si emites datos compatibles con OpenTelemetry, Phoenix los visualiza.
- **Privacidad:** Puedes correrlo localmente en un contenedor Docker o directamente en un entorno de Notebook. Los datos no tienen que salir de tu infraestructura para ser analizados.
- **Enfoque:** Análisis de *embeddings*, detección de *drift* (deriva de datos) y evaluaciones de RAG (groundedness).

LangSmith: La Experiencia Integrada de LangChain

Desarrollado por el equipo detrás de LangChain, es la opción natural si tu stack está construido sobre LCEL (LangChain Expression Language) o LangGraph.

- **Ventaja Principal:** Integración "plug-and-play". Con solo configurar dos variables de entorno, cada paso de tu aplicación LangChain se registra automáticamente con una granularidad asombrosa.

- **Debugging:** Permite editar y re-ejecutar prompts directamente desde la interfaz para ver cómo cambia el output sin necesidad de reiniciar tu servidor.
 - **Datasets:** Muy fuerte en la creación de conjuntos de datos de prueba a partir de trazas de producción para realizar pruebas de regresión.
-

4. Implementación y Trazado Avanzado

El Enfoque OpenTelemetry en Arize Phoenix

Para un arquitecto de IA, la portabilidad es clave. Phoenix utiliza **OpenInference**, lo que significa que el código que escribas para observar tu sistema seguirá siendo válido incluso si decides cambiar de proveedor de backend de observabilidad en el futuro.

Al instrumentar una aplicación con Phoenix, el flujo de datos es:

1. **Instrumentation:** Librerías que "envuelven" tus llamadas a OpenAI/Anthropic/LangChain.
2. **Exporter:** Envía los datos capturados al colector de Phoenix.
3. **Frontend:** Visualización de trazas en `localhost:6006`.

Monitoreo de Latencia y Costos

Uno de los mayores errores de principiante es monitorear solo la latencia promedio. En producción, lo que importa son los **percentiles (p95, p99)**.

- **p50:** La mediana del tiempo de respuesta.
- **p95:** El tiempo en el que el 5% de tus usuarios están experimentando retrasos. Si tu p95 es de 45 segundos, tienes un problema grave de experiencia de usuario, incluso si tu promedio es de 3 segundos.

Tanto Phoenix como LangSmith calculan automáticamente el costo basado en los **tokens de entrada y salida** informados en los metadatos de la respuesta del modelo, permitiéndote detectar qué agentes o usuarios están consumiendo el presupuesto de forma desproporcionada.

5. Detección de Alucinaciones y Evaluadores (Evals)

El monitoreo de software tradicional usa "checks" booleanos. En IA, usamos **Evaluadores** (o *LLM-as-a-Judge*).

La Tríada de Evaluación en RAG:

Cuando implementamos RAG (como vimos en el Módulo 3 y 4), la observabilidad nos permite medir tres métricas críticas para detectar alucinaciones:

1. **Faithfulness (Fidelidad):** ¿La respuesta del LLM se deriva exclusivamente del contexto recuperado? Si el LLM usa conocimiento interno para responder algo que no estaba en los documentos, se considera una alucinación (incluso si la respuesta es correcta en el mundo real), porque rompe el control del sistema.
2. **Answer Relevance:** ¿La respuesta realmente aborda la pregunta del usuario?
3. **Context Precision:** ¿Los documentos recuperados de la Vector DB eran realmente útiles para responder la pregunta?

Ejemplo de Error Común: Muchos desarrolladores intentan solucionar una mala respuesta cambiando el prompt, cuando la traza en Phoenix muestra que el problema es de **Context Recall**: el retriever no trajo la información necesaria de Pinecone. Sin ver el span de recuperación, estarías optimizando el componente equivocado.

6. Errores Comunes y Mejores Prácticas

Error 1: Instrumentar demasiado tarde

No esperes a llegar al despliegue para activar LangSmith o Phoenix. La observabilidad es una herramienta de **desarrollo**. Ver las trazas mientras programas te ayuda a entender cómo fluye el estado en tus grafos de LangGraph de manera visual, lo cual es mucho más eficiente que llenar el código de `print()`.

Error 2: Ignorar las versiones de los Prompts

Una alucinación puede ser causada por un cambio de una sola palabra en un System Prompt.

- **Mejor Práctica:** Siempre loguea el `prompt_template_version` como un tag en tus trazas. LangSmith permite gestionar esto nativamente con su "Prompt Hub".

Error 3: Confiar ciegamente en los auto-evaluadores

Los evaluadores basados en LLM (como los de Phoenix) pueden tener sus propios sesgos. No reemplazan las pruebas unitarias tradicionales para lógica determinística ni la revisión humana para casos críticos. Úsalos como señales de alerta temprana, no como verdades absolutas.

7. Síntesis y Conexión

Hemos pasado de ver la IA como una "caja negra" a entenderla como un flujo de datos estructurado y medible.

- **Tracing** nos da la historia de lo que pasó.
- **Spans** nos dicen dónde falló o se ralentizó.
- **Evals** nos dicen si el resultado es confiable.

En la próxima unidad, combinaremos esta visibilidad con la infraestructura necesaria para escalar: aprenderemos cómo **FastAPI y Redis** permiten que este sistema trazable responda a miles de usuarios de forma asíncrona sin bloquear el hilo principal de ejecución. La observabilidad que configures hoy será tu salvavidas cuando el sistema crezca mañana.

Práctica

Título del ejercicio: Instrumentación y Análisis de Trazas con Phoenix

Propósito del ejercicio

Este ejercicio tiene como objetivo que pases de la teoría a la práctica mediante la instrumentación real de un sistema de agentes. Aprenderás a "ver" lo que ocurre dentro de tus grafos de razonamiento y a tomar decisiones basadas en datos sobre la latencia y la calidad de las respuestas.

Pasos sugeridos

- 1. Preparación del entorno:** * Lanza una instancia local de Arize Phoenix: `bash pip install arize-phoenix python -m phoenix.server.main serve` * Verifica que puedes acceder al dashboard en `http://localhost:6006`.
- 2. Instrumentación de tu código:** * Utiliza el código de tu sistema multi-agente desarrollado en el módulo anterior. * Configura el exportador de OpenInference para que apunte a tu servidor de Phoenix local. Asegúrate de instrumentar tanto el SDK del LLM como las llamadas a tu base de datos vectorial (Chroma o Pinecone).
- 3. Generación de Tráfico:** * Ejecuta al menos 5 consultas complejas que requieran que tus agentes utilicen herramientas, realicen búsquedas RAG y colaboren entre sí. * Provoca intencionalmente un fallo (por ejemplo, desconecta el acceso a una herramienta o pide algo fuera del alcance de los documentos) para observar cómo se registra el error en la traza.
- 4. Análisis en el Dashboard:** * Navega por las trazas en Phoenix. * Identifica el **Span de mayor latencia**: ¿Fue la generación del LLM o la recuperación de documentos? * Analiza el **Costo**: Revisa cuántos tokens consumió el agente en su proceso de razonamiento más largo.
- 5. Documentación de Hallazgos:** * Crea un documento (URL compartible) con el análisis solicitado en el entregable.

Criterios de Aceptación

- El dashboard de Phoenix debe mostrar trazas completas con jerarquía de spans (agente -> herramienta -> modelo).
- El análisis debe distinguir claramente entre latencia de red (llamada API) y latencia de procesamiento.

- Se debe identificar correctamente si una respuesta fue "grounded" (basada en el contexto) o si hubo evidencia de alucinación.

Errores comunes a evitar

- **No instrumentar la DB Vectorial:** Si no instrumentas el cliente de la base de datos, verás una "mancha ciega" en tu traza y no sabrás cuánto tiempo toma la recuperación.
- **Mezclar datos:** No limpiar el historial de Phoenix antes de una nueva sesión de pruebas, lo que puede confundir las métricas de latencia p95.

Entregable

Tipo: URL

Un enlace (URL) a un documento compartido (Google Doc, Notion, o PDF en la nube) que contenga: 1. Capturas de pantalla del dashboard de Arize Phoenix mostrando las trazas generadas. 2. Análisis detallado de una traza específica identificando latencia por componente. 3. Identificación de al menos un punto de mejora o fallo (alucinación, latencia alta o error de herramienta) y la propuesta de solución.

Observabilidad y Trazado: Arize Phoenix y LangSmith

Aprende a implementar sistemas de observabilidad y trazado avanzado utilizando Arize Phoenix y LangSmith para monitorear latencia, costos y calidad en aplicaciones de IA de producción.

Entregable

Tipo: URL

Un enlace (URL) a un documento compartido (Google Doc, Notion, o PDF en la nube) que contenga: 1. Capturas de pantalla del dashboard de Arize Phoenix mostrando las trazas generadas. 2. Análisis detallado de una traza específica identificando latencia por componente. 3. Identificación de al menos un punto de mejora o fallo (alucinación, latencia alta o error de herramienta) y la propuesta de solución.

Instrucciones

Título del ejercicio: Instrumentación y Análisis de Trazas con Phoenix

Propósito del ejercicio

Este ejercicio tiene como objetivo que pases de la teoría a la práctica mediante la instrumentación real de un sistema de agentes. Aprenderás a "ver" lo que ocurre dentro de tus grafos de razonamiento y a tomar decisiones basadas en datos sobre la latencia y la calidad de las respuestas.

Pasos sugeridos

- 1. Preparación del entorno:** * Lanza una instancia local de Arize Phoenix: `bash pip install arize-phoenix python -m phoenix.server.main serve` * Verifica que puedes acceder al dashboard en `http://localhost:6006`.
- 2. Instrumentación de tu código:** * Utiliza el código de tu sistema multi-agente desarrollado en el módulo anterior. * Configura el exportador de OpenInference para que apunte a tu servidor de Phoenix local. Asegúrate de instrumentar tanto el SDK del LLM como las llamadas a tu base de datos vectorial (Chroma o Pinecone).
- 3. Generación de Tráfico:** * Ejecuta al menos 5 consultas complejas que requieran que tus agentes utilicen herramientas, realicen búsquedas RAG y colaboren entre sí. * Provoca intencionalmente un fallo (por ejemplo, desconecta el acceso a una herramienta o pide algo fuera del alcance de los documentos) para observar cómo se registra el error en la traza.
- 4. Análisis en el Dashboard:** * Navega por las trazas en Phoenix. * Identifica el **Span de mayor latencia**: ¿Fue la generación del LLM o la recuperación de documentos? * Analiza el **Costo**: Revisa cuántos tokens consumió el agente en su proceso de razonamiento más largo.
- 5. Documentación de Hallazgos:** * Crea un documento (URL compartible) con el análisis solicitado en el entregable.

Criterios de Aceptación

- El dashboard de Phoenix debe mostrar trazas completas con jerarquía de spans (agente -> herramienta -> modelo).
- El análisis debe distinguir claramente entre latencia de red (llamada API) y latencia de procesamiento.
- Se debe identificar correctamente si una respuesta fue "grounded" (basada en el contexto) o si hubo evidencia de alucinación.

Errores comunes a evitar

- **No instrumentar la DB Vectorial:** Si no instrumentas el cliente de la base de datos, verás una "mancha ciega" en tu traza y no sabrás cuánto tiempo toma la recuperación.
- **Mezclar datos:** No limpiar el historial de Phoenix antes de una nueva sesión de pruebas, lo que puede confundir las métricas de latencia p95.

Objetivos de aprendizaje

- Diferenciar entre trazado, métricas y evaluaciones en el contexto de sistemas de IA.
- Implementar instrumentación basada en OpenInference para capturar trazas detalladas de modelos y herramientas.
- Identificar cuellos de botella de latencia y picos de costo analizando la jerarquía de spans en una traza.

- Configurar evaluadores automáticos para detectar alucinaciones y medir la fidelidad de las respuestas en sistemas RAG.
- Elegir la herramienta de observabilidad adecuada (Phoenix vs. LangSmith) según los requerimientos de privacidad y el stack tecnológico.

Rúbrica de Evaluación: Instrumentación y Análisis de Trazas con Phoenix

Esta rúbrica evalúa la capacidad del estudiante para implementar observabilidad en sistemas de IA multi-agente, analizar métricas de latencia y costos, e identificar fallos de razonamiento o integración mediante el uso de Arize Phoenix.

Criterio	Descripción	Peso
Instrumentación y Configuración Técnica	Capacidad para integrar el SDK de OpenInference y capturar trazas jerárquicas de todos los componentes del sistema (LLM, herramientas y base de datos vectorial).	35%
Análisis de Latencia y Desempeño	Evaluación crítica de los tiempos de respuesta, distinguiendo entre latencia de red, tiempo de inferencia del LLM y tiempos de recuperación (retrieval).	25%
Detección de Errores y Calidad de Respuesta	Identificación proactiva de fallos (alucinaciones, errores de herramientas o falta de fundamentación) y propuesta de soluciones técnicas.	25%
Análisis de Costos y Eficiencia de Tokens	Revisión del consumo de tokens y análisis de la eficiencia del razonamiento del agente.	15%

Total: 100 pts | Aprobación: 65 pts

• Código

Escalabilidad Asíncrona con FastAPI y Redis

Aprende a desacoplar la ejecución de tus agentes de IA de la respuesta de la API para crear sistemas que escalen horizontalmente.

Escalabilidad Asíncrona con FastAPI y Redis

El problema de la latencia en IA

Cuando construimos sistemas multi-agente complejos, nos enfrentamos a un reto de ingeniería: **el tiempo de respuesta**. Un flujo de agentes que consulta herramientas, navega por internet y procesa datos en una base vectorial puede tardar 30, 60 o incluso 120 segundos.

Mantener una conexión HTTP abierta durante tanto tiempo es peligroso: los firewalls cortan la conexión por timeout, el usuario piensa que el sistema "se colgó" y tu servidor de API se queda sin recursos esperando a que el LLM termine.

La Solución: Arquitectura de Procesamiento Desacoplado

La técnica estándar en la industria para manejar tareas pesadas es el **procesamiento asíncrono con colas de mensajes**. En este esquema:

- 1. FastAPI (El Recepcionista):** Recibe la petición, valida los datos, le da al usuario un "Ticket ID" (Job ID) y guarda la tarea en un buzón. Responde en milisegundos.
- 2. Redis (El Buzón):** Actúa como *Message Broker*. Almacena las tareas pendientes de forma segura hasta que alguien las recoja.
- 3. Worker (El Especialista):** Un proceso independiente (que puede escalar a múltiples instancias) que saca tareas de la cola, ejecuta el grafo de LangGraph y guarda el resultado final en Redis.

¿Por qué Redis?

Redis no es solo un caché. Gracias a sus estructuras de datos como **Lists** (para colas simples) o **Streams** (para arquitecturas de eventos), es el compañero ideal para FastAPI. Su velocidad en memoria permite coordinar miles de tareas con latencia casi nula.

Patrón: Polling (Sondeo)

En este nivel intermedio, implementaremos el patrón de **Polling**: el cliente pregunta periódicamente al endpoint `/status/{job_id}` si la tarea ya terminó. Es simple, robusto y fácil de implementar comparado con WebSockets en infraestructuras de nube complejas.

Errores Comunes

- **Ejecutar IA en el endpoint directamente:** Bloqueas el hilo de ejecución y tu API deja de responder a otros usuarios.
- **No persistir el estado inicial:** Si el worker es más rápido que la escritura del estado "pending", el cliente podría consultar el estado y recibir un error 404 porque el registro aún no existe. Siempre registra el estado *antes* de encolar la tarea.

Ejercicio de código

Código inicial

```
import asyncio
import uuid
```

```

import json
from fastapi import FastAPI, HTTPException
from redis import asyncio as aioredis
from pydantic import BaseModel

# Configuración de Redis
REDIS_URL = "redis://localhost:6379"
QUEUE_NAME = "tasks_queue"
STATUS_PREFIX = "status:"

app = FastAPI()
redis_client = aioredis.from_url(REDIS_URL, decode_responses=True)

class TaskRequest(BaseModel):
    query: str

@app.post("/process")
async def create_task(request: TaskRequest):
    """
    TODO:
    1. Generar un job_id único.
    2. Guardar en Redis el estado inicial (status: pending).
    3. Poner el job_id en la lista (cola) de Redis para que el worker lo vea.
    4. Retornar el job_id al cliente inmediatamente.
    """
    pass

@app.get("/status/{job_id}")
async def get_status(job_id: str):
    """
    TODO: Consultar el estado y resultado del job_id en Redis.
    """
    pass

async def task_worker():
    """
    TODO: Implementar un bucle infinito que:
    1. Extraiga job_ids de la cola usando blpop.
    2. Simule el procesamiento de LangGraph (asyncio.sleep).
    3. Actualice el estado en Redis a 'completed' con el resultado final.
    """
    print("Worker iniciado...")
    while True:
        # Implementar lógica de consumo aquí
        await asyncio.sleep(1)

```

```

if __name__ == "__main__":
    import uvicorn
    # Nota: Creamos la tarea del worker para que corra junto con la API en este ejemplo
    loop = asyncio.get_event_loop()
    loop.create_task(task_worker())
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Código solución

```

import asyncio
import uuid
import json
from fastapi import FastAPI, HTTPException
from redis import asyncio as aioredis
from pydantic import BaseModel

# --- CONFIGURACIÓN ---
REDIS_URL = "redis://localhost:6376"
QUEUE_NAME = "ai_tasks_queue"
STATUS_PREFIX = "task_status:"

app = FastAPI(title="Async AI Architect API")
redis_client = aioredis.from_url(REDIS_URL, decode_responses=True)

class TaskRequest(BaseModel):
    query: str

class TaskResponse(BaseModel):
    job_id: str
    status: str

# --- ENDPOINTS API ---

@app.post("/process", response_model=TaskResponse)
async def create_task(request: TaskRequest):
    """
    Recibe una solicitud, genera un job_id y encola la tarea en Redis.
    """
    job_id = str(uuid.uuid4())

    # Definir el estado inicial
    task_data = {

```



```

        "job_id": job_id,
        "query": request.query,
        "status": "pending",
        "result": None
    }

    # 1. Guardar estado inicial en Redis
    await redis_client.set(f"{STATUS_PREFIX}{job_id}", json.dumps(task_data))

    # 2. Encolar el job_id para que el worker lo procese
    await redis_client.rpush(Queue_NAME, job_id)

    return TaskResponse(job_id=job_id, status="pending")

@app.get("/status/{job_id}")
async def get_status(job_id: str):
    """
    Consulta el estado actual de una tarea en Redis.
    """
    data = await redis_client.get(f"{STATUS_PREFIX}{job_id}")
    if not data:
        raise HTTPException(status_code=404, detail="Job not found")

    return json.loads(data)

# --- LÓGICA DEL WORKER (Simulada para este archivo único) ---
# En una app real, esto correría en un comando de terminal aparte: `python worker.py`

async def main_worker():
    """
    Loop infinito que procesa tareas de la cola de Redis.
    """
    print("Worker iniciado y escuchando...")
    while True:
        # BLPOP bloquea de forma eficiente hasta que haya un elemento
        result = await redis_client.blpop(Queue_NAME, timeout=0)
        if result:
            _, job_id = result
            print(f"Procesando Job: {job_id}")

            # 1. Marcar como 'en progreso'
            raw_data = await redis_client.get(f"{STATUS_PREFIX}{job_id}")
            task_data = json.loads(raw_data)
            task_data["status"] = "processing"
            await redis_client.set(f"{STATUS_PREFIX}{job_id}", json.dumps(task_data))

```

```
# --- SIMULACIÓN DE LANGGRAPH / AGENTES ---
# Aquí llamarías a: await my_langgraph_agent.ainvoke(...)
await asyncio.sleep(5)
simulated_ai_response = f"Respuesta para: {task_data['query']} - Procesada por
Agente X"
```

```
# 2. Marcar como completado y guardar resultado
task_data["status"] = "completed"
task_data["result"] = simulated_ai_response
await redis_client.set(f"{STATUS_PREFIX}{job_id}", json.dumps(task_data))
print(f"Job {job_id} finalizado.")
```

Nota: Para ejecutar API y Worker juntos en este ejemplo:

```
if __name__ == "__main__":
    import uvicorn
    # En producción, usarías procesos separados. Aquí lanzamos el worker como tarea de fondo.
    loop = asyncio.get_event_loop()
    loop.create_task(main_worker())
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Pistas

- Para conectar FastAPI con Redis, utiliza `redis.asyncio`. Recuerda que en una arquitectura desacoplada, el API solo 'publica' el trabajo, no lo ejecuta. El ID del trabajo será tu brújula para que el cliente pueda preguntar '¿Ya terminaste?' más tarde.
- Al implementar el Worker, asegúrate de que sea un proceso independiente del API. Este debe escuchar la cola de Redis continuamente (por ejemplo, usando `blpop` para bloqueo eficiente) y actualizar el estado en Redis (un Hash es ideal) para que el API pueda leer el progreso.
- Para integrar LangGraph, el Worker debe inicializar el grafo y usar `graph.ainvoke(inputs)`. No olvides manejar excepciones dentro del worker; si algo falla en la IA, el estado del trabajo en Redis debe reflejar 'failed' para no dejar al cliente esperando por siempre.

Rúbrica: Escalabilidad Asíncrona con FastAPI y Redis

Evaluación del diseño e implementación de un sistema de procesamiento desacoplado para tareas de IA de larga duración, enfocado en robustez, gestión de estado y eficiencia asíncrona.

Criterio	Descripción	Peso
Arquitectura de Desacoplamiento (FastAPI + Redis)	Evalúa la correcta separación de responsabilidades entre el receptor de la petición y el procesador de la tarea.	35%
Gestión de Estados y Polling	Evalúa cómo se maneja el ciclo de vida de la tarea (pendiente, procesando, completado, error) y su consulta.	25%
Lógica del Worker y Concurrencia	Evalúa la implementación del proceso independiente que consume las tareas de la cola.	25%
Calidad del Código y Tipado (Pydantic/Python 3.12)	Uso de estándares modernos de Python y validación de datos.	15%

Total: 100 pts | Aprobación: 65 pts

• Verdadero o falso

Human-in-the-Loop: Supervisión de Tareas Críticas

Evaluación conceptual sobre la integración de supervisión humana en sistemas de IA para garantizar la fiabilidad en entornos de alta criticidad.

Human-in-the-Loop: Supervisión de Tareas Críticas

Bienvenido a la evaluación de la Unidad 3. En esta sección nos enfocaremos en uno de los pilares de la IA robusta en producción: la **intervención humana estratégica**.

Hasta ahora, hemos aprendido a construir sistemas asíncronos potentes y a observar su comportamiento con herramientas como LangSmith. Sin embargo, en arquitecturas de nivel industrial, delegar el 100% de la autonomía a un modelo (especialmente en sectores como finanzas, salud o legal) suele ser un riesgo inaceptable. Aquí es donde entra el patrón **Human-in-the-Loop (HITL)**.

¿Por qué esta evaluación es clave?

No se trata simplemente de "añadir un humano al proceso", sino de diseñar **arquitecturas de software asíncronas** que permitan pausar, persistir el estado, solicitar intervención basada en umbrales de incertidumbre y reanudar la ejecución sin perder el contexto.

A continuación, pondremos a prueba tu comprensión sobre:

- Mecanismos de escalado (umbrales de confianza vs. reglas de negocio).
- Diseño de flujos agentic con estados persistentes (checkpoints).
- La diferencia crítica entre supervisión operativa y entrenamiento de modelos.
- Buenas prácticas de ingeniería para evitar cuellos de botella humanos.

Recuerda que cada pregunta incluye una explicación detallada diseñada para reforzar tus criterios como **AI Architect**. ¡Comencemos!

Verdadero o falso

Aprobación: 70 · Intentos máximos: 3

1. En un sistema Human-in-the-loop (HITL) correctamente diseñado para escalabilidad, el revisor humano debe validar el 100% de las respuestas generadas por el LLM antes de que lleguen al usuario final.

☐ Verdadero

☒ Falso

Explicación: El enfoque Human-in-the-loop (HITL) busca eficiencia selectiva. Si un humano revisara cada salida, eliminaríamos las ventajas de escalabilidad y costo de la IA. Un sistema bien diseñado utiliza umbrales de confianza o triggers de riesgo para desviar solo los casos ambiguos o críticos al experto. Recuerda: HITL no es 'manual', es 'automatización asistida'.

2. La persistencia de estados (checkpoints) es un requisito técnico esencial para implementar flujos de aprobación humana en sistemas agénticos asíncronos.

☒ Verdadero

☐ Falso

Explicación: En sistemas complejos como los construidos con LangGraph, los 'checkpoints' permiten guardar el hilo de la conversación y el estado del agente en una base de datos. Esto es fundamental para HITL porque permite que el sistema se 'detenga', espere horas o días a la aprobación humana, y luego se reanude exactamente donde se quedó. Sin persistencia, no hay verdadera supervisión asíncrona.

3. Para garantizar la mejora continua, todas las correcciones realizadas por humanos en producción deben integrarse automáticamente en el siguiente ciclo de entrenamiento del modelo sin validación adicional.

☐ Verdadero

☒ Falso

Explicación: Automatizar el reentrenamiento con cualquier corrección es peligroso. Los humanos también cometen errores, son inconsistentes o pueden tener sesgos. Un pipeline robusto de MLOps requiere que el feedback humano pase por un proceso de validación y limpieza de datos antes de ser utilizado para el ajuste fino (fine-tuning) o la actualización de reglas del sistema.

4. Los mecanismos de escalado (handoff) en un sistema de IA pueden activarse tanto por baja confianza del modelo como por reglas de negocio predefinidas basadas en el riesgo de la tarea.

☒ Verdadero

☐ Falso

Explicación: El escalado por 'confianza' usa la probabilidad del modelo para decidir si necesita ayuda. Sin embargo, hay casos (como transacciones de alto valor o temas legales) donde, independientemente de qué tan seguro esté el modelo, una regla de negocio dicta que un humano debe intervenir. Un arquitecto de IA combina ambos criterios.

5. El patrón 'Human-on-the-loop' (HOTL) requiere que la IA se detenga obligatoriamente en cada paso para recibir una confirmación de ejecución de un operario.

☐ Verdadero

☒ Falso

Explicación: A diferencia del HITL (donde el humano es parte del flujo de decisión), el modelo HOTL permite que la IA actúe de forma autónoma, pero con un humano 'vigilando' el proceso de forma externa para intervenir solo si detecta una anomalía o fallo general. Es un patrón común en monitoreo de flotas de robots o sistemas de trading.

6. La integración de HITL permite no solo corregir errores puntuales, sino también cumplir con requisitos de auditoría y 'compliance' en sectores regulados como el financiero.

☒ Verdadero

☐ Falso

Explicación: En tareas críticas, el humano no solo corrige, sino que su intervención genera trazabilidad detallada. Registrar quién aprobó qué, bajo qué contexto y por qué motivo, es un requisito legal y técnico en industrias reguladas. HITL proporciona la evidencia de que el sistema opera bajo supervisión responsable.

Pre-entrega 7: API de producción y monitoreo activo

Instrucciones para la construcción de una API de producción que integra sistemas multi-agente con observabilidad y gestión de colas.

Pre-entrega 7: API de producción y monitoreo activo

Qué construir

Debes entregar el código fuente de una **API REST (FastAPI)** que exponga tu sistema multi-agente desarrollado en el Módulo 6, con las siguientes capacidades nuevas:

1. **Endpoints Asíncronos:** La API no debe bloquearse. Debe recibir la tarea, encolarla y devolver un ID.
2. **Gestión de Estado con Redis:** Uso de Redis para persistir el estado de los trabajos y los checkpoints de LangGraph.
3. **Capa de Observabilidad:** Integración activa de instrumentación para enviar trazas de cada ejecución a una plataforma de monitoreo.
4. **Flujo Human-in-the-loop (HITL):** Implementación de una pausa obligatoria para aprobación humana en tareas que el sistema identifique como "críticas" (ej. ejecución de una herramienta con efectos secundarios o costo muy alto).

Pasos Sugeridos

1. **Refactorización del Grafo:** Toma tu orquestador multi-agente del Módulo 6 y asegúrate de que use un `RedisSaver` o similar para persistir el estado. Define un punto de interrupción en una arista o nodo crítico.
2. **Wrappers de Observabilidad:** Configura las variables de entorno para LangSmith o inicializa el colector de Arize Phoenix mediante OpenInference. Asegúrate de decorar tus funciones de llamada a LLM.
3. **Arquitectura de la API:** Crea la aplicación FastAPI. Implementa el patrón de "Worker" (puede ser un thread asíncrono simple en esta etapa o un proceso separado con Celery/Arq si buscas excelencia).
4. **Pruebas de Carga Sencillas:** Envía 5 peticiones concurrentes y verifica en tu dashboard de monitoreo cómo se comportan las trazas y la latencia.

Errores comunes a evitar

- **Bloquear el Event Loop:** Evita hacer llamadas `sync` a bases de datos o LLMs dentro de los endpoints de FastAPI sin usar `run_in_threadpool` o `async` nativo. Esto matará el rendimiento.
 - **Ignorar el manejo de errores en Background Tasks:** Si el agente falla en segundo plano, el sistema debe capturar la excepción y actualizar el estado en Redis a `FAILED`, de lo contrario, el cliente se quedará esperando en un loop infinito de polling.
-

Práctica

1. Crea una estructura de proyecto Python 3.12+.
2. Implementa tu sistema multi-agente del Módulo 6 dentro de un orquestador que use Redis como almacenamiento de estado.
3. Desarrolla la API con FastAPI utilizando los endpoints asíncronos mencionados en el briefing.
4. Configura Arize Phoenix o LangSmith para capturar trazas de cada nodo del agente.
5. Implementa un nodo de 'Aprobación Humana' que detenga la ejecución del grafo hasta recibir un input externo.
6. Sube el código a un repositorio y documenta en el README cómo inicializar Redis y la API.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código de la API, archivo de requerimientos, configuración de Docker Compose (opcional pero recomendado) y capturas de pantalla del dashboard de observabilidad con trazas activas.

Pre-entrega: API de Producción y Monitoreo Activo

Instrucciones para la construcción de una API de producción que integra sistemas multi-agente con observabilidad y gestión de colas.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código de la API, archivo de requerimientos, configuración de Docker Compose (opcional pero recomendado) y capturas de pantalla del dashboard de observabilidad con trazas activas.

Instrucciones

1. Crea una estructura de proyecto Python 3.12+.
2. Implementa tu sistema multi-agente del Módulo 6 dentro de un orquestador que use Redis como almacenamiento de estado.
3. Desarrolla la API con FastAPI utilizando los endpoints asíncronos mencionados en el briefing.
4. Configura Arize Phoenix o LangSmith para capturar trazas de cada nodo del agente.
5. Implementa un nodo de 'Aprobación Humana' que detenga la ejecución del grafo hasta recibir un input externo.
6. Sube el código a un repositorio y documenta en el README cómo inicializar Redis y la API.

Objetivos de aprendizaje

- Desplegar una API asíncrona utilizando FastAPI y Redis para gestionar tareas de IA de larga duración.
- Configurar sistemas de observabilidad (Arize Phoenix o LangSmith) para monitorear costos y latencia.
- Implementar flujos de interacción humana (Human-in-the-loop) dentro de grafos de agentes asíncronos.
- Diseñar mecanismos de persistencia de estado para orquestadores multi-agente complejos.

Rúbrica de Evaluación: API de Producción y Monitoreo Activo

Evaluación de la capacidad para transicionar un sistema multi-agente a un entorno productivo, integrando persistencia de estado con Redis, endpoints asíncronos en FastAPI y observabilidad detallada.

Criterio	Descripción	Peso
Orquestación Asíncrona y API	Evaluación de la implementación de FastAPI y la gestión de endpoints asíncronos para el flujo de agentes.	30%
Persistencia de Estado con Redis	Capacidad para configurar e integrar Redis como almacenamiento de checkpoints para la persistencia de hilos de conversación.	25%
Observabilidad y Trazado	Implementación de herramientas de monitoreo (Arize Phoenix o LangSmith) para capturar el ciclo de vida de los mensajes.	20%
Intervención Humana (Human-in-the-loop)	Implementación de un nodo de aprobación que detenga el flujo hasta recibir una señal externa.	15%
Documentación y Estructura Profesional	Calidad del repositorio, README, archivo de requerimientos y despliegue inicial.	10%

Total: 100 pts | Aprobación: 65 pts

● Podcast

Recapitulación: Producción y robustez: observabilidad, costos y despliegue

Un episodio de recapitulación que conecta la observabilidad, la escalabilidad asíncrona y la supervisión humana como pilares de un sistema de IA robusto y listo para producción.

Recapitulación del Módulo: Producción y Robustez

¡Bienvenido al cierre del módulo! En este episodio, haremos un recorrido integral por las estrategias y herramientas que transforman un prototipo de IA en un sistema de grado de producción.

¿Qué aprenderás en este episodio?

Escuchando esta conversación, consolidarás cómo se conectan los pilares de la robustez:

- **Observabilidad:** Cómo herramientas como Arize Phoenix y LangSmith nos permiten "ver" dentro de la caja negra de los LLM.
- **Escalabilidad:** Por qué la asincronía con FastAPI y Redis es vital para no bloquear la experiencia del usuario.
- **Intervención Humana:** El rol crítico del *Human-in-the-Loop* en tareas donde la IA no puede (ni debe) decidir sola.
- **Operatividad:** La importancia de monitorear costos y latencias para un despliegue sostenible.

Puntos clave

1. La importancia de la **trazabilidad** para depurar flujos complejos.
 2. El desacoplamiento de tareas pesadas mediante **colas de mensajes**.
 3. La implementación de puntos de control para la **supervisión humana**.
 4. La transición de "funciona en mi computadora" a "es robusto en la nube".
-

MÓDULO 8

Capstone: entrega final

Consolidar todas las capas del sistema inteligente en un entregable final profesional y listo para producción.

• Entrega final

Entrega final: sistema Intelligence de grado de producción

Briefing detallado para la entrega final del programa, integrando todos los componentes del sistema de IA escalable.

Entrega final: Sistema Intelligence de grado de producción

El Camino Recorrido

Para llegar aquí, has superado hitos técnicos fundamentales que ahora se ensamblan:

- 1. Cimientos Asíncronos & Abstracción:** Implementaste clientes robustos que manejan streaming y latencia.
- 2. RAG Avanzado:** Diseñaste estrategias de recuperación híbrida y gestión de metadatos para reducir alucinaciones.
- 3. Inteligencia Colectiva:** Creaste una arquitectura de agentes donde un **Supervisor** delega tareas de forma autónoma.
- 4. Robustez & Despliegue:** Preparaste el entorno para producción con observabilidad (Arize Phoenix) y escalabilidad (Redis/FastAPI).

Descripción del Entregable

Debes entregar un **repositorio de GitHub** que contenga un sistema de IA completo ("The Intelligence System"). Este sistema debe permitir a un usuario final (o proceso automatizado) resolver una tarea compleja que requiera:

- Acceso a conocimiento específico (vía RAG).
- Razonamiento multi-paso (vía Agentes).
- Interfaz de API profesional y documentada.
- Trazabilidad completa de lo que ocurre "bajo el capó".

Errores Comunes a Evitar

- **Hard-coding:** Evita poner IDs de índices de Pinecone o modelos directamente en el código; usa variables de entorno.

- **Sincronicidad:** No mezcles llamadas síncronas (`requests`) dentro de funciones `async def` ; usa `httpx` o los SDKs asíncronos oficiales.
- **Falta de Trazas:** Entregar un sistema que "funciona" pero cuya ejecución es una caja negra (sin integración de logs o traces).

Cierre del Programa

Este sistema es tu carta de presentación como **AI Architect**. Has construido una pieza de ingeniería capaz de razonar, recordar y escalar. Tras esta entrega, estarás listo para enfrentar retos de implementación de IA en entornos corporativos reales. **¡Es hora de consolidar tu visión! Estamos ansiosos por ver tu arquitectura en funcionamiento.**

Práctica

1. **Integración de Código:** Mueve la lógica de tus pre-entregas de los Módulos 6 y 7 a una estructura de proyecto unificada.
2. **Refactorización:** Asegúrate de que toda la comunicación con LLMs y bases de datos sea asíncrona (`async/await`).
3. **Persistencia:** Implementa un `Checkpointeer` en LangGraph para que las conversaciones no se pierdan al reiniciar el servidor.
4. **Validación:** Usa Pydantic para validar todas las entradas y salidas de la API y de las herramientas de los agentes.
5. **Observabilidad:** Realiza al menos 5 pruebas del sistema y asegúrate de que las trazas se visualicen correctamente en Arize Phoenix o LangSmith.
6. **README:** Escribe la documentación incluyendo un diagrama de flujo del grafo de agentes.
7. **Despliegue Local:** Asegúrate de que el proyecto pueda levantarse con una sola instrucción (ej. Docker Compose o un script `run.sh`).

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código del sistema completo (Python 3.12+), archivos de configuración, documentación técnica y trazas de observabilidad configuradas.

Entrega Final: Sistema Intelligence de Grado de Producción

Briefing detallado para la entrega final del programa, integrando todos los componentes del sistema de IA escalable.

Entregable

Tipo: REPOSITORY

Repositorio de GitHub con el código del sistema completo (Python 3.12+), archivos de configuración, documentación técnica y trazas de observabilidad configuradas.

Instrucciones

- 1. Integración de Código:** Mueve la lógica de tus pre-entregas de los Módulos 6 y 7 a una estructura de proyecto unificada.
- 2. Refactorización:** Asegúrate de que toda la comunicación con LLMs y bases de datos sea asíncrona (`async/await`).
- 3. Persistencia:** Implementa un `Checkpointter` en LangGraph para que las conversaciones no se pierdan al reiniciar el servidor.
- 4. Validación:** Usa Pydantic para validar todas las entradas y salidas de la API y de las herramientas de los agentes.
- 5. Observabilidad:** Realiza al menos 5 pruebas del sistema y asegúrate de que las trazas se visualicen correctamente en Arize Phoenix o LangSmith.
- 6. README:** Escribe la documentación incluyendo un diagrama de flujo del grafo de agentes.
- 7. Despliegue Local:** Asegúrate de que el proyecto pueda levantarse con una sola instrucción (ej. Docker Compose o un script `run.sh`).

Objetivos de aprendizaje

- El estudiante será capaz de consolidar múltiples componentes de un sistema de IA (RAG, Agentes, API) en un único repositorio profesional.
- El estudiante será capaz de aplicar principios de arquitectura limpia y desarrollo asíncrono para asegurar la producción del sistema.
- El estudiante será capaz de configurar y demostrar la observabilidad del sistema mediante trazas y monitoreo de costos.
- El estudiante será capaz de documentar decisiones técnicas de arquitectura para una audiencia de ingeniería senior.

Rúbrica de Evaluación Final: Sistema Intelligence de Grado de Producción

Esta rúbrica evalúa la capacidad del estudiante para integrar componentes de IA, bases de datos vectoriales, orquestación de agentes y observabilidad en un sistema unificado, asíncrono y listo para producción, reflejando las competencias de un AI Architect.

Criterio	Descripción	Peso
Arquitectura y Coherencia del Sistema	Evalúa la integración de las capas de los módulos anteriores (LLMs, RAG, Agentes) en un flujo unificado y lógico.	30%
Robustez Técnica: Asincronía y Validación	Uso de patrones async/await en Python 3.12 y validación de datos con Pydantic.	25%
Persistencia y Observabilidad	Implementación de Checkpointers en LangGraph y trazado de ejecución con herramientas externas.	20%
Documentación y Despliegue	Claridad del README, diagrama de arquitectura y facilidad de ejecución (Docker/Scripts).	15%
Configuración de Entorno y Estándares de Código	Uso de Python 3.12+, gestión de dependencias y variables de entorno.	10%

Total: 100 pts | Aprobación: 65 pts

Material complementario

Palabras clave

- **Asyncio:** Es una librería de Python diseñada para escribir código que maneja múltiples tareas al mismo tiempo sin bloquear la ejecución del programa. En sistemas de IA, permite que tu aplicación realice otras operaciones mientras espera la respuesta de un servidor externo de lenguaje, mejorando drásticamente la fluidez y velocidad del sistema.
- **RAG (Retrieval-Augmented Generation):** Es una arquitectura que combina la capacidad de respuesta de los modelos de IA con el acceso a datos externos específicos y actualizados. Funciona buscando información relevante en una base de datos propia antes de generar una respuesta, lo que reduce errores y asegura que la IA no invente datos inexistentes.
- **LCEL (LangChain Expression Language):** Es una sintaxis declarativa que facilita la unión de diferentes componentes de software como si fueran piezas de un rompecabezas para crear flujos de datos. Permite pasar información de un proceso a otro de forma automática, manejando errores y tareas paralelas de manera simplificada dentro del ecosistema de LangChain.
- **Vector Database (Base de Datos Vectorial):** Es un tipo especializado de base de datos que almacena información en forma de coordenadas numéricas llamadas vectores. Esto permite a la IA buscar datos basándose en el significado o concepto de las palabras en lugar de buscar coincidencias exactas, permitiendo encontrar documentos similares semánticamente.
- **LangGraph:** Es una herramienta diseñada para construir flujos de trabajo cíclicos donde la IA puede volver sobre sus propios pasos para corregir errores o analizar mejor un problema. A diferencia de las cadenas

lineales, permite crear agentes que razonan en bucle hasta que consideran que han resuelto la tarea asignada.

- **Tool Calling (Llamado a Herramientas):** Es la capacidad que tiene un modelo de lenguaje para decidir por sí mismo cuándo necesita utilizar una función externa, como una calculadora o un buscador web. La IA genera los parámetros necesarios para ejecutar la acción y luego interpreta los resultados obtenidos para continuar con su respuesta.
- **Observabilidad:** Es el proceso de monitorear y rastrear cada paso interno que da un sistema de IA desde que recibe una petición hasta que entrega un resultado. Utilizando herramientas como LangSmith, los desarrolladores pueden ver dónde ocurrió un error, cuánto tiempo tardó cada proceso y cuánto dinero costó cada interacción.
- **Agentic Workflow (Flujo Agéntico):** Se refiere a un diseño de software donde la IA actúa como un agente con autonomía para tomar decisiones sobre qué pasos seguir a continuación. En lugar de seguir una ruta fija, el sistema evalúa la situación y elige dinámicamente qué herramientas o agentes auxiliares necesita para completar un objetivo.
- **Chunking:** Es la técnica de dividir documentos extensos en fragmentos más pequeños y manejables antes de procesarlos. Esto es vital para que la IA no se pierda en textos demasiado largos y pueda encontrar con precisión el párrafo exacto que contiene la respuesta a una pregunta específica.
- **Re-ranking:** Es una capa de optimización que revisa los resultados obtenidos de una búsqueda inicial y los vuelve a clasificar según su relevancia real para el usuario. Ayuda a filtrar ruidos y asegura que los mejores fragmentos de información sean los que finalmente se entreguen al modelo de IA para generar la respuesta.

Análisis profundo

El Arte del Manejo de Estados en Sistemas de IA Asíncronos

En el desarrollo de software tradicional, el estado suele ser lineal; sin embargo, en la arquitectura de un 'AI Architect', el manejo asíncrono del estado es el sistema circulatorio que permite la escalabilidad. ¿Por qué es crucial? Porque los LLMs son servicios externos con latencias impredecibles que pueden bloquear tu aplicación si no se gestionan con Python 3.12 y Asyncio. Si un desarrollador ignora el paradigma no bloqueante, el sistema colapsará bajo carga mínima, creando cuellos de botella donde los usuarios esperan segundos por una respuesta que podría haberse gestionado en paralelo.

Imagina que estás coordinando una cocina de alta gama. Si el chef principal (el hilo de ejecución) espera a que el agua hierva sin hacer nada más, la cocina es ineficiente. `Asyncio` permite que el chef prepare la salsa mientras el agua alcanza la temperatura. En el contexto de IA, esto significa que mientras esperamos los tokens de OpenAI, nuestro sistema puede estar consultando un metadato en ChromaDB o validando un esquema intermedio. Empresas como Uber o Netflix utilizan patrones de asincronía similares para manejar millones de peticiones concurrentes sin multiplicar linealmente sus recursos de hardware.

Un error común es confundir 'paralelismo' con 'asincronía'. Muchos estudiantes intentan usar `threading` para LLMs, lo cual es ineficiente debido al Global Interpreter Lock (GIL) de Python. La clave aquí es el Event Loop de la versión 3.12, que optimiza la conmutación de tareas I/O-bound. Al dominar esto, transformas un script de

juguete en un motor de producción capaz de sostener flujos complejos de streaming de datos, reduciendo el 'Time to First Token' percibido por el usuario final, un KPI vital en la industria actual.

La Dualidad del RAG: De la Geometría Vectorial al Re-Ranking Semántico

Muchos desarrolladores creen que implementar un sistema Retrieval-Augmented Generation (RAG) consiste simplemente en subir PDFs a una base de datos vectorial. Esta simplificación es peligrosa: produce sistemas que sufren de 'alucinaciones informadas', donde el modelo recupera datos irrelevantes pero con alta similitud matemática. El problema real que resolvemos no es de almacenamiento, sino de relevancia semántica. ¿Cómo distinguimos un documento que 'se parece' de uno que realmente 'responde'?

El proceso comienza con la geometría del lenguaje: los embeddings. Estos transforman palabras en coordenadas en un espacio de n-dimensiones. Sin embargo, la similitud de coseno a veces falla porque el lenguaje es ambiguo. Aquí es donde entra el 'Re-ranking' y la recuperación híbrida. Mientras que herramientas como Pinecone gestionan la escala masiva de vectores, un re-ranker (como los modelos de Cohere) actúa como un filtro secundario de alta precisión que reordena los resultados basándose en el contexto real de la pregunta.

Considera una aplicación de soporte legal: un error en el chunking (fragmentación) de un contrato podría separar una cláusula de su excepción, llevando a la IA a dar un consejo erróneo. Los arquitectos senior no solo eligen una Vector DB; diseñan estrategias de 'Parent Document Retrieval' o 'Small-to-Big chunks' para asegurar que el contexto nunca se pierda. En el programa, entender esta dualidad es la diferencia entre un sistema que 'lee' y uno que realmente 'comprende' la base documental de una empresa, evitando que el ruido nuble la señal crítica.

Grafos de Agentes: Superando la Rigidez de las Cadenas Secuenciales

En los módulos iniciales usamos LangChain para crear cadenas, pero en el mundo real, los problemas no son lineales. El concepto de 'Agentes con LangGraph' aborda el desafío de la autonomía y el razonamiento cíclico. El problema con las cadenas tradicionales es que si el paso 2 falla, toda la secuencia muere. Los grafos permiten ciclos: si un agente detecta que su propia respuesta es insuficiente o que la herramienta devolvió un error, puede volver atrás, corregir y reintentar.

Piensa en este sistema como una mesa redonda de especialistas. No es una línea de ensamblaje, sino un equipo dinámico. En LangGraph, definimos 'Nodos' (los cerebros o herramientas) y 'Aristas' con lógica condicional (el camino que decide el flujo). El patrón 'Supervisor' que exploramos es el equivalente a un Project Manager en software: un agente cuya única función es decidir a qué otro agente especialista (QA, Codificador, Investigador) delegar la siguiente tarea.

Una confusión habitual es pensar que más agentes siempre es mejor. La realidad es que la 'entropía de agentes' puede disparar los costos y la latencia. La ingeniería de sistemas de IA busca el equilibrio: usar estados compartidos bien definidos para que cada agente sepa exactamente qué hizo el anterior. Al implementar checkpoints y persistencia, permitimos que procesos largos (como una auditoría de código que dura 10

minutos) puedan pausarse y reanudarse sin perder el progreso, una característica fundamental para cualquier aplicación de nivel empresarial que aspire a la resiliencia.

Preguntas frecuentes

P: ¿Por qué usamos Asyncio y Python 3.12 desde el inicio si los scripts de IA suelen ser lineales?

R: Muchos desarrolladores asumen que la IA es una tarea puramente computacional que ocurre en su máquina, pero en realidad, las aplicaciones de producción pasan el 90% del tiempo esperando respuestas de red de APIs externas (como OpenAI o Anthropic). Usar `asyncio` nos permite manejar cientos de peticiones simultáneas sin bloquear el hilo principal. Python 3.12 introduce mejoras significativas en la gestión de tareas asíncronas y tipos estáticos que son vitales para la robustez del sistema. Imagina que tu sistema es un restaurante: sin asincronía, el camarero (tu código) se quedaría parado frente a la cocina esperando a que salga el plato, sin poder atender a nadie más. Con la arquitectura que aprenderás en el Módulo 1, el camarero toma el pedido y sigue atendiendo mesas mientras la comida se prepara, maximizando la eficiencia de tu infraestructura.

P: ¿Cuál es la diferencia real entre LangChain (Cadenas) y LangGraph (Agentes) y cuándo debo saltar de uno a otro?

R: Esta es una confusión común. LangChain (LCEL) está diseñado para flujos acíclicos y deterministas: Paso A -> Paso B -> Paso C. Es excelente para pipelines de extracción de datos o RAG simple. Sin embargo, los problemas complejos del mundo real requieren 'bucles' de razonamiento donde el sistema decide si la respuesta es suficiente o si debe reintentar una tarea. Ahí entra LangGraph. Muchos estudiantes intentan forzar LangChain para que se comporte como un agente usando lógica compleja de 'if/else', pero eso se vuelve inmanejable. La regla de oro que aplicaremos en este programa es: si tu flujo de trabajo es una línea recta o un árbol de decisión predefinido, usa LCEL (Módulo 2); si tu aplicación requiere que la IA 'piense', use herramientas y corrija sus propios errores en un ciclo, utiliza LangGraph (Módulo 5).

P: ¿Por qué aprender ChromaDB local si luego usaremos Pinecone en la nube?

R: Muchos estudiantes creen que las bases de datos vectoriales son intercambiables y que empezar por una local es un paso innecesario, pero la realidad de la ingeniería es distinta. ChromaDB nos permite entender la 'geometría del lenguaje' y el manejo de archivos localmente sin latencia de red ni costos de infraestructura, lo cual es ideal para la etapa de prototipado y unit testing. Una vez que entiendes las operaciones CRUD y el chunking (Módulo 3), escalar a Pinecone (Módulo 4) se vuelve un reto de arquitectura y no de conceptos básicos. En la industria, a menudo desarrollarás y testearás localmente para ahorrar costos, y solo desplegarás en soluciones cloud como Pinecone cuando necesites escalabilidad horizontal, alta disponibilidad y gestión de millones de vectores en producción.

P: ¿Es realmente necesario implementar 'Human-in-the-loop' en sistemas autónomos?

R: Existe la idea errónea de que un sistema de IA es 'mejor' cuanto más autonomía tenga. Sin embargo, en entornos corporativos de alta responsabilidad (finanzas, salud, legal), un sistema 100% autónomo es un riesgo inaceptable. El concepto de Human-in-the-loop (HITL) que vemos en el Módulo 7 no es un paso atrás, sino una

característica de seguridad avanzada. Aprenderás a diseñar puntos de interrupción donde el agente 'levanta la mano' para pedir validación humana ante una duda o una acción crítica. Piensa en ello como el sistema de piloto automático de un avión: es extremadamente capaz, pero está diseñado para que el piloto humano supervise y tome el control en momentos clave. Esto es lo que diferencia a un proyecto escolar de una arquitectura de grado profesional.

P: ¿Cómo puedo evaluar si mis optimizaciones de RAG están funcionando realmente?

R: Muchos desarrolladores se frustran porque ajustan sus prompts o el tamaño de sus chunks 'por intuición', viendo si la respuesta les 'parece buena'. En este programa, eliminamos la subjetividad mediante métricas de evaluación automática y observabilidad (Módulo 4 y 7). Aprenderás a medir el 'Recall' (¿encontró el sistema la información relevante?) y la 'Precisión' (¿lo que encontró era realmente necesario?). Utilizaremos herramientas como Arize Phoenix y LangSmith para trazar cada paso de la cadena. Si no puedes medirlo, no puedes optimizarlo. El enfoque de AI Architect consiste en tratar la evaluación de la IA como si fueran tests unitarios en el software tradicional: si cambias un parámetro, debes correr un set de evaluación para confirmar que el sistema no ha empezado a alucinar.

P: ¿Qué tan avanzado debe ser mi nivel de Python para no perderme en el curso?

R: El nivel solicitado es INTERMEDIO. Esto no significa que debas ser un experto en algoritmos complejos, sino que debes sentirte cómodo con la programación orientada a objetos (clases y herencia), decoradores y tipado básico. Lo más importante es que estés dispuesto a adoptar el paradigma funcional de LangChain y la lógica asíncrona de Python 3.12. Muchos desarrolladores seniors de otros lenguajes se frustran con la IA porque intentan escribir Python como si fuera Java o C#. Aquí aprenderás a escribir Python 'idiomático' para IA, aprovechando las librerías modernas. Si conoces cómo estructurar un proyecto con módulos y entiendes qué es una promesa o un 'future', tienes la base necesaria para escalar con nosotros hacia arquitecturas complejas.

Conclusiones

De Scripts Lineales a Arquitecturas de Agentes Autónomos

Has trascendido la fase de prototipado básico para dominar el diseño de sistemas inteligentes circulares y modulares. La transición desde el Módulo 1 (abstracción base) hasta el Módulo 6 (multi-agentes) representa un cambio de paradigma profesional: dejas de programar flujos de texto rígidos para diseñar ecosistemas que razonan y colaboran. Este enfoque te permite abordar problemas de negocio complejos donde una sola consulta no es suficiente, estructurando soluciones a través de supervisores que delegan tareas a especialistas. Ahora puedes gestionar la incertidumbre de la IA mediante estados persistentes en LangGraph, permitiendo que tus sistemas retomen procesos complejos exactamente donde los dejaron, una habilidad crítica para aplicaciones empresariales de larga duración.

La Dualidad RAG: Escalabilidad del Conocimiento y Verdad Técnica

A través de la integración de los Módulos 3 y 4, has construido un puente entre la lógica del LLM y los datos privados de una organización. La progresión de una base vectorial local (ChromaDB) a una infraestructura en la

nube (Pinecone) no es solo un cambio de herramienta, sino una lección de escalabilidad horizontal. Has aprendido que un RAG profesional no se trata solo de 'conectar datos', sino de optimizar la recuperación mediante re-ranking e ingesta con metadatos avanzados para minimizar alucinaciones. En el mundo profesional, esta competencia te permite transformar silos de información estática en activos dinámicos que alimentan la generación de contenido con precisión quirúrgica y fundamentos sólidos.

Ingeniería de Producción: Observabilidad y el Factor Humano

El conocimiento técnico es incompleto sin la robustez operativa explorada en los Módulos 7 y 8. Has integrado una mentalidad de 'AI-Ops' que garantiza que tus sistemas no sean cajas negras. Al dominar herramientas como LangSmith y Arize Phoenix, ahora posees la capacidad de rastrear cada token, evaluar costos en tiempo real y detectar fallos de lógica antes de que afecten al usuario final. La incorporación de patrones 'Human-in-the-loop' asegura que, para decisiones críticas, tu arquitectura mantenga un equilibrio entre autonomía y supervisión humana. Este es el diferenciador que te permite presentar proyectos a niveles directivos con métricas claras de latencia, seguridad y retorno de inversión.

Capacidad de Transformación Profesional: El Arquitecto de IA de 2026

Al finalizar este programa, tu perfil ha evolucionado de un desarrollador de software tradicional a un Arquitecto de IA capaz de orquestar infraestructuras complejas. Lo que antes era un desafío de integración asíncrona y gestión de prompts, ahora es un flujo de trabajo optimizado que utiliza Python 3.12 y Redis para servir miles de peticiones simultáneas. Estás capacitado para liderar equipos técnicos en la creación de productos que van desde sistemas de soporte autónomos hasta analistas de datos multi-agente. Tu valor en el mercado actual radica en la síntesis de tres pilares que has dominado: orquestación lógica, gestión persistente de la memoria y despliegue escalable en entornos de producción real.